
Statistical Foundations of Machine Learning - Practicals Handbook

Pascal Tribel

Apr 22, 2026

CONTENTS

Welcome. In the following pages, you will find the practicals reference for the course of [INFO-F422 - Statistical Foundations of Machine Learning](#) given at the [Université Libre de Bruxelles](#).

Teaching staff

The 2024-2025 teaching staff is made of:

- [✉ Gianluca Bontempi](#): Teacher
- [✉ Cédric Simar](#): Teaching Assistant
- [✉ Pascal Tribel](#): Teaching Assistant
- [✉ Gian Marco Paldino](#): PhD Student

Reference

- [✉ Statistical Foundations of Machine Learning: the handbook](#) by **Gianluca Bontempi**, Machine Learning Group, Computer Science Department, ULB
- *Introduction*
- *Notation Sheet*
- *Introduction to Probabilistic Methods and Monte Carlo Simulations*
- *Linear Models*
- *Data Preprocessing, Local Models and Tree-Based Models*
- *Neural Networks for Regression*
- *Ensembles of models and feature selection*
- *Classification*
- *Conclusion*
- *Bibliography*

Mistakes, Remarks, or Compliments

For anything you would like to share to the team, feel free to write an e-mail to [Pascal Tribel](#).

INTRODUCTION

1.1 Welcome

Welcome to the practicals handbook of the *Statistical Foundations of Machine Learning* course, part of the Master's program in Computer Science at ULB. This course provides the essential mathematical and algorithmic foundations behind the modern techniques used in machine learning, with a strong focus on statistics, probabilistic modeling, and regression/classification methods. It is the perfect starting point for anyone aiming to understand, build, and evaluate predictive models rigorously. The present document aims at bringing additional examples, tools and exercises to help understanding the main course's contents.

1.2 Why This Book?

Machine learning is not magic. It is grounded in solid mathematical ideas, especially probability theory, statistics, linear algebra, and optimization. Understanding **how and why** algorithms work is crucial for designing good systems that are not only performant but also interpretable and reliable.

This book will:

- Build your **intuition** for probabilistic reasoning and statistical inference.
- Introduce **core regression and classification models**, from linear regression to neural networks.
- Teach you how to **implement algorithms**, visualize results, and understand performance metrics.
- Provide **practical tools and code exercises** using Python and popular libraries such as NumPy, Matplotlib, scikit-learn, and PyTorch.
- Develop your ability to **choose, analyze, and compare models**, considering trade-offs like bias/variance, overfitting, and interpretability.

Whether your goal is to become a data scientist, ML engineer, or researcher, these foundations will serve as the most important base for more advanced courses and projects.

1.3 Course Overview

As 2026 is the first year of exploitation of this handbook, chapters will be delivered practical after practicals. If you notice any mistake, typo, unclarity, please [send me and email](#) ! Here's what you can expect:

1.3.1 Introduction to Probabilistic Methods and Monte Carlo Simulations

We start with the building blocks of randomness and probability:

- Simulating data with `NumPy`
 - Visualizing distributions with `Matplotlib`
 - Understanding Monte Carlo simulations
 - Working with **multivariate Gaussians**
 - Learning about **minimization** in statistical modeling
 - Exploring **Gaussian Mixture Models**
-

1.3.2 Linear Models

From classical regression to more flexible transformations:

- Ordinary Least Squares (OLS) regression
 - Analyzing residuals and model assumptions
 - Polynomial and **Radial Basis Function (RBF)** transformations
 - Ridge regression and the role of regularization
-

1.3.3 Preprocessing, Local Models and Tree-Based Models

Preparing your data and working with interpretable machine learning models:

- Data preprocessing: normalization, handling missing data
 - **k-Nearest Neighbors (kNN)** for regression
 - **Decision trees** and **random forests**
 - Using **scikit-learn pipelines**
 - Understanding the **bias-variance tradeoff** in practice
-

1.3.4 Neural Networks for Regression

Deep learning in its most intuitive form:

- Normalization and standardization for stable learning
 - MLPs (Multilayer Perceptrons) for regression tasks
 - Cost functions and model training
 - **Matrix-based backpropagation**
 - Introduction to **CNNs**
 - Dealing with **overfitting**, **cross-validation**
 - Model persistence: saving and loading in **PyTorch**
-

1.3.5 Ensemble Models and Feature Selection

Combining models and selecting the most useful inputs:

- Error decomposition and performance metrics
 - **Filter, wrapper, and embedded** feature selection
 - Building **ensembles** of models for improved accuracy
-

1.3.6 Classification

From regression to classification, we shift the focus:

- Evaluation metrics for classification (accuracy, precision, recall, F1)
 - **Bayes classifier** and probabilistic foundations
 - **Logistic regression** and ROC curves
 - Working with **unequal costs**
 - Alternative classification models
-

1.4 Practical Exercises

This handbook serves as the practicals reference for the course. The [theoretical handbook](#) remains the main reference. We want to focus here in the implementations, the algorithms and the existing libraries for implementing the topics covered in the course. The student should refer as much as possible to the theoretical handbook before delving into the coding. Here, we try, as much as possible, to include:

- **Hands-on coding exercises**
- **Mini-projects and simulations**
- **Graphical visualizations** of models and results

- Opportunities to **experiment** with real-world datasets

Doing the exercises is **not optional** if you want to truly master the material. They are designed to:

- Reinforce theoretical concepts
 - Develop your implementation skills
 - Sharpen your critical thinking about model performance and assumptions
-

1.5 Final Notes

This is a **theory-meets-practice** course. You'll build up solid statistical thinking and programming fluency, while also learning to critically assess and apply machine learning models. Every part of the course prepares you for future work in AI, Data Science, or scientific computing.

Take it seriously. Be curious. Challenge your understanding. And above all: have fun exploring the possibilities of statistical thinking in machine learning.

Let's begin!

NOTATION SHEET

In the handbook, we follow the notation sheet provided in the [theoretical handbook](#). Generally speaking, we denote

- Random Variables (r.v.) as **boldface** lowercase letters: $\mathbf{x}$, $\mathbf{y}$, $\mathbf{z}$;
- The probability of realisation of such a random variable as $\text{Prob}(\mathbf{x} = x)$, or, shorter, $p(x)$ when the random variable is clear;
- Numbers or vectors, such as a, b, c, x, y, z , as numbers or vectors;
- Matrices, such as A, X, Y , as uppercase letters, with in particular I being the identity matrix;
- Sets, as uppercase italic letters;
- Parameters, such as α, β , as greek letters, and uppercase greek letters, as Θ , as sets of parameters;
- Estimators, as $\hat{\theta}$, with the *hat* notation;
- Transposes of vectors and matrices as a^T or A^T ;

Combinations of such notations may occur as well: as an example, vectors of random parameters are given by uppercase boldface greek letters: $\mathbf{\Theta}$.

INTRODUCTION TO PROBABILISTIC METHODS AND MONTE CARLO SIMULATIONS

Probabilistic methods and Monte Carlo simulations represent fundamental techniques in computational mathematics, offering powerful approaches to solve complex problems that would be analytically intractable. These methods have found widespread applications across diverse domains including statistical physics, financial mathematics, quantum mechanics, machine learning, and computational biology. The course of INFO-F305 - *Modélisation et Simulation* proposes a good introduction to Monte Carlo simulations. Exercises are available in the [practical handbook](#) of this course.

This chapter introduces the fundamental concepts, tools, and techniques of probabilistic methods with a particular focus on Monte Carlo simulations. We will explore how to generate random variables from different distributions, estimate integrals, solve optimization problems, and analyze statistical properties of random processes. Throughout the chapter, we will emphasize both the theoretical foundations and practical implementations using Python and its scientific computing ecosystem.

Note, the proper understanding of this set of methods is extremely important for having a good approach of many modern Machine Learning problems. Real-life data collection can indeed, in many cases, be modeled by a Monte Carlo simulation, and the statistical/probabilistic study of the simulated problem often offers valuable insights about the real-life dynamics.

By the end of this chapter, you will have developed a solid understanding of how to apply Monte Carlo techniques to solve a variety of mathematical and statistical problems, setting the foundation for more advanced applications in subsequent chapters.

3.1 Necessary Libraries

To run this practical work, the following libraries must be installed:

- NumPy: Vector and matrix manipulation, random number generation,
- SciPy: For advanced numerical statistical methods,
- Pandas: For advanced data manipulation
- Matplotlib, seaborn: For data visualization,
- tqdm: For beautiful loading bars.

If the libraries are already installed, you can skip this step.

If you are using Jupyter Notebook, you can install the dependencies directly by running the cell below:

```
#!pip install numpy matplotlib tqdm pandas seaborn numpyarray_to_latex
```

This `pprint` function will allow to render the output of some codes in LaTeX, such as the vectors and the matrices. Remark that calling this function only works in Jupyter. If you code in plain Python, you should rather use the default `str` and `print` functions.

We make some imports and define a `pprint` function.

```
import numpy as np
from IPython.display import display, Math
from numpyarray_to_latex.jupyter import to_ltx
def pprint(*args):
    res = ""
    for i in args:
        if type(i) == np.ndarray:
            res += to_ltx(i, brackets='[]')
        elif type(i) == str:
            res += i
    display(Math(res))
```

3.2 Introduction to NumPy

NumPy is an essential library for scientific computations in Python. It offers:

- `ndarray` objects to represent multi-dimensional arrays.
- A collection of mathematical functions to perform fast operations on these arrays.

3.2.1 Manipulation of NumPy Arrays

Here are some examples for creating and manipulating NumPy arrays:

```
v0 = np.array([1, 2, 3])
v1 = np.arange(0, 3, 0.4)
v2 = np.linspace(0, 5, 10)
v3 = np.linspace(0, 2, 10)
M = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
pprint("v_0=", v0)
pprint("v_1=", v1)
pprint("v_2=", v2)
pprint("v_3=", v3)
pprint("M=", M)
```

$$v_0 = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \end{bmatrix}$$

$$v_1 = \begin{bmatrix} 0.0000 \\ 0.4000 \\ 0.8000 \\ 1.2000 \\ 1.6000 \\ 2.0000 \\ 2.4000 \\ 2.8000 \end{bmatrix}$$

$$v_2 = \begin{bmatrix} 0.0000 \\ 0.5556 \\ 1.1111 \\ 1.6667 \\ 2.2222 \\ 2.7778 \\ 3.3333 \\ 3.8889 \\ 4.4444 \\ 5.0000 \end{bmatrix}$$

$$v_3 = \begin{bmatrix} 0.0000 \\ 0.2222 \\ 0.4444 \\ 0.6667 \\ 0.8889 \\ 1.1111 \\ 1.3333 \\ 1.5556 \\ 1.7778 \\ 2.0000 \end{bmatrix}$$

$$M = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \\ 7.0000 \\ 8.0000 \\ 9.0000 \end{bmatrix}$$

3.2.2 Basic Operations

Remark, those functions (and basically, most of the NumPy functions) are *vectorial*. This means that they are applied in parallel on all the components of the vectors. Later on, when we will present the PyTorch library, we will show how this parallelisation can benefit from using specialized hardware, such as the GPU's.

```
s = np.sum(v0)
p = np.dot(v2, v3)
print("Sum of v0 components:", s)
print("Scalar product of v2 and v3:", p)
```

```
Sum of v0 components: 6
Scalar product of v2 and v3: 35.18518518518518
```

We encourage you to consult the official [NumPy documentation](#) to explore its many features. A larger introduction to NumPy and Matplotlib is available in the 5th practical of the course INFO-F305 - *Modélisation et Simulation*.

3.3 Probabilistic Foundations

The course of MATH-F315 - *Probabilités et Statistiques* has presented all the necessary materials used in this chapter. This is let as reference for the formal demonstrations of the tools and theorems used here. However, we propose a small review of the most important notions that will help you understand the exercises at the end of the chapter. Remember: the exercises are not meant to make you familiar with one specific practical implementation of the techniques and tools, they are intended to make you apply some well understood and completely digested statistical and probabilistic concepts. We strongly recommend the reader to delve in depth in the theoretical concepts related to the exercises.

3.3.1 Expectation and Variance

The expectation of a random variable $\mathbf{x}$, denoted $E[\mathbf{x}]$, is defined as:

$$E[\mathbf{x}] = \begin{cases} \int_{-\infty}^{\infty} x f_{\mathbf{x}}(x) dx & \text{for continuous } \mathbf{x} \\ \sum_x x p_{\mathbf{x}}(x) & \text{for discrete } \mathbf{x} \end{cases}$$

The variance of $\mathbf{x}$, denoted $\text{Var}(\mathbf{x})$, measures the dispersion around the mean:

$$\text{Var}(\mathbf{x}) = E[(\mathbf{x} - E[\mathbf{x}])^2] = E[\mathbf{x}^2] - (E[\mathbf{x}])^2$$

The variance is the square of the standard deviation.

For two random variables $\mathbf{x}$ and $\mathbf{y}$, their covariance is defined as:

$$\text{Cov}(\mathbf{x}, \mathbf{y}) = E[(\mathbf{x} - E[\mathbf{x}])(\mathbf{y} - E[\mathbf{y}])] = E[\mathbf{xy}] - E[\mathbf{x}]E[\mathbf{y}]$$

The correlation coefficient, which measures the linear dependence between $\mathbf{x}$ and $\mathbf{y}$, is:

$$\rho_{\mathbf{x}, \mathbf{y}} = \frac{\text{Cov}(\mathbf{x}, \mathbf{y})}{\sqrt{\text{Var}(\mathbf{x}) \cdot \text{Var}(\mathbf{y})}}$$

3.3.2 Common Probability Distributions

Uniform Distribution

A random variable $\mathbf{x}$ follows a uniform distribution on the interval $[a, b]$, denoted $\mathbf{x} \sim \mathcal{U}(a, b)$, if its PDF is:

$$f_{\mathbf{x}}(x) = \begin{cases} \frac{1}{b-a} & \text{if } x \in [a, b] \\ 0 & \text{otherwise} \end{cases}$$

The mean and variance are $E[\mathbf{x}] = \frac{a+b}{2}$ and $\text{Var}(\mathbf{x}) = \frac{(b-a)^2}{12}$, respectively. In NumPy, the normal distribution can be sampled with the `numpy.random.normal` function.

Normal (Gaussian) Distribution

A random variable $\mathbf{x}$ follows a normal distribution with parameters μ and σ^2 , denoted $\mathbf{X} \sim \mathcal{N}(\mu, \sigma^2)$, if its PDF is:

$$f_{\mathbf{x}}(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

where μ is the mean and σ^2 is the variance. In NumPy, the uniform distribution can be sampled with the `numpy.random.normal` function.

3.4 Introduction to the `numpy.random` Module

The NumPy library includes a `numpy.random` submodule dedicated to random number generation. This module offers:

- Random number generators for various distributions (uniform, normal, binomial, etc.).
- Tools for drawing samples and manipulating random sequences.

3.4.1 Reproducibility of Results

To ensure the reproducibility of simulations, it is crucial to set a random seed.

```
np.random.seed(42)
x = np.random.rand(5)
pprint("x=", x)
np.random.seed(43)
x = np.random.rand(5)
pprint("x=", x)
np.random.seed(42)
x = np.random.rand(5)
pprint("x=", x)
print("# Same as the first !")
```

$$x = \begin{bmatrix} 0.3745 \\ 0.9507 \\ 0.7320 \\ 0.5987 \\ 0.1560 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.1151 \\ 0.6091 \\ 0.1334 \\ 0.2406 \\ 0.3271 \end{bmatrix}$$

$$x = \begin{bmatrix} 0.3745 \\ 0.9507 \\ 0.7320 \\ 0.5987 \\ 0.1560 \end{bmatrix}$$

```
# Same as the first !
```

3.4.2 Available Distributions

Here are some common distributions:

- `numpy.random.uniform(low, high, size)` : Uniform distribution.
- `numpy.random.normal(loc, scale, size)` : Normal (Gaussian) distribution.
- `numpy.random.binomial(n, p, size)` : Binomial distribution.

3.4.3 Sampling

Sampling is the process of randomly selecting elements from a population. For example:

```
population = np.array([1, 2, 3, 4, 5])
s = np.random.choice(population, size=(3, 3), replace=True)
pprint("s=", s)
```

$$s = \begin{bmatrix} 3.0000 \\ 3.0000 \\ 3.0000 \\ 5.0000 \\ 4.0000 \\ 3.0000 \\ 5.0000 \\ 2.0000 \\ 4.0000 \end{bmatrix}$$

3.4.4 Statistical Analysis

NumPy implements a huge diversity of estimators for the different tools that have been presented in the probability introduction. As an example, the sample average is an unbiased estimator of the expected value. In NumPy, the sample average of an array can be computed with the `numpy.mean` function. The sample variance of an array can be computed by squaring the result of the `numpy.std` function. The covariance between two vectors can be computed with the `numpy.cov` function. Finally, the correlation coefficient between two vectors can be computed with the `numpy.corrcoef` function.

Moreover, SciPy offers tools to analyse statistical distributions. As an example, if wan to get density, repartition function and quantiles of a normal distribution $\mathcal{N}(0, 1)$:

```
import scipy as sp
```

The probability density function of a normal distribution is given by

$$f(x) = \frac{\left(\frac{e^{-\frac{(x-\mu)^2}{2\sigma^2}}}{\sqrt{2\pi}} \right)}{\sigma}$$

for μ and σ the mean and the standard deviation of the distribution.

```
mu, sigma = 0, 1
print(sp.stats.norm.pdf(1, loc=mu, scale=sigma))
```

```
0.24197072451914337
```

The quantiles can be obtained using the `ppf` function:

```
print(sp.stats.norm.ppf(0.75, loc=mu, scale=sigma))
```

```
0.6744897501960817
```

Finally, the cumulative distribution function can be obtained with `cdf`:

```
print(sp.stats.norm.cdf(0.5, loc=mu, scale=sigma))
```

```
0.6914624612740131
```

3.4.5 Exercise: Histogram - Probability Density Function of a $\mathcal{N}(0, 1)$ vs Histogram

Using the previous `pdf` function, plot the histogram of a sampled normal $\mathcal{N}(0, 1)$ and check that it behaves as the analytical probability density function. Show on the same graph the three quarter quantiles (0.25, 0.5, 0.75).

Solution

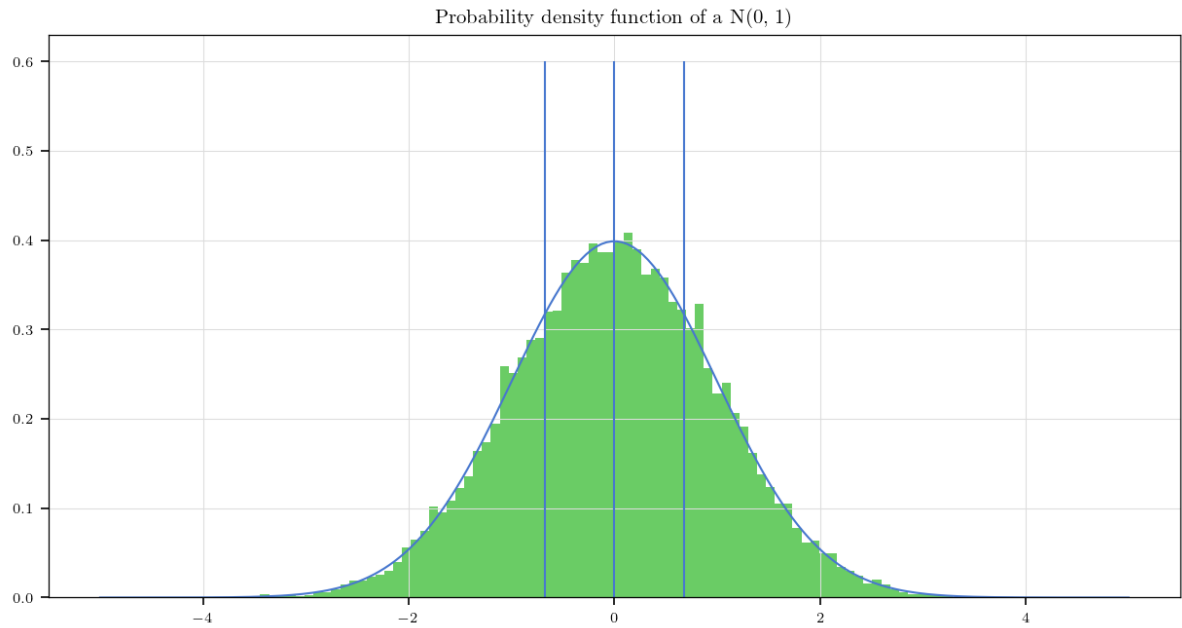
In the following code, we sample a normal $\mathcal{N}(0, 1)$, and, using `Matplotlib`, we display the histogram of the sampled values, along the probability density function provided by `Scipy`. We also display the three first quarters using the `scipy.stats.norm.ppf` function.

```
import matplotlib.pyplot as plt
plt.style.use(['seaborn-v0_8-muted', 'practicals.mplstyle'])
pprint("s=", sp.stats.norm.ppf(np.array([0.25, 0.5, 0.75]), loc=mu, scale=sigma))
```

$$s = \begin{bmatrix} -0.6745 \\ 0.0000 \\ 0.6745 \end{bmatrix}$$

Try changing the number of bins. What happens if it is too low ?

```
n_bins = 100
x = np.linspace(-5, 5, 200)
plt.figure(figsize=(10, 5))
plt.plot(x, sp.stats.norm.pdf(x, loc=mu, scale=sigma))
plt.hist(np.random.normal(size=(25000,)), bins=n_bins, density=True)
for q in sp.stats.norm.ppf(np.array([0.25, 0.5, 0.75]), loc=mu, scale=sigma):
    plt.vlines(q, 0, 0.6)
plt.grid("on")
plt.title("Probability density function of a  $\mathcal{N}(0, 1)$ ")
plt.show()
```



3.5 Multivariate Gaussian Distributions

Multivariate Gaussian distributions are multidimensional generalizations of the normal distribution (that you already know well). They can be described by mean vector μ and a covariance matrix Σ , which shows the covariance between each of the distribution's dimensions. As an example, if this matrix is diagonal, the variables corresponding to each dimension are independent. In the same way, this matrix has to be symmetric (can you tell why?). Numpy proposes the function `random.multivariate_normal` that allows the sampling of a multivariate Gaussian, given its mean vector and covariance matrix.

3.5.1 Exercise

Sample 750 points following a Multivariate Gaussian Distribution given

$$\mu = [1, -\frac{1}{2}, 0]$$

$$\Sigma = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Then, using `scatter` from `matplotlib`, plot the points under each pairs of dimensions.

Solution

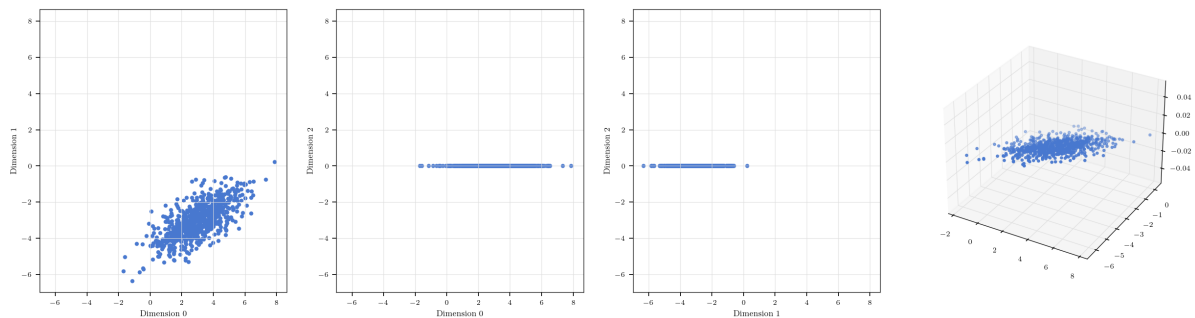
This exercise is important: understanding how the values in the mean vector and covariance matrix influence the distribution of the sampled points might be a difficult task, at first. Play with the values: how does changing the values in the mean vector affect the cloud of points ? What happens if you set values of the covariance matrix to 0 ? Under which circumstances the covariance matrix is valid ? Why ? Take the necessary time for grasping every subtlety of this way of generating data. You will require this intuition in many real-life scenarios.

```
mu = np.array([3, -3, 0])
sigma = np.array([[2, 1, 0],
                  [1, 1, 0],
                  [0, 0, 0]])

n_points = 750

points = np.random.multivariate_normal(mu, sigma, n_points)
```

```
fig, ax = plt.subplots(1, 4, figsize=(20, 5))
ax[0].scatter(points[:, 0], points[:, 1])
ax[0].set_xlabel("Dimension 0")
ax[0].set_ylabel("Dimension 1")
ax[1].scatter(points[:, 0], points[:, 2])
ax[1].set_xlabel("Dimension 0")
ax[1].set_ylabel("Dimension 2")
ax[2].scatter(points[:, 1], points[:, 2])
ax[2].set_xlabel("Dimension 1")
ax[2].set_ylabel("Dimension 2")
for i in range(3):
    ax[i].grid(True)
    ax[i].set_xlim((1.1*np.min(points), 1.1*np.max(points)))
    ax[i].set_ylim((1.1*np.min(points), 1.1*np.max(points)))
ax[3].remove()
ax[3] = fig.add_subplot(1, 4, 4, projection='3d')
ax[3].scatter(points[:, 0], points[:, 1], points[:, 2], marker='.')
plt.show()
```



3.6 Monte Carlo Simulation

The core principle of Monte Carlo methods lies in using random sampling to approximate numerical results. Rather than attempting to solve problems through analytical formulas, Monte Carlo techniques leverage the law of large numbers to estimate solutions through repeated random sampling. This approach is particularly valuable when dealing with high-dimensional spaces, complex geometries, or systems with multiple degrees of freedom. The simplest visual example, to understand the notion of Monte Carlo, is the following. You want to measure the surface of a lake. Its shape is complex, difficult to measure and to compute the area it covers. However, you can easily surround it by a square for which the side can accurately be measured. You then decide to randomly throw stones with a catapult on the square. The proportion of number of stones that fall into the lake compared to the total number of thrown stones (if the number of stones is infinite) is the same proportion as the proportion of the surface of the lake compared to the square surface. The more stone you throw, the more accurate the estimation is. This idea is close to the one used for Monte Carlo integration.

3.6.1 Monte Carlo Integration

Consider the problem of evaluating the integral:

$$I = \int_{\Omega} f(x) dx$$

for which the analytical solution is not available.

The Monte Carlo estimate of this integral is:

$$\hat{I}_n = \frac{|\Omega|}{n} \sum_{i=1}^n f(x_i)$$

where $x_1, x_2, \dots, x_n$ are i.i.d. random variables uniformly distributed over Ω , and $|\Omega|$ is the measure (volume) of Ω . For a definite integral over $[a, b]$, the Monte Carlo estimate is:

$$\int_a^b f(x) dx \approx (b - a) \cdot \frac{1}{n} \sum_{i=1}^n f(x_i)$$

where $x_i \sim \mathcal{U}(a, b)$.

Think of it like this: you throw a bunch of points in a certain subspace whose size is known, and you determine which proportion of those points falls under the curve of the function. Remember, this requires the function to be efficiently computable (which is often the case, even when an analytical solution for its integral is unavailable).

3.6.2 Exercise: Integral Estimation

Implement a Monte Carlo simulation to estimate the integral of the function $f(x) = x^2$ over the interval $[0, 1]$.

Solution

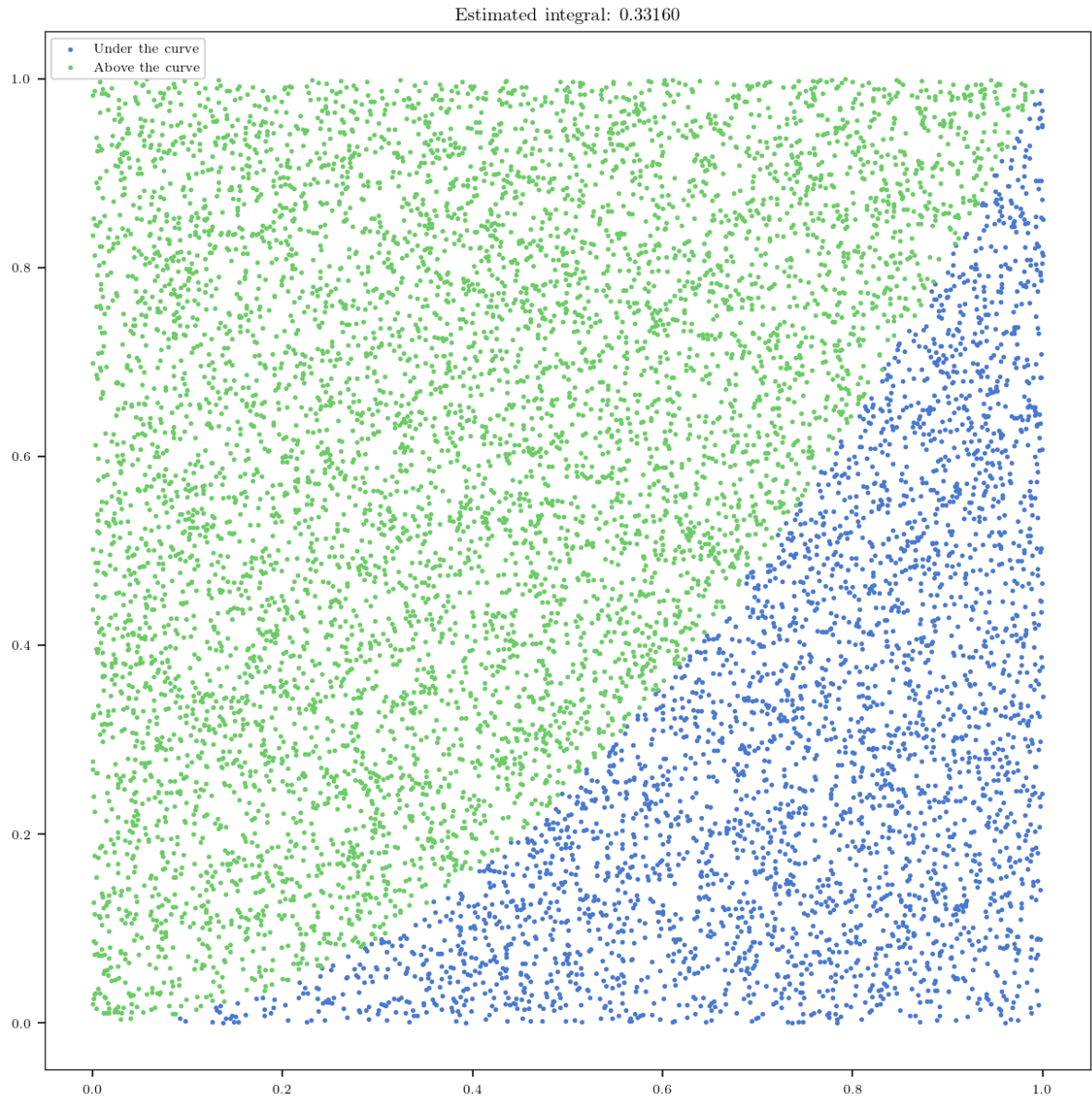
The indefinite integral is given by

$$\int x^2 dx = \frac{x^3}{3}$$

and therefore the definite interval is

$$\int_0^1 x^2 dx = \frac{1^3}{3} - \frac{0^3}{3} = \frac{1}{3}$$

```
n_points = 10000
approximations = []
points = np.random.rand(n_points, 2)
under_curve = np.array(points[:, 1]) <= np.array(points[:, 0])**2
estimation = np.sum(under_curve) / n_points
plt.figure(figsize=(10, 10))
plt.scatter(points[under_curve, 0], points[under_curve, 1], marker='.', label='Under_
↳the curve')
plt.scatter(points[~under_curve, 0], points[~under_curve, 1], marker='.', label=
↳'Above the curve')
plt.title(f'Estimated integral: {estimation:.5f}')
plt.legend(loc="upper left")
plt.show()
```



3.6.3 Exercise: Probability Estimation

Use a normal distribution to model a random variable, with parameters $\mu = 0$ and $\sigma^2 = \frac{1}{2}$ and calculate the probability that it takes a value in the interval $[-0.25, 0.25]$.

Don't skip this part! Rather, take a sheet of paper, and understand properly every step. You should be able to redo this computation yourself without the solution. Try also changing the values: don't worry, then, you will be able to validate your theoretical answer by the simulation.

Solution

Let us first show how to analytically compute the requested probability. The probability that X lies within the interval is given by:

$$P(-0.25 \leq \mathbf{x} \leq 0.25) = \int_{-0.25}^{0.25} f_{\mathbf{x}}(x) dx$$

where the Probability Density Function $f_{\mathbf{x}}(x)$ is:

$$f_{\mathbf{x}}(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

In our case, $\mu = 0$ and $\sigma^2 = 1/2$, so:

$$f_{\mathbf{x}}(x) = \frac{1}{\sqrt{\pi}} e^{-2x^2}$$

To simplify the calculation, we convert $\mathbf{x}$ to the standard normal variable $\mathbf{z}$:

$$\mathbf{z} = \frac{\mathbf{x} - \mu}{\sigma}, \quad \text{where } \sigma = \sqrt{1/2} = \frac{\sqrt{2}}{2}$$

For the bounds of the interval:

$$z_{-0.25} = \frac{-0.25 - 0}{\sigma} = -0.25 \cdot \frac{2}{\sqrt{2}} = -\frac{\sqrt{2}}{4}, \quad z_{0.25} = \frac{0.25 - 0}{\sigma} = \frac{\sqrt{2}}{4}$$

The probability is then expressed using the repartition function $\Phi(z)$ of the standard normal distribution:

$$P(-0.25 \leq \mathbf{x} \leq 0.25) = P\left(-\frac{\sqrt{2}}{4} \leq \mathbf{z} \leq \frac{\sqrt{2}}{4}\right) = \Phi\left(\frac{\sqrt{2}}{4}\right) - \Phi\left(-\frac{\sqrt{2}}{4}\right)$$

Using the symmetry of the standard normal distribution:

$$\Phi(-z) = 1 - \Phi(z)$$

This simplifies to:

$$P(-0.25 \leq \mathbf{x} \leq 0.25) = 2 \cdot \Phi\left(\frac{\sqrt{2}}{4}\right) - 1$$

To find $\Phi\left(\frac{\sqrt{2}}{4}\right)$, we can use values in tables. It yields:

$$\frac{\sqrt{2}}{4} \approx 0.3536$$

From standard normal tables or software (we can use `SciPy`):

```
import scipy
scipy.stats.norm.cdf((2**0.5)/4)
```

```
np.float64(0.6381631950841185)
```

$$\Phi(0.3536) \approx 0.6388$$

Thus:

$$P(-0.25 \leq \mathbf{x} \leq 0.25)$$

is given by

```
2*scipy.stats.norm.cdf((2**0.5)/4)-1
```

```
np.float64(0.2763263901682369)
```

The Monte Carlo simulation yields a pretty close value:

```
n_points = 100000
approximations = []
points = np.random.normal(0, 0.5**0.5, size=(n_points,))
in_interval = (np.abs(np.array(points)) <= 0.25)
estimation = np.sum(in_interval) / n_points
print(estimation)
```

```
0.27714
```

3.6.4 Exercise: Monte Carlo Approximation of Parameters

Use Monte Carlo simulation to approximate numerically the mean of a Normal r.v. $\mathbf{z} \sim N(\mu, \sigma^2)$ and of the variable $\mathbf{y} = \mathbf{z}^2$. Verify that MC returns a good approximation of the analytical result. As a hint, note that $\text{Var}[\mathbf{z}] = E[\mathbf{z}^2] - E[\mathbf{z}]^2$

Solution

First, analytically, we can note that, if μ is the expected value of $\mathbf{z}$, and σ^2 is its variance, then the expected value of $\mathbf{y} = \mathbf{z}^2$ is given by

$$E[\mathbf{z}^2] = \text{Var}[\mathbf{z}] + (E[\mathbf{z}])^2 = \sigma^2 + \mu^2$$

Secondly, the variance of $\mathbf{y} = \mathbf{z}^2$ is given by

$$\text{Var}[\mathbf{z}^2] = E[\mathbf{z}^4] - (E[\mathbf{z}^2])^2$$

The second term is already known ($E[\mathbf{z}^2] = \sigma^2 + \mu^2$). The first term is the fourth moment (for the development, see the notion *moment generating function* in the course of *Probabilités et Statistiques*) of a normal variable, i.e.:

$$E[\mathbf{z}^4] = 3\sigma^4 + 6\sigma^2\mu^2 + \mu^4$$

If we study a concrete example, let us fix $\mu = \frac{1}{2}$ and $\sigma^2 = 1.7$:

```
R = 100000
Ez = 0.5
Vz = 1.7
z = np.random.normal(Ez, np.sqrt(Vz), R)
y = z**2
pprint(f"E[\mathbf{z}]=\{Ez\}; \ MC\ E[\mathbf{z}]=\{np.mean(z) :.4f\}")
pprint(f"Var[\mathbf{z}]=\{Vz\}; \ MC\ Var[\mathbf{z}]=\{np.var(z) :.4f\}")
pprint(f"E[\mathbf{z}^2]=\{Vz+Ez**2\}; \ MC\ E[\mathbf{y}=\mathbf{z}^2]=\{np.mean(y) :.4f\}
↵")
pprint(f"Var[\mathbf{z}^2]=\{(3*Vz**2+6*Vz*Ez**2+Ez**4) - (Vz+Ez**2)**2 :.4f\}; \ MC\
↵Var[\mathbf{y}=\mathbf{z}^2]=\{np.std(y)**2 :.4f\}")
```

$$E[\mathbf{z}] = 0.5; \text{ MC } E[\mathbf{z}] = 0.5013$$

$$\text{Var}[\mathbf{z}] = 1.7; \text{MC Var}[\mathbf{z}] = 1.6934$$

$$E[\mathbf{z}^2] = 1.95; \text{MC } E[\mathbf{y} = \mathbf{z}^2] = 1.9448$$

$$\text{Var}[\mathbf{z}^2] = 7.4800; \text{MC Var}[\mathbf{y} = \mathbf{z}^2] = 7.4767$$

3.6.5 Exercise: Absolute Value

Use MC to approximate numerically $E[\mathbf{k}]$ and $\text{Var}[\mathbf{k}]$ where $\mathbf{k} = |\mathbf{z}|$ and check that the approximation is good w.r.t. the analytical results in [this article](#).

Solution

The process is truly similar to the previous one.

```
Ez = 0
Vz = 1
z = np.random.normal(Ez, np.sqrt(Vz), R)
k = np.abs(z)

Ek = np.sqrt(2/np.pi)
Vk = 1 - 2/np.pi

pprint(f"E[\mathbf{k}] = {Ek:.4f}; \ MC\ E[\mathbf{k}=|\mathbf{z}|] = {np.mean(k):.4f}
↪")
pprint(f"Var[\mathbf{k}] = {Vk:.4f}; \ MC\ Var[\mathbf{k}=|\mathbf{z}|] = {np.var(k):.4f}
↪")
```

$$E[\mathbf{k}] = 0.7979; \text{MC } E[\mathbf{k} = |\mathbf{z}|] = 0.7992$$

$$\text{Var}[\mathbf{k}] = 0.3634; \text{MC Var}[\mathbf{k} = |\mathbf{z}|] = 0.3634$$

3.6.6 Exercise: Expected Value for any Function

Use MC to approximate numerically the value of $E[f(\mathbf{x})]$ and $f(E[\mathbf{x}])$ for a given deterministic function f , where $\mathbf{x}$ is a normal random variable.

Solution

The following script show the MC process to estimate the expected value of any Python function.

Change the function f . Is the process valid in all the cases ? Try to find an example of function for which this process could fail. Hint: some functions might not be defined depending on their input.

```

np.random.seed(0)
R = 10000

def f(x):
    return -x**2

FX = []
X = []

a = 0
b = 1

for r in range(R):
    x = np.random.normal(a, b)
    FX.append(f(x))
    X.append(x)

muX = np.mean(X)

pprint(f"E[\mathbf{x}] = {np.mean(FX) :.4f}")
pprint(f"f(E[\mathbf{x}]) = {f(muX) :.4f}")

```

$$E[f(\mathbf{x})] = -0.9756$$

$$f(E[\mathbf{x}]) = -0.0003$$

3.6.7 Exercise: Variance of Uniform Random Variable

Use MC to approximate numerically the value of the variance $\text{Var}[\mathbf{z}]$ where $\mathbf{z} \sim U(a, b)$ is uniformly distributed. Check that the result is a good approximation of the analytical value.

Solution

The following script estimates the variance of an uniformly distributed variable by Monte Carlo. The distribution is uniform between -1 and 1 .

```

R = 50000
a = -1.0
b = 1.0
muz = (b + a) / 2

Z = np.zeros(R)
for r in range(R):
    z = np.random.uniform(a, b)
    Z[r] = (z - muz)**2

pprint(f"Var[\mathbf{z}] = {(b-a)**2/12 :.4f};\ MC\ Var[\mathbf{z}] = {np.mean(Z) :.4f}")

```

$$\text{Var}[\mathbf{z}] = 0.3333; \text{MC Var}[\mathbf{z}] = 0.3340$$

3.6.8 Exercise: Covariance of a Multiple of a Random Variable

Use MC to approximate the value of the covariance of $\mathbf{x}$ and $\mathbf{y} = K\mathbf{x}$ and verify that the result is a good approximation of the analytical derivation. Check for different distributions of $\mathbf{x}$. As a hint, note that

$$\begin{aligned}\text{Cov}(\mathbf{x}, \mathbf{y}) &= E[\mathbf{xy}] - KE[\mathbf{x}]E[\mathbf{x}] \\ &= E[K\mathbf{x}^2] - K(E[\mathbf{x}])^2 \\ &= K(\text{Var}(\mathbf{x}) + E[\mathbf{x}]^2) - K(E[\mathbf{x}])^2 \\ &= K\text{Var}(\mathbf{x})\end{aligned}\tag{3.1}$$

Solution

The following code shows the covariance estimation of a sampled random variable with a multiple of this random variable. The estimated covariance is close to the analytical value.

```
R = 50000
distr = "uniform"
XY = []
X = []
K = 2
for r in range(R):
    if distr == "uniform":
        a, b = 1, 20
        x = np.random.uniform(a, b)
        VX = 1/12 * (b-a)**2
    else:
        mu, sigma = 1, 1
        x = np.random.normal(mu, sigma**2)
    y = K * x
    XY.append(x * y)
    X.append(x)
print(f"Analytical covariance: {K*VX}")
print(f"Monte Carlo covariance: {np.mean(XY) - K*np.mean(X)**2}")
```

```
Analytical covariance: 60.166666666666664
Monte Carlo covariance: 60.455461621858774
```

3.7 Monte Carlo and Multivariate Gaussian Distributions

3.7.1 Exercise: 2D Monte Carlo Simulation

Use MC simulation approximate the mean and covariance of 2-dimensional Gaussian random data with the following parameters:

- Mean vector: $[2, 3]$
- Covariance matrix: $\begin{bmatrix} 2 & 0.8 \\ 0.8 & 1 \end{bmatrix}$

Visualize the used data as a scatter plot. Compare the sample mean and the given mean vector. Hint: use `scipy.stats.multivariate_normal` for generation and `numpy` to calculate the sample mean, and use `numpy.cov` for covariance estimation. Ensure you understand the difference between population and sample covariance.

Solution

The `rvs` from `scipy.stats.multivariate_normal` (for *random variable sample* allows to sample a bivariate Gaussian adequately defined. The following code shows how to sample such kind of distribution, how to display it in Matplotlib, and how to estimate the covariance matrix.

```
from scipy.stats import multivariate_normal as mvn

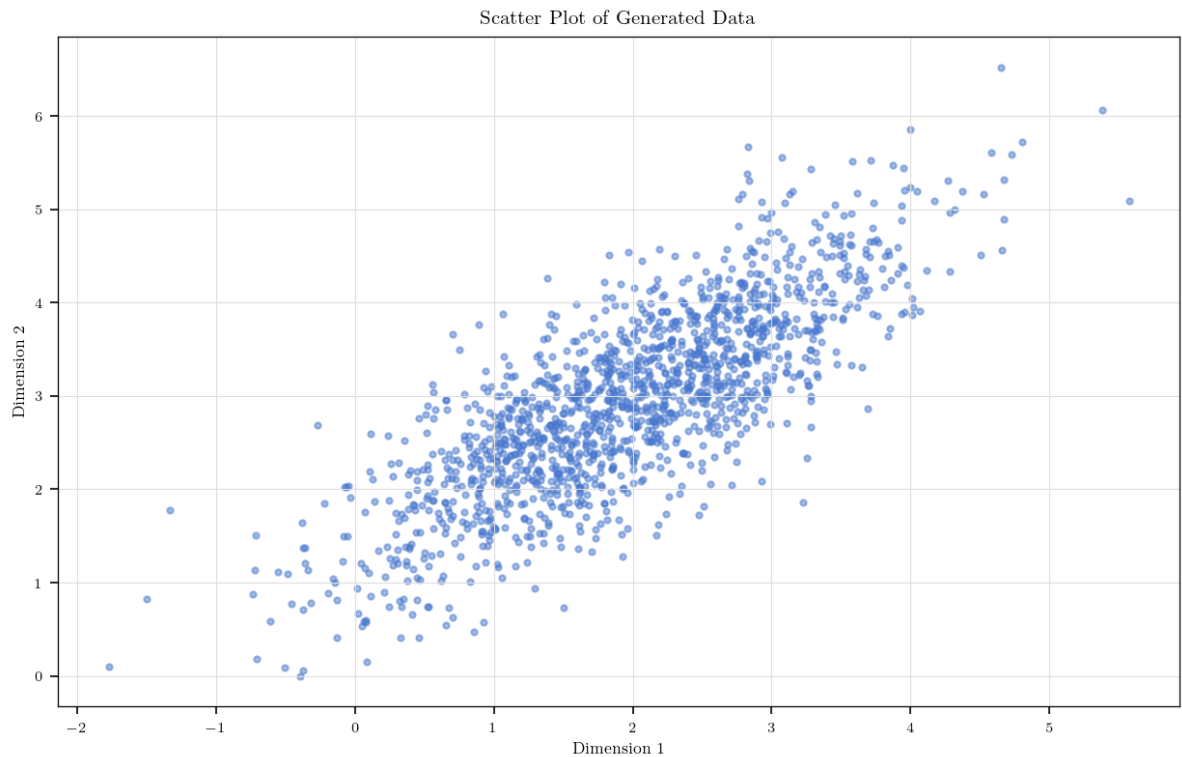
mean = np.array([2, 3])
cov = np.array([[1, 0.8], [0.8, 1]])
n_samples = 1500

data = mvn(mean=mean, cov=cov).rvs(size=n_samples)

sample_mean = np.mean(data, axis=0)
plt.figure(figsize=(10, 6))
plt.scatter(data[:, 0], data[:, 1], alpha=0.5)
plt.title("Scatter Plot of Generated Data")
plt.xlabel("Dimension 1")
plt.ylabel("Dimension 2")
plt.grid("on")
plt.show()

sample_cov = np.cov(data, rowvar=False)

pprint("\\mu=", mean)
pprint("\\hat{\\mu}=", sample_mean)
pprint("Cov:", np.array(cov))
pprint("MC\\ Cov:", sample_cov)
```



$$\mu = \begin{bmatrix} 2.0000 \\ 3.0000 \end{bmatrix}$$

$$\hat{\mu} = \begin{bmatrix} 1.9620 \\ 2.9493 \end{bmatrix}$$

$$Cov : \begin{bmatrix} 1.0000 \\ 0.8000 \\ 0.8000 \\ 1.0000 \end{bmatrix}$$

$$MC Cov : \begin{bmatrix} 0.9969 \\ 0.7940 \\ 0.7940 \\ 0.9862 \end{bmatrix}$$

3.7.2 Exercise:

We define a bivariate gaussian distribution $(\mathbf{z}_1, \mathbf{z}_2)$ with the following parameters:

- Mean vector: $[0.25, 1.0]$
- Covariance matrix: $\begin{bmatrix} 1.0 & 0.5 \\ 0.5 & 1.0 \end{bmatrix}$

Plot the regressions of $\mathbf{z}_1 | \mathbf{z}_2$ and $\mathbf{z}_2 | \mathbf{z}_1$. What is their intersection ? Hint: those are given by expressing $\mathbf{z}_1$ in terms of $\mathbf{z}_2$ (or the opposite): $\mathbf{z}_1 = a_1 + b_1 \mathbf{z}_2$.

Solution

For the following derivation, we redirect the student to the theoretical handbook, Section 2.15.1 (Bivariate normal distribution). The following code generates a set of samples from a bivariate normal distribution with mean vector $\mu = \begin{bmatrix} 0.25 \\ 1 \end{bmatrix}$ and covariance matrix $\Sigma = \begin{bmatrix} 1 & 0.5 \\ 0.5 & 1 \end{bmatrix}$, then computes and visualizes the regression lines corresponding to the conditional expectations $\mathbb{E}[\mathbf{z}_1 | \mathbf{z}_2]$ and $\mathbb{E}[\mathbf{z}_2 | \mathbf{z}_1]$. The regression coefficients are computed as $b_1 = \frac{\Sigma_{12}}{\Sigma_{22}}$ and $b_2 = \frac{\Sigma_{12}}{\Sigma_{11}}$, with intercepts $a_1 = \mu_1 - b_1 \mu_2$ and $a_2 = \mu_2 - b_2 \mu_1$. The resulting lines are plotted on the $(\mathbf{z}_2, \mathbf{z}_1)$ plane, showing the best linear predictors of each variable given the other.

```
mean = np.array([0.25, 1.])
cov = np.array([[1., 0.5], [0.5, 1.]])

data = mvn(mean=mean, cov=cov).rvs(n_points)

b_1 = cov[0, 1] / cov[1, 1]
b_2 = cov[0, 1] / cov[0, 0]

a1 = mean[0] - b_1 * mean[1]
a2 = mean[1] - b_2 * mean[0]

pprint("\text{Regression of } \mathbf{z}_1 \text{ on } \mathbf{z}_2: a_1 =", str(a1),
      "\ b_1 =", str(b_1))
pprint("\text{Regression of } \mathbf{z}_2 \text{ on } \mathbf{z}_1: a_2 =", str(a2),
      "\ b_2 =", str(b_2))

z2 = np.linspace(-4, 4, 100)

z1_pred = a1 + b_1 * z2
z2_pred = (z2 - a2) / b_2
```

(continues on next page)

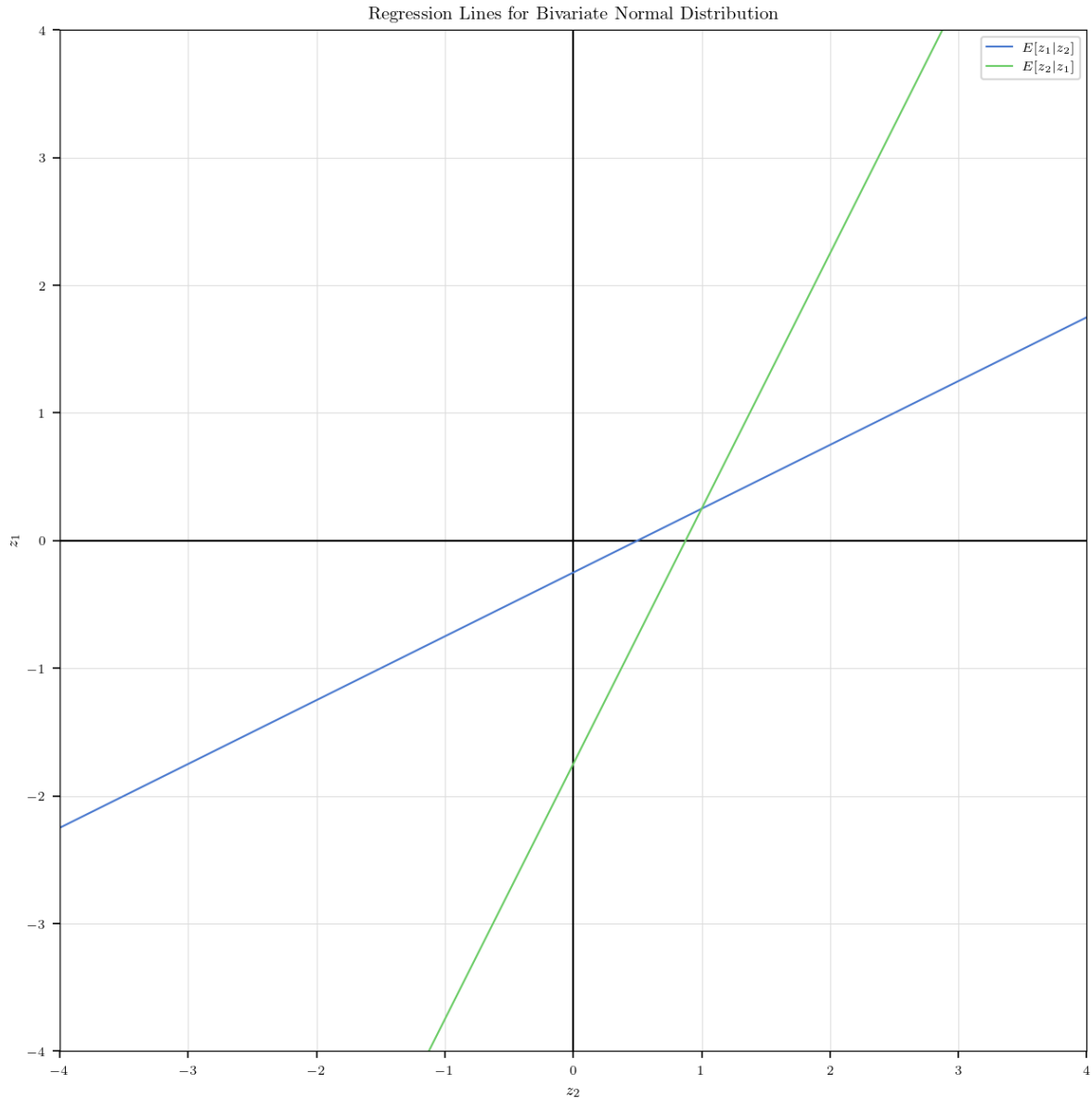
(continued from previous page)

```
fig, ax = plt.subplots(figsize=(10, 10))
ax.set_xlim(-4, 4)
ax.set_ylim(-4, 4)
ax.axhline(0, color='black', lw=1)
ax.axvline(0, color='black', lw=1)

plt.plot(z2, z1_pred, label="$E[z_1 | z_2]$")
plt.plot(z2, z2_pred, label="$E[z_2 | z_1]$")
plt.xlabel("$z_2$")
plt.ylabel("$z_1$")
plt.legend()
plt.grid("on")
plt.title("Regression Lines for Bivariate Normal Distribution")
plt.show()
```

Regression of z_1 on z_2 : $a_1 = -0.25$, $b_1 = 0.5$

Regression of z_2 on z_1 : $a_2 = 0.875$, $b_2 = 0.5$



3.7.3 Exercise: Trivariate Gaussian Data

Finally, generate 1000 samples of 3-dimensional Gaussian random data with the following parameters:

- Mean vector: $[1, 4, 7]$
- Covariance matrix: $\begin{bmatrix} 3 & 1.5 & 0.8 \\ 1.5 & 2 & 0.5 \\ 0.8 & 0.5 & 1 \end{bmatrix}$

Estimate the covariance matrix using the generated data. Visualize the pairwise scatter plots of the dimensions (use a scatter plot matrix). Hints: use `pandas.plotting.scatter_matrix` or `seaborn.pairplot` for visualization. Pay attention to the interpretation of correlations between dimensions.

Solution

The following code generates a 3-dimensional dataset of 1000 samples from a multivariate normal distribution with a specified mean vector $[1, 4, 7]$ and covariance matrix $\begin{bmatrix} 3 & 1.5 & 0.8 \\ 1.5 & 2 & 0.5 \\ 0.8 & 0.5 & 1 \end{bmatrix}$. It stores the generated samples in a Pandas DataFrame with columns labeled “Dimension 1”, “Dimension 2”, and “Dimension 3”. It then computes the empirical covariance matrix of the generated data using `np.cov`, prints both the given and estimated covariance matrices, and visualizes the pairwise relationships between the three dimensions using Seaborn’s `pairplot`.

```
import pandas as pd
import seaborn as sns

mean_3d = [1, 4, 7]
cov_3d = [[3, 1.5, 0.8],
          [1.5, 2, 0.5],
          [0.8, 0.5, 1]]

n_samples_3d = 1000

data_3d = mvn(mean=mean_3d, cov=cov_3d).rvs(size=n_samples_3d)

df = pd.DataFrame(data_3d, columns=["Dimension 1", "Dimension 2", "Dimension 3"])

sample_cov_3d = np.cov(data_3d, rowvar=False)

pprint("Cov:", np.array(cov_3d))
pprint("MC\ Cov:", sample_cov_3d)

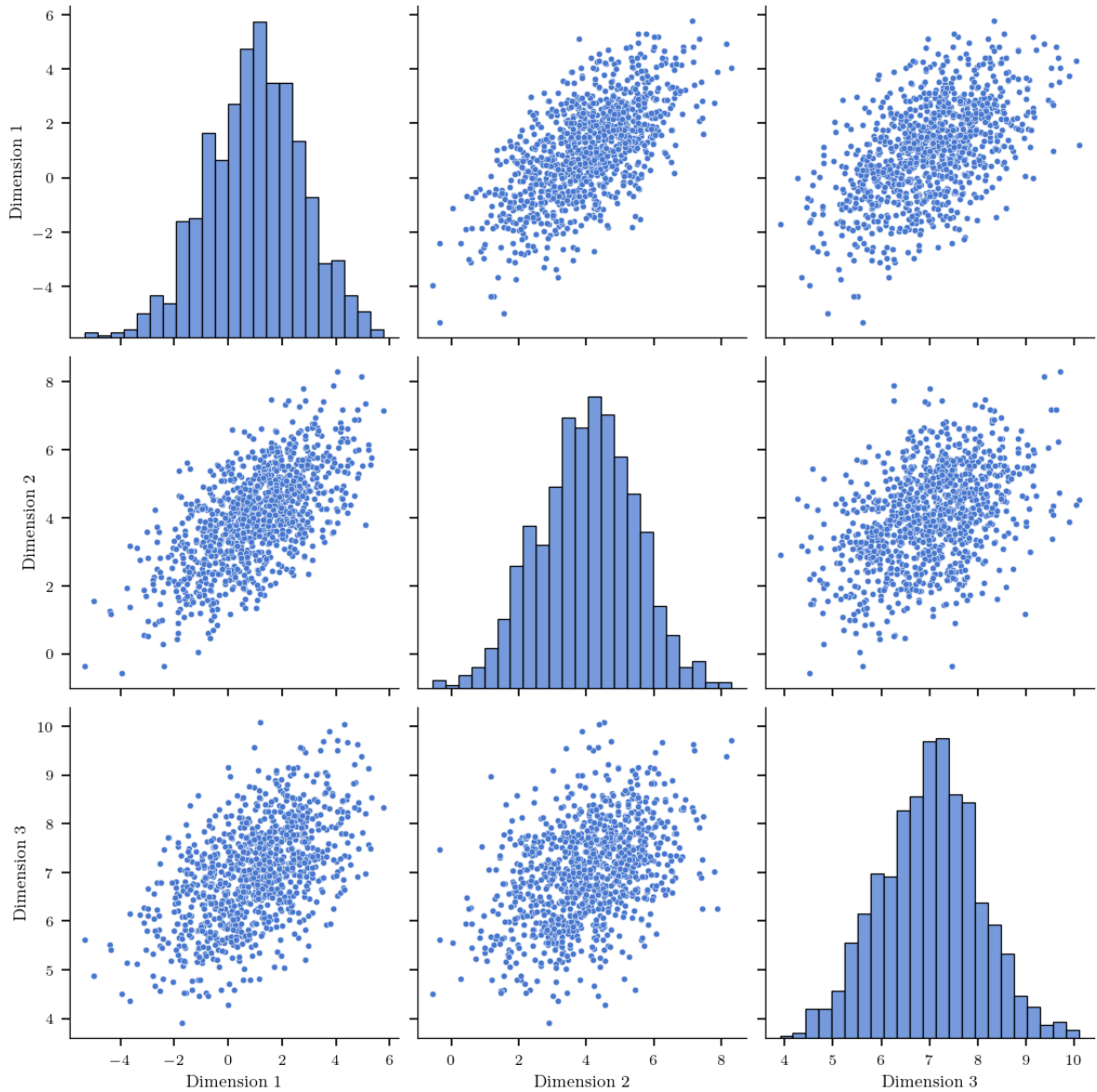
sns.pairplot(df)
plt.show()
```

Cov :

$$\begin{bmatrix} 3.0000 \\ 1.5000 \\ 0.8000 \\ 1.5000 \\ 2.0000 \\ 0.5000 \\ 0.8000 \\ 0.5000 \\ 1.0000 \end{bmatrix}$$

MC Cov :

$$\begin{bmatrix} 3.1791 \\ 1.6075 \\ 0.9758 \\ 1.6075 \\ 2.0246 \\ 0.6169 \\ 0.9758 \\ 0.6169 \\ 1.0465 \end{bmatrix}$$



3.7.4 Exercise: Partial Correlation

Given two normal $\mathcal{N}(0, 1)$ r.v. $\mathbf{z}_1, \mathbf{z}_2$ and a third r.v. $\mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2$, compute the correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$, then compute the partial correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$ given $\mathbf{y}$. Hint: use the function `corrcoef` of Numpy to compute the correlation coefficients. What happens to the partial correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$ given $\mathbf{y}$ if now $\mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2 + \mathbf{z}_3$, for $\mathbf{z}_3$ another $\mathcal{N}(0, 1)$ r.v. ?

Solution

This code defines a function `partial_corr(X, Y, Z)` that computes the partial correlation between two variables $\mathbf{x}$ and $\mathbf{y}$ given a third variable $\mathbf{z}$, using the formula $\rho_{\mathbf{xy}|\mathbf{z}} = \frac{\rho_{\mathbf{xy}} - \rho_{\mathbf{xz}}\rho_{\mathbf{yz}}}{\sqrt{(1-\rho_{\mathbf{xz}}^2)(1-\rho_{\mathbf{yz}}^2)}}$. It then generates one million samples of three independent standard normal variables $\mathbf{z}_1, \mathbf{z}_2, \mathbf{z}_3$, defines $\mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2$, and prints the empirical correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$, as well as the partial correlations between $\mathbf{z}_1$ and $\mathbf{z}_2$ given $\mathbf{y}$ and given $\mathbf{y} + \mathbf{z}_3$, illustrating that the dependence between $\mathbf{z}_1$ and $\mathbf{z}_2$ vanishes when conditioning on their sum.

```
def partial_corr(X, Y, Z):
    rhoXY = np.corrcoef(X.T, Y.T) [0, 1]
    rhoXZ = np.corrcoef(X.T, Z.T) [0, 1]
    rhoYZ = np.corrcoef(Y.T, Z.T) [0, 1]
    return (rhoXY - rhoXZ * rhoYZ) / (np.sqrt((1 - rhoXZ**2) * (1 - rhoYZ**2)))

N = int(1e6)

Z1 = np.random.normal(size=(N, 1))
Z2 = np.random.normal(size=(N, 1))
Z3 = np.random.normal(size=(N, 1))

Y = Z1 + Z2

pprint("\text{Estimations}")
pprint("\text{Correlation between } \mathbf{z}_1 \text{ and } \mathbf{z}_2: ", str(np.
    corrcoef(Z1.T, Z2.T) [0, 1]) [:8])
pprint("\text{Partial Correlation between } \mathbf{z}_1 \text{ and } \mathbf{z}_2 \text{ given } \mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2: ", str(partial_corr(Z1, Z2,
    Y)) [:8])
pprint("\text{Partial Correlation between } \mathbf{z}_1 \text{ and } \mathbf{z}_2 \text{ given } \mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2 + \mathbf{z}_3: ", str(partial_
    corr(Z1, Z2, Z1 + Z2 + Z3)) [:8])
```

Estimations

Correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$: -0.00073

Partial Correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$ given $\mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2$: -0.99999

Partial Correlation between $\mathbf{z}_1$ and $\mathbf{z}_2$ given $\mathbf{y} = \mathbf{z}_1 + \mathbf{z}_2 + \mathbf{z}_3$: -0.50070

3.8 Minimization

Scipy offers a convenient tool to perform the minimization of a derivable univariate function: in the module `optimize`, we have the `minimize` function.

3.8.1 Exercise: Minimization

Define a function

$$f(x) = x^2 + (\sin x + 25)^2$$

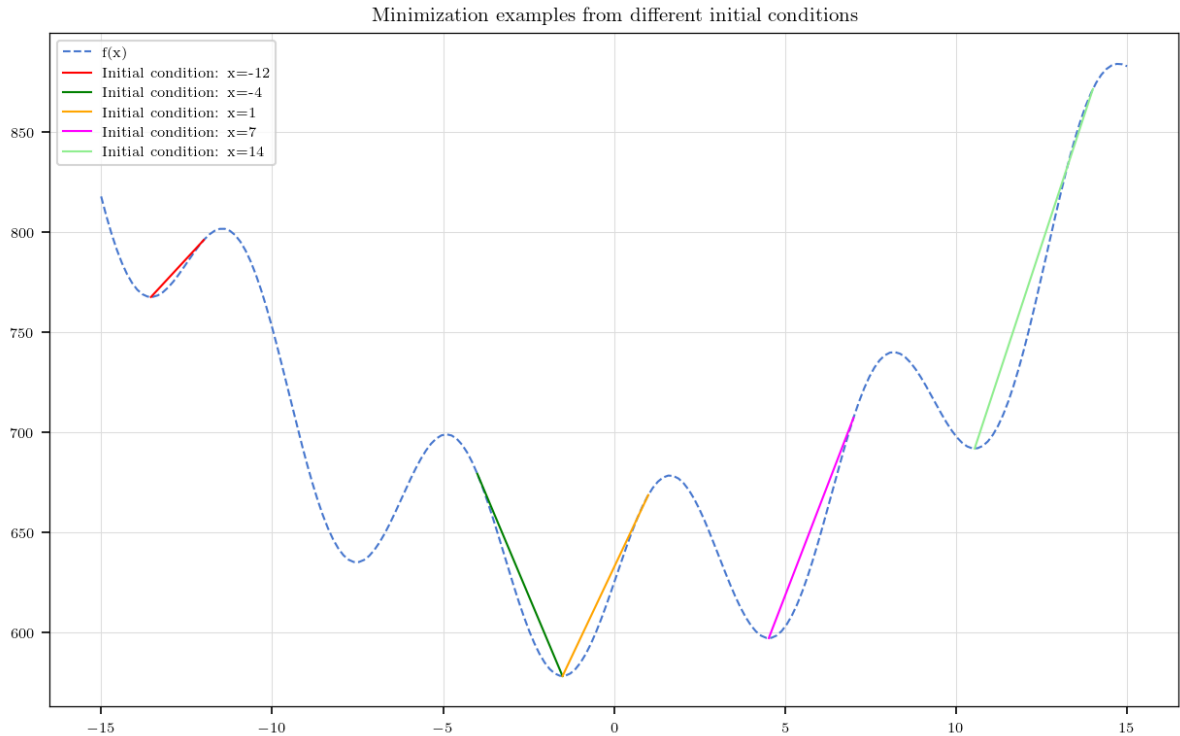
and use the `sp.optimize.minimize` function to find different local minima. Plot the function and show the different minimas. On which parameter of the `minimize` function can you play?

Solution

The following code defines a function $f(x) = x^2 + (\sin(x) + 25)^2$, then plots it over the interval $[-15, 15]$. For several initial guesses $x_0 \in \{-12, -4, 1, 7, 14\}$, the script uses the SciPy optimization routine `minimize` to find a local minimum of $f(x)$ starting from x_0 . Each minimization path is illustrated as a dashed line connecting the initial point to the corresponding minimum, with different colors representing different initial guesses. The plot includes a legend, grid, and title to clearly display the function and minimization results.

```
def f(x):
    return (x**2 + (np.sin(x)+25)**2)

x_vec = np.linspace(-15, 15, 200)
plt.figure(figsize=(10, 6))
plt.plot(x_vec, f(x_vec), linestyle='dashed', label="f(x)")
for c in (c_i:={"red": -12, "green": -4, "orange": 1, "magenta": 7, "lightgreen": 14}
    ↪):
    minimum = sp.optimize.minimize(f, x0=c_i[c])
    line_x, line_y = [c_i[c], minimum.x[0]], [f(c_i[c]), minimum.fun]
    plt.plot(line_x, line_y, color=c, label=f"Initial condition: x={c_i[c]}")
plt.legend()
plt.grid("on")
plt.title("Minimization examples from different initial conditions")
plt.show()
```



3.9 Mixture of Gaussians

Mixture of Gaussians are particular cases of linear combinations of Gaussians r.v. Formally, they are defined as

$$p = \sum_{i=1}^m w_i \mathbf{p}_i$$

where $\mathbf{p}_i$ are normal r.v. and $\sum_i w_i = 1$. As an example, we show here how to define (and plot) the mixture of three Gaussians:

```
means = [-2, 0, 3]
stds = [0.8, 1.2, 0.5]
weights = [1/3, 1/3, 1/3]

x = np.linspace(-5, 6, 1000)

y = np.zeros_like(x)
for mean, std, weight in zip(means, stds, weights):
    y += weight * sp.stats.norm.pdf(x, mean, std)

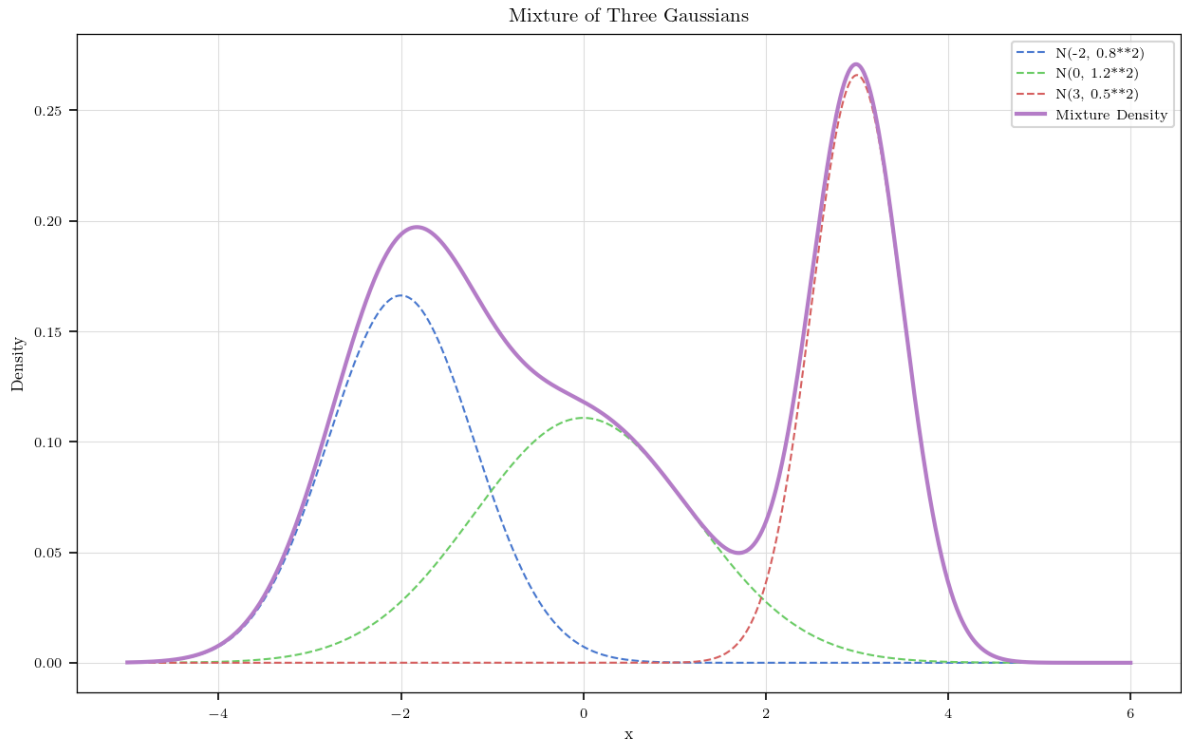
plt.figure(figsize=(10, 6))
for mean, std, weight in zip(means, stds, weights):
    plt.plot(x, weight * sp.stats.norm.pdf(x, mean, std), '--', label=f'N({mean},
    ↵ {std}*2)')

plt.plot(x, y, label='Mixture Density', linewidth=2)
plt.legend()
plt.xlabel('x')
plt.ylabel('Density')
```

(continues on next page)

(continued from previous page)

```
plt.grid(True)
plt.title('Mixture of Three Gaussians')
plt.show()
```



3.10 Conclusion

Throughout this chapter, we have presented a huge variety of probabilistic and statistical tools that form the basis of the next chapters content. Every topic was linked to a concrete implementation in `Numpy`, with which you should be familiar and comfortable before delving into the next chapters. We have also studied how to sample different distributions, allowing systematic numerical study of statistical distributions, and preparing the necessary grounds for Monte Carlo simulations.

Monte Carlo methods represent a powerful paradigm in computational mathematics, offering solutions to problems that would otherwise be analytically intractable. Throughout this chapter, we've explored the fundamental principles of probabilistic methods and Monte Carlo simulations, covering essential concepts from probability theory and their practical applications.

We've seen how random sampling can be leveraged to approximate integrals, estimate statistical properties, and solve complex problems in high-dimensional spaces.

As computational resources continue to advance, Monte Carlo techniques will remain at the forefront of scientific computing, enabling researchers and practitioners to tackle increasingly complex challenges. The principles covered in this chapter provide a solid foundation for exploring more advanced Monte Carlo techniques such as Markov Chain Monte Carlo (MCMC), importance sampling, particle filters... But, in our case, Monte Carlo processes can be seen as good abstraction of real-life processes, and Machine Learning techniques then serve as tools to study the outcomes of Monte Carlo experiments.

Remember that Monte Carlo methods are not just computational techniques but represent a probabilistic way of thinking about problems. This mindset of using randomness to solve deterministic problems opens up new approaches to complex

challenges across all quantitative disciplines.

LINEAR MODELS

4.1 Introduction

Linear models form one of the foundational pillars of statistical learning. Their simplicity, interpretability, and strong theoretical underpinnings make them an essential first step when analyzing data and building predictive models. Despite their apparent simplicity, linear models offer a surprising level of flexibility, especially when combined with feature transformations such as polynomials or radial basis functions.

In this chapter, we explore the principles and practical implementation of linear models. We begin with the generation of synthetic multivariate data to simulate a controlled environment for experimentation. We then dive into the manual computation of the Ordinary Least Squares (OLS) solution using matrix algebra, before comparing our results to those produced by well-established Python libraries such as `statsmodels`.

The chapter also highlights the importance of residual analysis for model diagnostics, offering tools to detect violations of assumptions like linearity, homoscedasticity, and normality. We further extend the linear model framework through input transformations, allowing us to capture non-linear patterns while preserving the model's linear structure in parameters.

Finally, we introduce **Ridge Regression**, a regularized variant of linear regression, and examine its role in mitigating overfitting—particularly in scenarios involving multicollinearity or high-dimensional input spaces.

We now assume basic familiarity with matrix algebra and Python packages such as NumPy, Pandas, and Statsmodels.

Don't forget: if some of those libraries are not installed on your computer, you can always install them using [Pip](#).

We define the necessary imports as well as the `pprint` function.

```
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
from scipy.stats import probplot
from IPython.display import display, Math
from numpyarray_to_latex.jupyter import to_ltx
import matplotlib.pyplot as plt
plt.style.use(['seaborn-v0_8-muted', 'practicals.mplstyle'])
def pprint(*args):
    res = ""
    for i in args:
        if type(i) == np.ndarray:
```

(continues on next page)

(continued from previous page)

```

    res += to_ltx(i, brackets='[]')
    elif type(i) == str:
        res += i
    display(Math(res))

```

4.2 Generating Synthetic Multivariate Data

We'll create a dataset where Y depends on three features: x_1 , x_2 , x_3 . The true data-generating process (DGP) is:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$.

4.2.1 Exercise

1. Create N observations of (x_1, x_2, x_3) .
2. Define true coefficients `beta_true`.
3. Generate noise and produce y .
4. Store everything in a `DataFrame` for convenience.

Solution

The following code generates a synthetic dataset for a linear regression problem with three explanatory variables (x_1 , x_2 , x_3) and one target variable (y). It sets a random seed for reproducibility and defines true regression coefficients (`beta_true`) and a standard deviation for the noise (`sigma_true`). The explanatory variables are drawn from different distributions: uniform for x_1 and x_2 , and normal for x_3 . The true response `Y_true` is computed as a linear combination of the variables with the defined coefficients, and Gaussian noise is added to simulate observed values of y . The resulting dataset, stored in a `pandas DataFrame`, includes the predictors and the noisy target variable, and the first few rows are displayed using `df.head()`.

```

np.random.seed(123)

N = 200
sigma_true = 2.0

beta_true = np.array([5.0, 1.5, -2.0, 0.5])

x1 = np.random.uniform(-5, 5, size=N)
x2 = np.random.uniform(-10, 10, size=N)
x3 = np.random.normal(0, 2, size=N)

df = pd.DataFrame({'x1':x1, 'x2':x2, 'x3':x3})

y_true = (beta_true[0]
          + beta_true[1]*df['x1']
          + beta_true[2]*df['x2']
          + beta_true[3]*df['x3'])

noise = np.random.normal(0, sigma_true, size=N)

```

(continues on next page)

(continued from previous page)

```
df['y'] = y_true + noise
df.head()
```

	x1	x2	x3	y
0	1.964692	0.852719	-3.510804	5.687964
1	-2.138607	-8.664511	-0.697215	16.509223
2	-2.731485	3.067297	-0.385230	-4.050829
3	0.513148	9.921727	0.898271	-12.453060
4	2.194690	5.387947	-0.290727	-1.697682

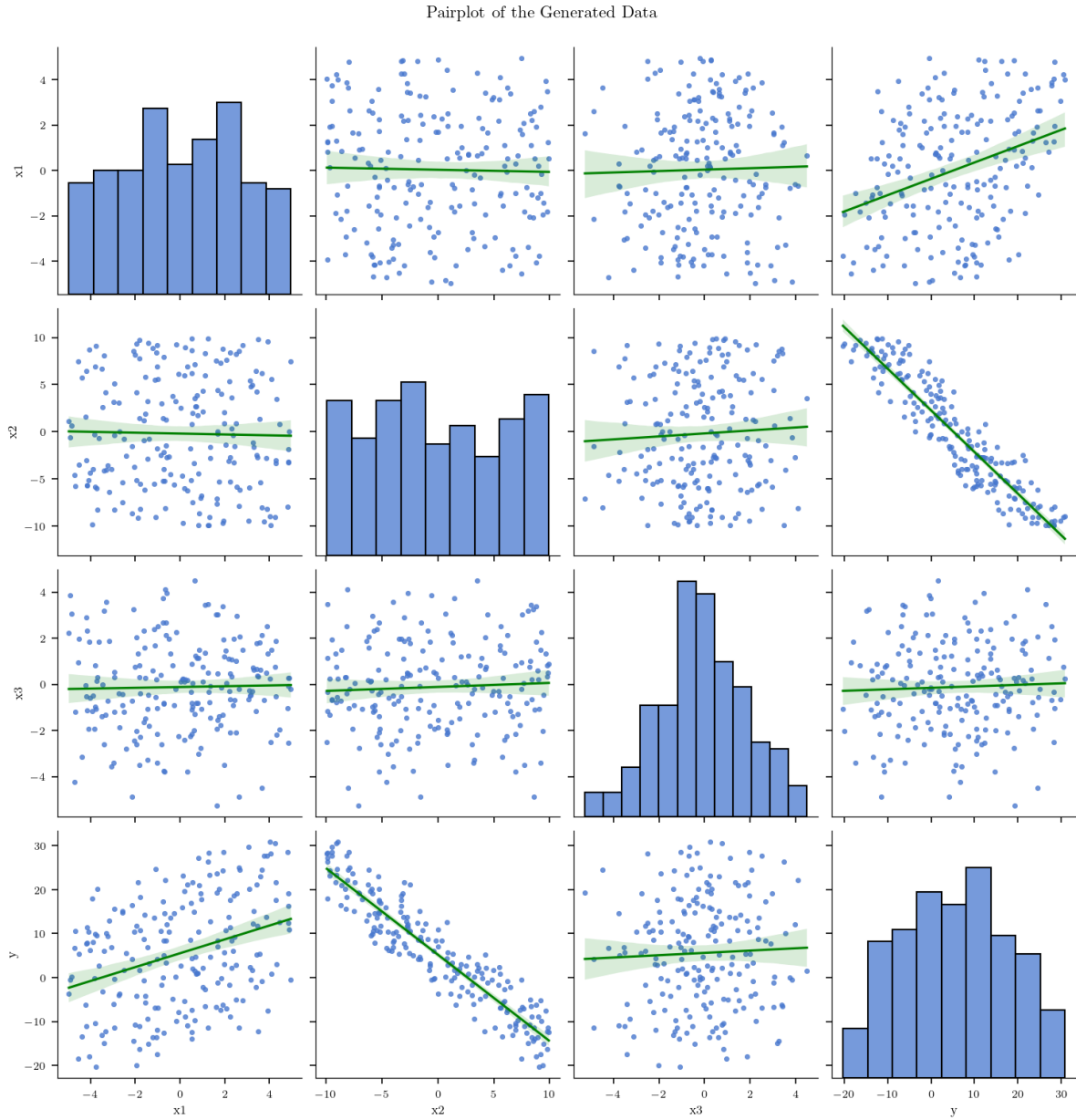
4.2.2 Quick Data Inspection

The following code performs exploratory data analysis on a DataFrame `df`. It first prints summary statistics of the DataFrame using `df.describe()`, which includes metrics like mean, standard deviation, min, and max for each numerical column. Then, it creates a pairplot using Seaborn with regression lines (`kind='reg'`) to visualize pairwise relationships between all numerical features in the DataFrame. The regression lines are colored red, as specified in the `plot_kws`. A title is added slightly above the plot using `plt.suptitle()`, and finally, the complete visualization is displayed with `plt.show()`.

Doing a first data inspection is extremely important. In real-life cases, you could discover patterns in the data, discard particular features, or even, sometimes, solve the problem you are tackling just by extracting visual information by the data. This step also helps understanding the problem you are dealing with. The more visualization of the data you make, the more expert you become on the dataset. Never underestimate the importance of the base knowledge of the problem: *data is more than just numbers*.

```
print(df.describe())
sns.pairplot(df, kind='reg', plot_kws={'line_kws':{'color':'green'}})
plt.suptitle("Pairplot of the Generated Data", y=1.02)
plt.show()
```

	x1	x2	x3	y
count	200.000000	200.000000	200.000000	200.000000
mean	0.032708	-0.193044	-0.112239	5.572700
std	2.663947	5.907073	1.963533	12.473036
min	-4.973119	-9.929356	-5.262876	-20.342999
25%	-1.953937	-5.254828	-1.286318	-4.258475
50%	0.207716	-0.714124	-0.101728	5.417761
75%	2.181854	5.168943	1.206797	14.861563
max	4.953585	9.921727	4.501352	30.849943



4.3 Ordinary Least Squares (OLS) – Manual Computation Using Matrix Algebra

We begin by specifying the standard linear regression model in matrix form:

$$y = X\beta + \epsilon,$$

where:

- y is an $N \times 1$ vector of observed responses,
- X is an $N \times p$ design matrix, which includes a column of ones to account for the intercept,
- β is a $p \times 1$ vector of regression coefficients (including the intercept),

- ϵ is an $N \times 1$ vector of errors or residuals.

The goal of Ordinary Least Squares is to find the vector $\hat{\beta}$ that minimizes the sum of squared residuals. There is a well-known closed-form solution for this optimization problem, given by:

$$\hat{\beta} = (X^T X)^{-1} X^T y.$$

This formula provides the best linear unbiased estimator (*BLUE*) of the coefficients under standard assumptions (linearity, independence, homoscedasticity, and normally distributed errors).

In the following steps, we will:

1. Manually construct the design matrix X , ensuring it includes a column of ones for the intercept.
2. Compute the estimated coefficients $\hat{\beta}$ using NumPy's matrix operations, specifically `np.linalg.inv(...)`.
3. Compare the computed estimates with the known or true values of the coefficients (`beta_true`) used to generate the data.
4. Validate our manual computation by comparing it to the output from the `statsmodels OLS` implementation.

```
X_mat = np.column_stack([
    np.ones(N),
    df['x1'].values,
    df['x2'].values,
    df['x3'].values
])

y_vec = df['y'].values.reshape(-1,1)

print("X_mat shape:", X_mat.shape)
print("y_vec shape:", y_vec.shape)
```

```
X_mat shape: (200, 4)
y_vec shape: (200, 1)
```

```
XTX = X_mat.T @ X_mat
XTX_inv = np.linalg.inv(XTX)
XTy = X_mat.T @ y_vec
beta_hat_manual = XTX_inv @ XTy

beta_hat_manual = beta_hat_manual.flatten()

pprint("\\text{Manual OLS estimates}:", beta_hat_manual)
pprint("\\text{True coefficients}:", beta_true)
```

Manual OLS estimates :

$$\begin{bmatrix} 5.2043 \\ 1.4772 \\ -1.9629 \\ 0.5243 \end{bmatrix}$$

True coefficients :

$$\begin{bmatrix} 5.0000 \\ 1.5000 \\ -2.0000 \\ 0.5000 \end{bmatrix}$$

We see the manually computed OLS estimates are close to the true ones.

4.3.1 Comparison with statsmodels

We'll let statsmodels compute OLS, then compare the results in detail.

Note, concretely, you are expected to be able to compute all the algorithms *by hand*, meaning, *in plain Python code*. However, in real-life, you can of course use the existing libraries, and knowing the most common ones is part of the basic knowledge expected from a modern data scientist.

```
X_design_sm = sm.add_constant(df[['x1', 'x2', 'x3']])
model_ols = sm.OLS(df['y'], X_design_sm).fit()
print(model_ols.summary())

beta_hat_sm = model_ols.params.values
print("statsmodels OLS:", beta_hat_sm)
print("Manual OLS:", beta_hat_manual)
```

```

                                OLS Regression Results
=====
Dep. Variable:                  y      R-squared:                0.976
Model:                          OLS    Adj. R-squared:           0.976
Method:                        Least Squares    F-statistic:            2657.
Date:                          Thu, 09 Apr 2026    Prob (F-statistic):      2.01e-158
Time:                          15:13:25    Log-Likelihood:         -415.03
No. Observations:              200    AIC:                    838.1
Df Residuals:                  196    BIC:                    851.2
Df Model:                      3
Covariance Type:               nonrobust
=====
                                coef    std err          t      P>|t|      [0.025    0.975]
-----
const                5.2043      0.138     37.720     0.000      4.932      5.476
x1                   1.4772      0.052     28.497     0.000      1.375      1.579
x2                  -1.9629      0.023    -83.872     0.000     -2.009     -1.917
x3                   0.5243      0.070      7.446     0.000      0.385      0.663
=====
Omnibus:                  1.025    Durbin-Watson:           2.042
Prob(Omnibus):            0.599    Jarque-Bera (JB):        1.117
Skew:                    -0.117    Prob(JB):                0.572
Kurtosis:                 2.719    Cond. No.                 5.91
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

statsmodels OLS: [5.20431058 1.47721924 -1.96286883 0.52430039]

Manual OLS: [5.20431058 1.47721924 -1.96286883 0.52430039]

4.3.2 Observations

- The *plain Python* results match `statsmodels` results (modulo tiny numerical differences).
- Both are near the true coefficients $(5.0, 1.5, -2.0, 0.5)$.

4.4 Residual Analysis and Tests

Once we have the fitted model, let's analyze **residuals**:

$$e_i = y_i - \hat{y}_i.$$

```
residuals = model_ols.resid
fitted_vals = model_ols.fittedvalues
```

When evaluating the quality of a regression model, it is crucial to check whether the underlying assumptions hold. Two common residual diagnostic plots are:

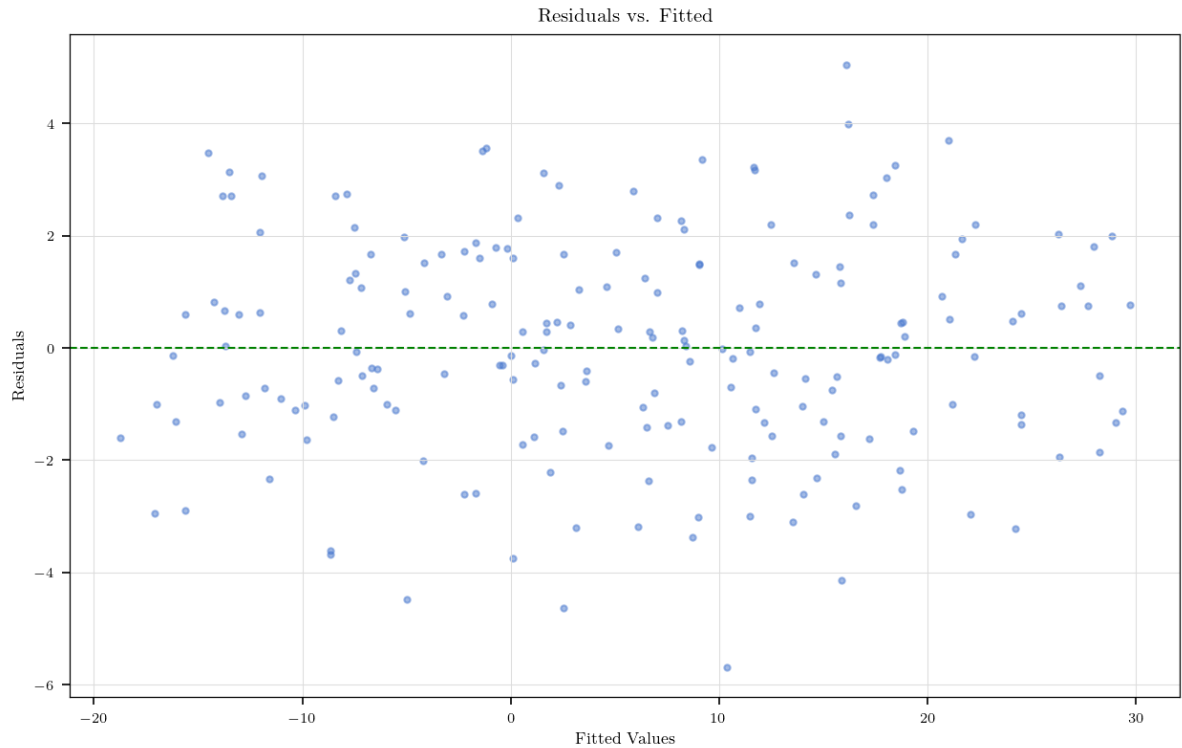
- The residuals vs. fitted plot
- The QQ-plot.

We present both in details in the following sections.

4.4.1 Residuals vs. Fitted Plot

- **Purpose:** This plot helps detect **non-linearity** and **heteroscedasticity** (unequal variance of residuals).
- **How it works:**
 - The **x-axis** represents the fitted (predicted) values from the regression model.
 - The **y-axis** represents the residuals (errors).
 - Ideally, the residuals should be randomly scattered around zero, forming a **homogeneous “cloud”**.
- **Interpretation:**
 - If a clear pattern (e.g., a curve) appears, it suggests **non-linearity**, meaning the model might be missing key features or transformations.
 - If the spread of residuals **increases or decreases systematically**, it indicates **heteroscedasticity**, meaning the variance of errors is not constant, which violates regression assumptions.

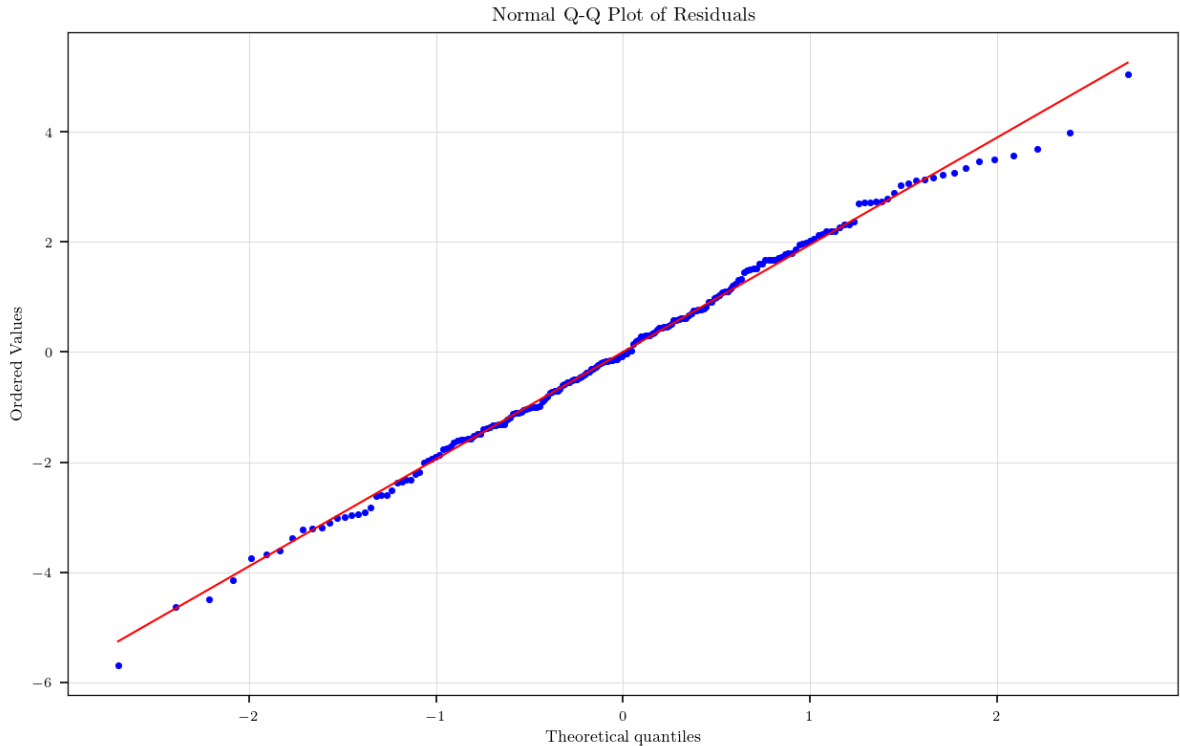
```
plt.figure(figsize=(10, 6))
plt.scatter(fitted_vals, residuals, alpha=0.5)
plt.axhline(y=0, color='green', linestyle='--')
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs. Fitted")
plt.grid("on")
plt.show()
```



4.4.2 Normal Q-Q Plot (Quantile-Quantile Plot)

- **Purpose:** Checks whether the residuals follow a **normal distribution**, a key assumption in many regression analyses.
- **How it works:**
 - The **x-axis** shows the theoretical quantiles (expected values under normality).
 - The **y-axis** shows the actual quantiles from the residuals.
 - If the residuals are normally distributed, they should closely follow a **straight diagonal line**.
- **Interpretation:**
 - **Straight-line alignment:** Residuals are normally distributed.
 - **Deviations (e.g., S-shape, heavy tails, skewed distribution):** Suggests non-normal residuals, which might indicate missing variables, skewed data, or influential outliers.

```
plt.figure(figsize=(10, 6))
probplot(residuals, dist="norm", plot=plt)
plt.title("Normal Q-Q Plot of Residuals")
plt.grid("on")
plt.show()
```



Comments

- Ideally, residuals vs. fitted values have **no clear pattern** and appear *cloud-like*.
- The Q-Q plot should be fairly linear if they are normally distributed.

We can also check the standard deviation or run further tests (e.g., Shapiro-Wilk, Breusch-Pagan for heteroscedasticity) if desired.

4.5 Input Transformations - Polynomial Features

When we talk about **linear regression**, we mean that the model is **linear in parameters**, not necessarily in the input variables X . That is, the target variable y is modeled as a **linear combination of parameters**:

$$y = \beta_0 + \beta_1 f_1(X) + \beta_2 f_2(X) + \dots + \beta_n f_n(X) + \epsilon$$

where:

- $\beta_0, \beta_1, \dots, \beta_n$ are the regression coefficients.
- $f_i(X)$ are **functions of the input features** (which can be non-linear transformations like squares or interaction terms).
- ϵ is the error term.

4.5.1 Example: Still a Linear Model

Even if we introduce **non-linear transformations** of X , as long as the model is **linear in the parameters** β_i , it is still considered a **linear model**. For example:

$$y = \beta_0 + \beta_1 X + \beta_2 X^2 + \epsilon$$

This model includes a quadratic term (X^2), but is still a **linear regression model** because it remains a linear combination of the parameters $\beta_0, \beta_1, \beta_2$.

4.5.2 What Would NOT Be a Linear Model?

A regression model becomes **non-linear in parameters** if the parameters appear in a non-linear way. For example:

$$y = \beta_0 + e^{\beta_1 X} + \epsilon$$

or

$$y = \frac{1}{\beta_0 + \beta_1 X} + \epsilon$$

These models are **non-linear in parameters** because the relationship between y and the β coefficients is no longer a simple linear combination.

4.6 Why Use Polynomial Features in Linear Regression?

Since linear regression allows us to use **non-linear transformations of X** while keeping the model linear in parameters, we can **extend linear regression** to capture more complex relationships.

4.6.1 Polynomial Feature Expansion

Given original features X_1, X_2, X_3 , we can introduce:

- **Squared terms:** $X_1^2, X_2^2, X_3^2 \rightarrow$ captures curvature.
- **Interaction terms:** $X_1 X_2, X_1 X_3, X_2 X_3 \rightarrow$ captures relationships between variables.

By expanding the features in this way, we allow a linear regression model to **approximate non-linear patterns** in the data, making it more flexible while maintaining its interpretability.

4.6.2 Example

A second-degree polynomial regression model:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_1^2 + \beta_5 X_2^2 + \beta_6 X_3^2 + \beta_7 X_1 X_2 + \beta_8 X_1 X_3 + \beta_9 X_2 X_3 + \epsilon$$

This is still a **linear model in parameters** but now captures non-linear relationships in X .

4.6.3 How to Implement Polynomial Features in Python

In Python, we can generate polynomial features using `sklearn.preprocessing.PolynomialFeatures`.

```
from sklearn.preprocessing import PolynomialFeatures

poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(df[['x1', 'x2', 'x3']])
print("Original shape:", df[['x1', 'x2', 'x3']].shape)
print("Polynomial shape:", X_poly.shape)

X_poly_mat = np.column_stack([np.ones(N), X_poly])
```

```
Original shape: (200, 3)
Polynomial shape: (200, 9)
```

The OLS is obtained using the same closed-form formulation:

```
XTX_poly = X_poly_mat.T @ X_poly_mat
XTX_poly_inv = np.linalg.inv(XTX_poly)
XTy_poly = X_poly_mat.T @ y_vec
beta_hat_poly_manual = XTX_poly_inv @ XTy_poly
beta_hat_poly_manual = beta_hat_poly_manual.flatten()

print("Manual OLS (polynomial) shape of beta:", beta_hat_poly_manual.shape)
pprint("\\text{beta\\_hat\\_poly\\_manual}:", beta_hat_poly_manual)
```

```
Manual OLS (polynomial) shape of beta: (10,)
```

```
beta_hat_poly_manual :
```

5.0526
1.4773
-1.9646
0.5315
0.0036
0.0054
0.0249
0.0037
0.0133
-0.0026

We can compare the results offered by `statsmodel`:

```
X_poly_sm = sm.add_constant(X_poly)
model_poly_sm = sm.OLS(df['y'], X_poly_sm).fit()
beta_hat_poly_sm = model_poly_sm.params.values

pprint("\\text{Statsmodels polynomial OLS}:", beta_hat_poly_sm)
pprint("\\text{Manual polynomial OLS}:", beta_hat_poly_manual)
print("\\nPolynomial OLS summary:\\n", model_poly_sm.summary())
```

Statsmodels polynomial OLS :

```
[5.0526
 1.4773
-1.9646
 0.5315
 0.0036
 0.0054
 0.0249
 0.0037
 0.0133
-0.0026]
```

Manual polynomial OLS :

```
[5.0526
 1.4773
-1.9646
 0.5315
 0.0036
 0.0054
 0.0249
 0.0037
 0.0133
-0.0026]
```

Polynomial OLS summary:

OLS Regression Results						
=====						
Dep. Variable:	y	R-squared:	0.976			
Model:	OLS	Adj. R-squared:	0.975			
Method:	Least Squares	F-statistic:	872.4			
Date:	Thu, 09 Apr 2026	Prob (F-statistic):	2.13e-149			
Time:	15:13:25	Log-Likelihood:	-413.46			
No. Observations:	200	AIC:	846.9			
Df Residuals:	190	BIC:	879.9			
Df Model:	9					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	5.0526	0.287	17.620	0.000	4.487	5.618
x1	1.4773	0.053	27.754	0.000	1.372	1.582
x2	-1.9646	0.024	-82.615	0.000	-2.012	-1.918
x3	0.5315	0.073	7.290	0.000	0.388	0.675
x4	0.0036	0.020	0.179	0.858	-0.036	0.043
x5	0.0054	0.009	0.571	0.569	-0.013	0.024
x6	0.0249	0.028	0.879	0.381	-0.031	0.081
x7	0.0037	0.005	0.803	0.423	-0.005	0.013
x8	0.0133	0.012	1.074	0.284	-0.011	0.038
x9	-0.0026	0.028	-0.095	0.924	-0.058	0.052
=====						
Omnibus:	0.925	Durbin-Watson:	2.024			
Prob(Omnibus):	0.630	Jarque-Bera (JB):	1.006			
Skew:	-0.090	Prob(JB):	0.605			
Kurtosis:	2.703	Cond. No.	97.1			
=====						

Notes:

(continues on next page)

(continued from previous page)

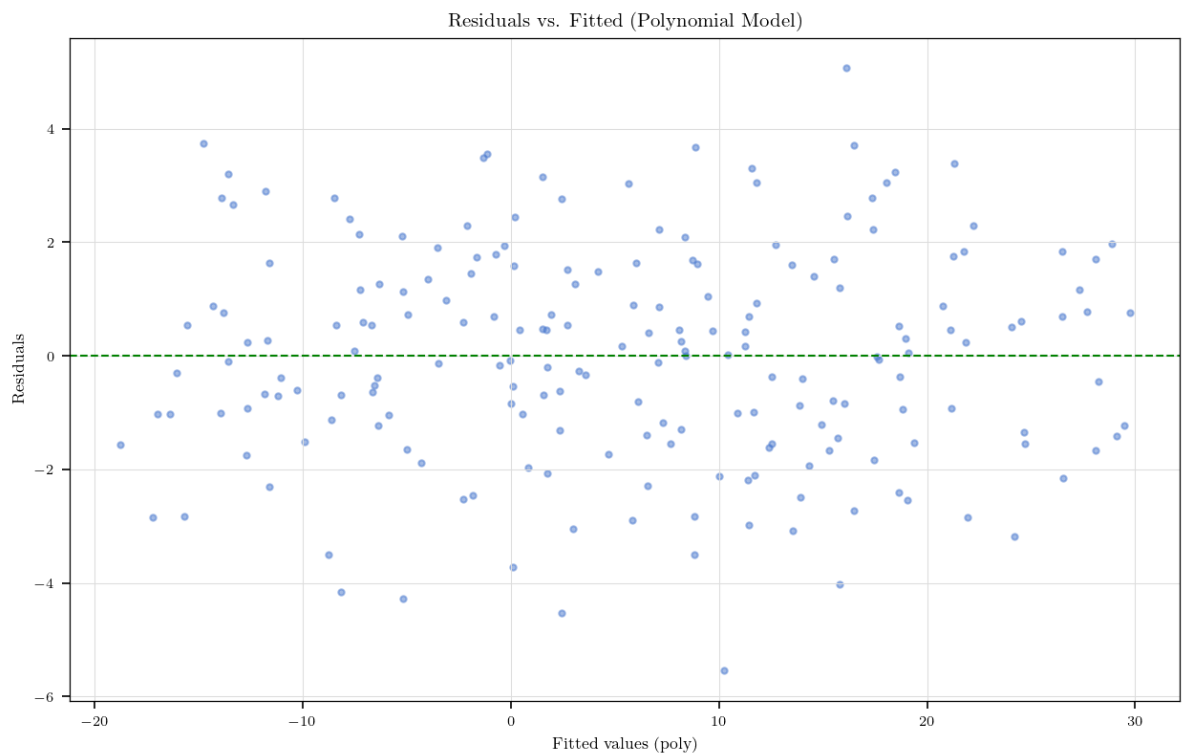
```
[1] Standard Errors assume that the covariance matrix of the errors is correctly
    specified.
```

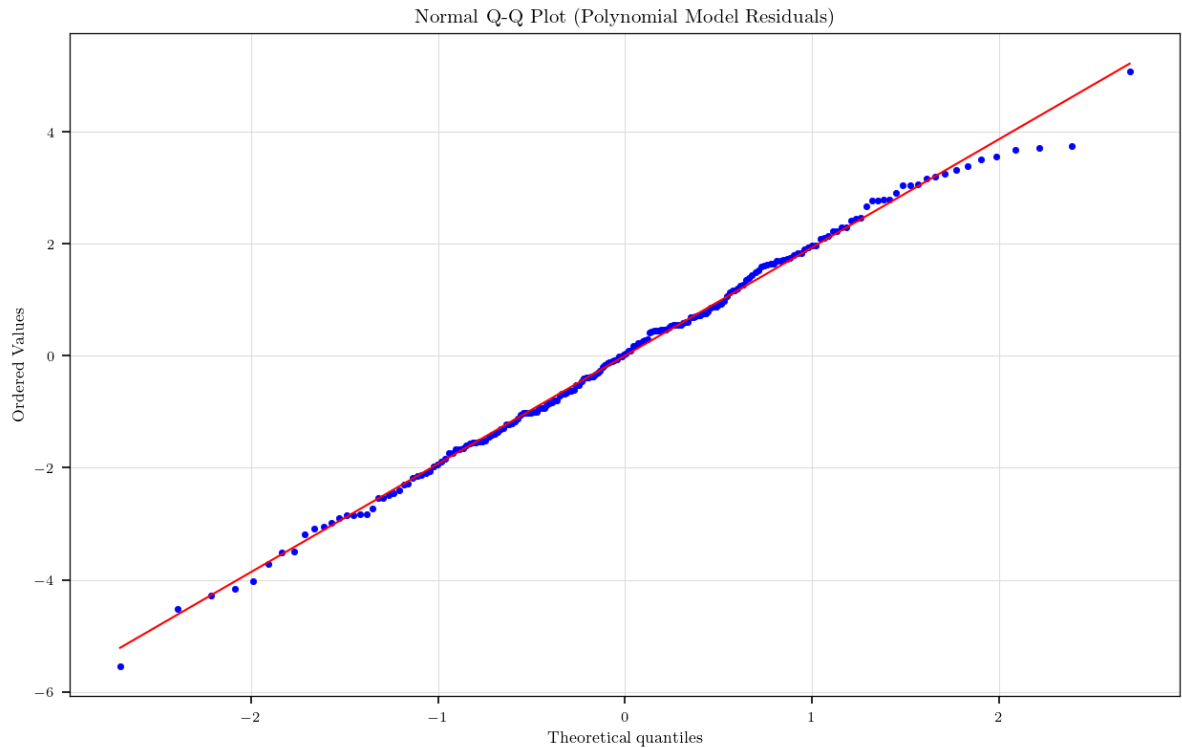
2.6.3.1. Residual analysis and QQ-plot

```
residuals_poly = model_poly_sm.resid
fitted_poly = model_poly_sm.fittedvalues

plt.figure(figsize=(10, 6))
plt.scatter(fitted_poly, residuals_poly, alpha=0.5)
plt.axhline(y=0, color='green', linestyle='--')
plt.xlabel("Fitted values (poly)")
plt.ylabel("Residuals")
plt.title("Residuals vs. Fitted (Polynomial Model)")
plt.grid("on")
plt.show()

plt.figure(figsize=(10, 6))
probplot(residuals_poly, dist="norm", plot=plt)
plt.title("Normal Q-Q Plot (Polynomial Model Residuals)")
plt.grid("on")
plt.show()
```



**Interpretation:**

- If the polynomial model captures more structure, we may see less pattern in the residuals plot.
- Alternatively, if the true relationship was already linear, polynomial terms might just overfit or remain insignificant.

4.7 RBF Expansion

Polynomial feature expansion is a powerful tool to introduce **non-linearity** into a linear regression model, but it has several limitations:

- **Feature explosion:** As the polynomial degree increases, the number of new features grows **exponentially**, making models computationally expensive and prone to overfitting.
- **Global transformations:** Polynomial terms like X^2 , X^3 affect the entire input space, meaning that small changes in X have a large impact everywhere, which might not always be desirable.
- **Poor handling of local variations:** Polynomial functions are not ideal when the relationship between X and y has local variations or is non-smooth.

This is where **Radial Basis Function (RBF) expansion** can be useful.

4.7.1 What is RBF Expansion?

RBF expansion **maps the original features into a higher-dimensional space** using functions that respond **locally** to the input values.

Instead of using polynomial transformations like:

$$y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_3 + \beta_4 X_1^2 + \beta_5 X_2^2 + \beta_6 X_3^2 + \beta_7 X_1 X_2 + \dots$$

We use **random Fourier features** to approximate an RBF kernel, generating transformed variables like:

$$\Phi(X) = \cos(WX) + \sin(WX)$$

where W is a random weight matrix sampled from a Gaussian distribution.

Why Use RBF Instead of Polynomial Expansion?

- **Better for local variations:** Unlike polynomials, RBFs can capture **localized** structures in data.
- **Fixed feature size:** The number of transformed features is independent of the input size, making it computationally efficient.
- **Good for high-dimensional data:** Works well when the input space is **high-dimensional** because it avoids feature explosion.

4.7.2 Implementing RBF Expansion with Random Fourier Features

We use `sklearn.kernel_approximation.RBFSampler` to generate **random Fourier features** that approximate the RBF kernel.

```
from sklearn.kernel_approximation import RBFSampler

rbf = RBFSampler(gamma=0.05, n_components=50, random_state=123)
X_rbf = rbf.fit_transform(df[['x1', 'x2', 'x3']])
print("RBF-transformed shape:", X_rbf.shape)

# We'll do OLS with statsmodels for brevity:
X_rbf_sm = sm.add_constant(X_rbf)
model_rbf_sm = sm.OLS(df['y'], X_rbf_sm).fit()

print(model_rbf_sm.summary())

resid_rbf = model_rbf_sm.resid
fitted_rbf = model_rbf_sm.fittedvalues

plt.figure(figsize=(10, 6))
plt.scatter(fitted_rbf, resid_rbf, alpha=0.5)
plt.axhline(y=0, color='green', linestyle='--')
plt.xlabel("Fitted values (RBF)")
plt.ylabel("Residuals")
plt.title("Residuals vs. Fitted (RBF)")
plt.grid("on")
plt.show()
```

RBF-transformed shape: (200, 50)

OLS Regression Results

```

=====
Dep. Variable:          y      R-squared:          0.980
Model:                OLS      Adj. R-squared:       0.973
Method:              Least Squares      F-statistic:       146.9
Date:                Thu, 09 Apr 2026      Prob (F-statistic):   5.70e-105
Time:                15:13:25      Log-Likelihood:      -396.22
No. Observations:      200      AIC:                894.4
Df Residuals:          149      BIC:                1063.
Df Model:              50
Covariance Type:      nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-43.4898	19.729	-2.204	0.029	-82.475	-4.505
x1	7.1364	11.512	0.620	0.536	-15.612	29.885
x2	8.9773	3.334	2.692	0.008	2.388	15.566
x3	2.6549	7.719	0.344	0.731	-12.598	17.908
x4	1.1046	1.872	0.590	0.556	-2.595	4.804
x5	137.0639	46.449	2.951	0.004	45.280	228.848
x6	16.5251	11.039	1.497	0.137	-5.288	38.338
x7	-3.4503	4.473	-0.771	0.442	-12.288	5.388
x8	2.9028	3.919	0.741	0.460	-4.842	10.648
x9	-0.5036	6.801	-0.074	0.941	-13.942	12.934
x10	16.0558	15.230	1.054	0.294	-14.040	46.151
x11	24.7962	12.423	1.996	0.048	0.249	49.344
x12	59.4056	7.430	7.995	0.000	44.723	74.088
x13	-6.8051	11.750	-0.579	0.563	-30.024	16.414
x14	12.4240	24.130	0.515	0.607	-35.258	60.106
x15	-19.3450	7.617	-2.540	0.012	-34.397	-4.293
x16	-266.9437	65.123	-4.099	0.000	-395.627	-138.260
x17	5.0042	4.000	1.251	0.213	-2.900	12.908
x18	11.1227	5.828	1.909	0.058	-0.393	22.639
x19	24.4491	13.325	1.835	0.069	-1.882	50.780
x20	-6.4866	3.810	-1.702	0.091	-14.016	1.042
x21	-136.7679	40.658	-3.364	0.001	-217.108	-56.428
x22	-0.5810	7.714	-0.075	0.940	-15.825	14.663
x23	29.9036	16.223	1.843	0.067	-2.153	61.960
x24	43.4672	31.745	1.369	0.173	-19.261	106.195
x25	-2.5586	1.587	-1.612	0.109	-5.695	0.578
x26	24.3295	50.242	0.484	0.629	-74.950	123.609
x27	24.6458	14.259	1.728	0.086	-3.530	52.821
x28	-9.9883	14.837	-0.673	0.502	-39.306	19.329
x29	98.4893	31.798	3.097	0.002	35.657	161.322
x30	-5.2150	10.521	-0.496	0.621	-26.004	15.574
x31	-2.7180	16.134	-0.168	0.866	-34.600	29.164
x32	2.4668	1.976	1.248	0.214	-1.438	6.372
x33	2.5672	2.217	1.158	0.249	-1.814	6.948
x34	200.9372	54.077	3.716	0.000	94.081	307.793
x35	42.1785	14.095	2.992	0.003	14.327	70.031
x36	-24.5715	128.680	-0.191	0.849	-278.845	229.702
x37	156.2145	31.866	4.902	0.000	93.246	219.183
x38	-7.9412	16.226	-0.489	0.625	-40.004	24.122
x39	-1.1294	1.594	-0.709	0.480	-4.279	2.020
x40	-225.6936	88.747	-2.543	0.012	-401.059	-50.328
x41	-12.5555	36.848	-0.341	0.734	-85.367	60.256

(continues on next page)

(continued from previous page)

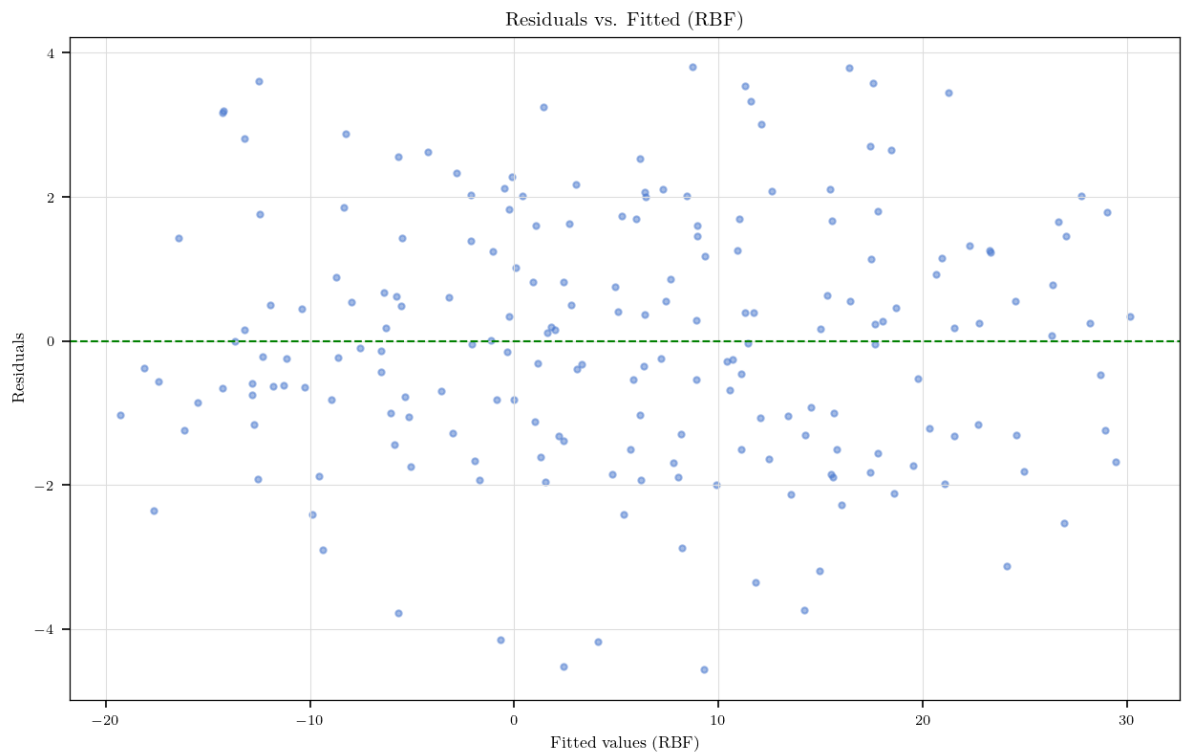
x42	-12.9924	9.099	-1.428	0.155	-30.972	4.987
x43	25.7314	13.007	1.978	0.050	0.029	51.433
x44	7.2633	15.228	0.477	0.634	-22.827	37.353
x45	-70.8756	20.960	-3.381	0.001	-112.293	-29.458
x46	-46.6215	172.329	-0.271	0.787	-387.147	293.904
x47	0.5652	3.121	0.181	0.857	-5.602	6.732
x48	-6.3771	10.449	-0.610	0.543	-27.025	14.271
x49	153.3197	46.325	3.310	0.001	61.781	244.858
x50	9.1846	5.433	1.691	0.093	-1.551	19.920

Omnibus:	0.557	Durbin-Watson:	2.049
Prob(Omnibus):	0.757	Jarque-Bera (JB):	0.682
Skew:	-0.041	Prob(JB):	0.711
Kurtosis:	2.726	Cond. No.	1.75e+03

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.75e+03. This might indicate that there are strong multicollinearity or other numerical problems.



Comments

- Each RBF feature is basically a sinusoidal basis function in random projection space.
- We could also attempt a manual approach:

$$\hat{\beta} = (X_{\text{rbf}}^{\top} X_{\text{rbf}})^{-1} X_{\text{rbf}}^{\top} y$$

if we want to replicate the normal equation style.

- The dimensionality (50) can be large, so be mindful of potential numerical issues and overfitting.

4.8 Ridge Regression

For complex expansions (like polynomial of high degree or many RBF components), **overfitting** can happen. Ridge regression adds an L2 penalty:

$$\min_{\beta} \|y - X\beta\|^2 + \alpha\|\beta\|^2.$$

4.8.1 Ridge Closed-form

Ridge also has a known closed-form:

$$\hat{\beta}_{\text{ridge}} = (X^{\top} X + \alpha I)^{-1} X^{\top} y.$$

We'll show a manual solution, then compare to scikit-learn. We'll do it on the **polynomial features** for illustration.

```
alpha_ridge = 10.0
I_p = np.eye(X_poly_mat.shape[1]) # dimension p x p, where p = number of poly-
    ↪ features + 1 (intercept)

# Usually we do not regularize the intercept, so we might set I_p[0,0] = 0.
# Let's do that to avoid shrinking the intercept:
I_p[0,0] = 0.0

XTX_ridge = X_poly_mat.T @ X_poly_mat + alpha_ridge*I_p
XTX_ridge_inv = np.linalg.inv(XTX_ridge)
XTy_ridge = X_poly_mat.T @ y_vec
beta_hat_ridge_manual = XTX_ridge_inv @ XTy_ridge
beta_hat_ridge_manual = beta_hat_ridge_manual.flatten()

print("Manual Ridge (polynomial) shape:", beta_hat_ridge_manual.shape)
pprint("\text{Manual Ridge Coeffs}:", beta_hat_ridge_manual)
```

```
Manual Ridge (polynomial) shape: (10,)
```

Manual Ridge Coeffs :

$$\begin{bmatrix} 5.0547 \\ 1.4667 \\ -1.9617 \\ 0.5238 \\ 0.0037 \\ 0.0051 \\ 0.0242 \\ 0.0037 \\ 0.0133 \\ -0.0038 \end{bmatrix}$$

```
from sklearn.linear_model import Ridge

ridge_sklearn = Ridge(alpha=alpha_ridge, fit_intercept=False)
# "fit_intercept=False" because we already included the intercept column in X_poly_mat
# We will manually set the intercept to match the first column or we can remove_
↪ intercept from X_poly_mat.

ridge_sklearn.fit(X_poly_mat, df['y'])
beta_hat_ridge_sklearn = ridge_sklearn.coef_

pprint("\\text{scikit-learn Ridge Coeffs}:", beta_hat_ridge_sklearn)
pprint("\\text{Difference (manual - sklearn)}:", (beta_hat_ridge_manual - beta_hat_
↪ ridge_sklearn))
```

scikit-learn Ridge Coeffs :

$$\begin{bmatrix} 4.1657 \\ 1.4733 \\ -1.9611 \\ 0.5235 \\ 0.0387 \\ 0.0073 \\ 0.0330 \\ 0.0122 \\ 0.0117 \\ 0.0322 \end{bmatrix}$$

Difference (manual - sklearn) :

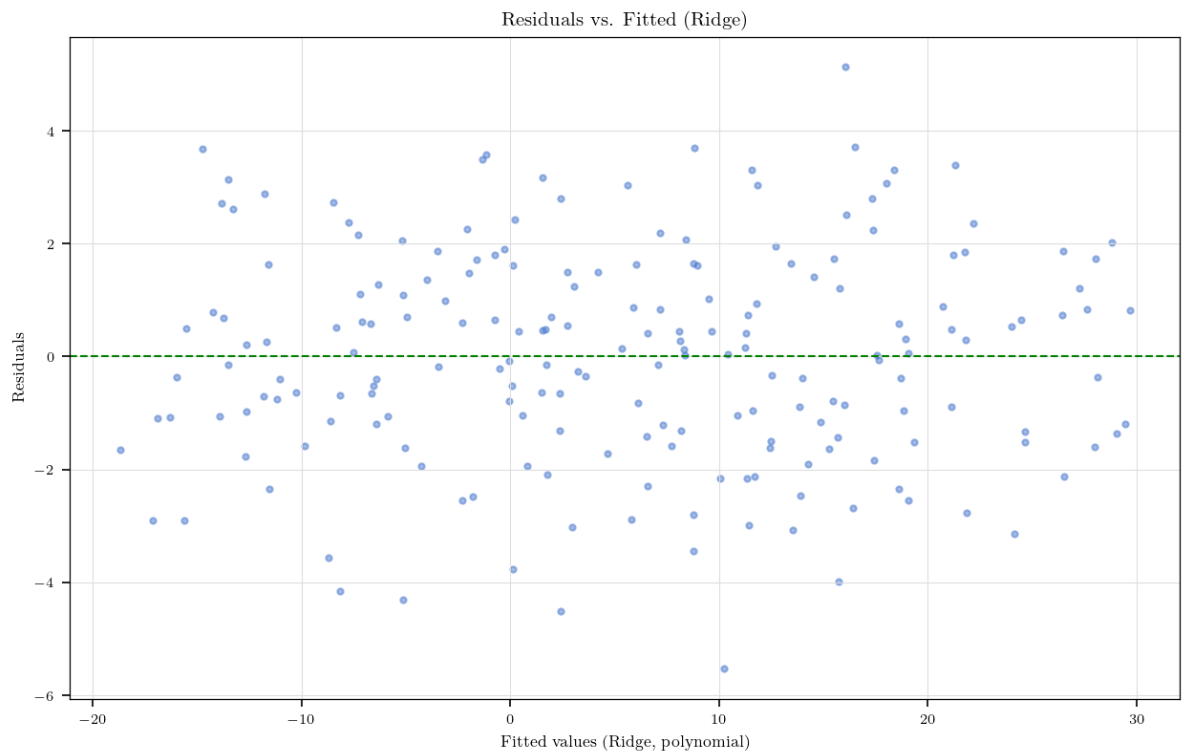
$$\begin{bmatrix} 0.8890 \\ -0.0067 \\ -0.0006 \\ 0.0003 \\ -0.0350 \\ -0.0022 \\ -0.0088 \\ -0.0085 \\ 0.0016 \\ -0.0360 \end{bmatrix}$$

They should be nearly identical (tiny floating discrepancies may occur).

4.8.2 Residual analysis for Ridge

```
y_pred_ridge = X_poly_mat @ beta_hat_ridge_manual
resid_ridge = df['y'] - y_pred_ridge

plt.figure(figsize=(10, 6))
plt.scatter(y_pred_ridge, resid_ridge, alpha=0.5)
plt.axhline(y=0, color='g', linestyle='--')
plt.xlabel("Fitted values (Ridge, polynomial)")
plt.ylabel("Residuals")
plt.title("Residuals vs. Fitted (Ridge)")
plt.grid("on")
plt.show()
```



Remarks:

- As `alpha_ridge` increases, the polynomial coefficients shrink more, typically reducing variance (but possibly increasing bias).
- We can further use cross-validation to select `alpha` automatically or do a hyperparameter grid.

4.9 Conclusion

This chapter has provided a comprehensive, hands-on exploration of linear models—starting from the manual derivation of the Ordinary Least Squares (OLS) estimator to the application of regularization techniques and feature transformations. Through synthetic data, we validated the accuracy of our estimators, examined residuals for assumption-checking, and used diagnostic plots to evaluate model fit.

We have seen how linear regression, while conceptually straightforward, remains a powerful modeling tool when used thoughtfully. The incorporation of polynomial and radial basis function expansions extends the expressive capacity of linear models, enabling them to approximate complex, non-linear relationships without abandoning interpretability.

We also introduced Ridge regression, which provides a principled approach to combat overfitting and multicollinearity, especially in high-dimensional settings. This regularized extension highlights a key transition in statistical learning: balancing model flexibility with generalization performance.

Ultimately, this chapter emphasizes not only how to implement linear models, but how to critically assess their assumptions and limitations. These foundational skills will be essential as we move on to more sophisticated models in the chapters ahead.

DATA PREPROCESSING, LOCAL MODELS AND TREE-BASED MODELS

5.1 Introduction

Machine Learning, as you now know, is based on statistical analysis of data. Data comes in many kinds of formats, resolutions, annotations, ... and therefore should be processed before performing analysis. This is the first content of this practical. We will focus on regression tasks, as already defined in the previous practicals, showing to commonly used strategies. The practical is structured around five key objectives:

1. Understanding and applying essential data preprocessing techniques

Before training a machine learning model, the data must be properly prepared. We will cover the following common preprocessing steps:

- Handling missing values: strategies to impute or remove data with missing entries.
- Feature scaling and normalization: ensuring features are on comparable scales to improve model performance.
- Encoding categorical variables: transforming non-numeric data (e.g., categories) into numerical representations suitable for machine learning algorithms.

2. Implementing the K-Nearest Neighbors (KNN) algorithm from scratch

We will write a simple, “vanilla” version of the KNN algorithm for regression tasks. This implementation will help demystify how KNN works by reinforcing the idea of prediction based on similarity in feature space.

3. Training and evaluating tree-based regression models

We will work with two popular types of decision-tree-based models for regression:

- Regression trees: models that recursively split the data space into regions to predict a continuous output.
- Random Forests: an ensemble method that combines multiple decision trees to improve predictive accuracy and reduce overfitting.

4. Interpreting feature importances in tree-based models

One advantage of tree-based models is their ability to estimate the relative importance of each input feature. We will learn how to extract and interpret these feature importances to gain insights into which variables are most influential in the prediction.

5. Using `scikit-learn` Pipelines to streamline preprocessing and modeling

To ensure clean and reproducible workflows, we will introduce `scikit-learn` pipelines, which allow us to:

- Chain together preprocessing steps and model training into a single object.
- Simplify cross-validation and hyperparameter tuning.
- Prevent data leakage by ensuring that preprocessing is properly applied to both training and test data.

By the end of this practical, you should be able to preprocess real-world data appropriately, implement a basic regression model, use tree-based models effectively, and combine preprocessing and modeling steps into a coherent, automated pipeline using `scikit-learn`.

5.1.1 Important precision about the evaluation metric used in this practical

In this practical, we will use the *Mean Squared Error* (MSE) as a convenient shorthand for the estimated *Mean Integrated Squared Error* (MISE). Indeed, in practice, we rarely know the true MISE and instead rely on estimates obtained via techniques like K -fold cross-validation or leave-one-out (LOO) cross-validation. So keep in mind that when we refer to the MSE in these contexts, what we are really computing is an estimation of MISE from our data.

5.1.2 Data preprocessing overview

Data preprocessing is the foundation of any successful machine learning project. Many Machine Learning models see their performances degraded when trained on poorly preprocessed data. Key steps include:

- **Handling Missing Values:** Missing data can bias results if not addressed. This problem affects many models—such as linear models, support vector machines (SVMs), and neural networks—but it is particularly critical for distance-based algorithms like KNN where missing feature values can disrupt the calculation of distances, leading to inaccurate outcomes. Common strategies for handling missing values include:
 - *Dropping* rows/columns with missing data.
 - *Imputing* missing data using simple statistics (mean, median) or more sophisticated methods like `KNNImputer`.
- **Normalization / Scaling:** Many algorithms assume features are on similar scales. Without proper scaling, features with larger numerical ranges may dominate those with smaller ranges, potentially biasing the model's results. Common scaling strategies include:
 - *Standardization:* Transform features to have zero mean and unit variance.
 - *Min-Max Scaling:* Scale features to a fixed $[0, 1]$ range.
- **Dealing with Categorical Variables:** Categorical variables must be converted to a numeric representation that preserves meaningful differences. Common strategies for dealing with categorical variables include:
 - *Label Encoding:* Assign an integer to each category (useful if there is an ordinal relationship).
 - *One-Hot Encoding (Dummy Encoding):* Create a new binary column for each category (common for nominal features). This approach can cause a significant increase in input dimensionality (see the curse of dimensionality).

5.2 K-Nearest Neighbors

Did you know ? The K -Nearest Neighbors algorithm was first proposed in .

The *K-Nearest Neighbors* (KNN) algorithm is a local modeling approach that infers its prediction from the data points closest (in some metric) to a query. Suppose we have a training set of inputs $\{x_i\}$ with associated labels or targets $\{y_i\}$. For a new query point q :

1. Distance computation: Compute the distance (e.g., Euclidean) between q and each training example x_i .
2. Ranking: Sort or rank the training points by increasing distance to q .
3. Neighborhood selection: Identify the subset of the K nearest neighbors $\{x_{[1]}, \dots, x_{[K]}\}$ where $y_{[i]}$ denotes the label of neighbor $x_{[i]}$.
4. Regression or classification: KNN can be viewed as a local estimator of the conditional expectation $\mathbb{E}[y|x = q]$.

For regression tasks, the simplest approach is to average the targets of the K nearest neighbors:

$$\hat{y}(q) = \frac{1}{K} \sum_{i=1}^K y_{[i]}.$$

Alternatively, a local linear model may be used:

$$\hat{y}(q) = \hat{a}^T q + \hat{b},$$

where $\hat{a}$ and $\hat{b}$ are estimated (locally) by least-squares on the neighbors.

For classification, one can estimate the conditional probability of belonging to a specific class (e.g., class “1”) by the proportion of neighbors labeled “1”:

$$\hat{p}_1(q) = \frac{1}{K} \sum_{i=1}^K y_{[i]}.$$

A threshold (e.g., 0.5) can then be used to decide the class label.

The key hyperparameter here is K . A *smaller* K reduces bias but increases variance, while a *larger* K produces a smoother model at the risk of higher bias. KNN’s performance also heavily depends on the distance metric, especially when features have different scales or include categorical variables (which may require specialized encodings or distance definitions).

5.3 Regression Trees

Regression trees rely on a tree-based structure of *internal nodes* (where decisions are made) and *terminal nodes* (leaves), which partition the input space into mutually exclusive regions, each associated with a simple (often constant) local model. Its construction begins with a tree growing phase: starting from the root node that contains all data, we recursively choose a split that best reduces the empirical error. Specifically, for a node t containing $N(t)$ samples, we define

$$R_{\text{emp}}(t) = \min_{\alpha_t} \sum_{i=1}^{N(t)} L(y_i, h_t(x_i, \alpha_t)),$$

where L is typically the squared error $(y - \hat{y})^2$ for regression. Given a potential split s dividing t into children t_l and t_r , we consider the decrease in empirical error:

$$\Delta E(s, t) = R_{\text{emp}}(t) - (R_{\text{emp}}(t_l) + R_{\text{emp}}(t_r)).$$

We choose the split s^* maximizing ΔE , partition the dataset accordingly, and repeat the procedure recursively until no further improvement is found or a stopping criterion is met. This exhaustive splitting often yields a very large tree that overfits the data.

To address overfitting, a cost-complexity pruning procedure is commonly used. Introducing a complexity parameter $\lambda \geq 0$ that penalizes the number of leaf nodes, we define the cost-complexity measure:

$$R_\lambda(T) = R_{\text{emp}}(T) + \lambda |T|,$$

where $|T|$ is the number of terminal nodes (leaves) in tree T .

- By gradually increasing λ , we obtain a *sequence* of subtrees:

$$T_{\max} \supset T_{L-1} \supset \dots \supset T_2 \supset T_1.$$

Each has fewer leaves. We consider all admissible subtrees $T_t \subset T$ of the large tree and compute the smallest

$$\lambda_t = \frac{R_{\text{emp}}(T) - R_{\text{emp}}(T_t)}{|T| - |T_t|}$$

that yields a lower cost. We select the subtree that minimizes $R_\lambda(T)$ (the best balances between empirical error and model complexity). The final tree structure is typically chosen via cross-validation or held-out validation to find the best λ . A more visual representation of tree-based models, an animated version of decision trees (classification) can be found [here](#).

Random Forest have been first presented in .

5.4 Random Forests

A Random Forest (RF), is an ensemble learning technique designed to reduce the variance of decision trees by combining two main ideas:

1. Bootstrap Sampling (Bagging): We generate B *bootstrap* datasets, each by sampling (with replacement) from the original training data.
2. Random Feature Selection (Feature Bagging): At each split, a *random subset* of $n' < n$ features is chosen, and the best split is found *only among those n' features*.

Hence, each tree $h_b(\mathbf{x}, \alpha_b)$ is built from a *different* resampled dataset and uses only a random subset of features at each split. As a result, the trees are more decorrelated and often are *not pruned* heavily.

Formally, for a regression task:

$$h_{\text{rf}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B h_b(\mathbf{x}, \alpha_b),$$

i.e., the average of all tree predictions. (In classification, we take the majority vote.)

5.4.1 Why does this reduce variance?

Suppose each tree h_b has variance σ^2 and they have pairwise correlation ρ . The variance of the RF predictor h_{rf} can be approximated as:

$$\text{Var}[h_{\text{rf}}] \approx \frac{(1 - \rho) \sigma^2}{B} + \rho \sigma^2,$$

which shows that increasing B (the number of trees) reduces the first term, while lowering the correlation ρ among trees reduces overall variance. Random feature selection helps reduce ρ , because each tree sees a different subset of features.

5.4.2 Main Hyperparameters

1. B : the number of trees in the forest.
2. n' : the number of randomly selected features at each split (often $\sqrt{n}$ by default in classification).
3. Tree parameters: e.g., maximum depth, minimum samples per leaf, etc.

In practice, random forests typically provide strong performance out of the box and are less sensitive to hyperparameter tuning compared to single trees or more complex models. They also allow computing a useful estimate of feature importance by measuring, for each variable, how much it contributes to the cost function improvement across all splits in all trees.

5.5 Feature Importances

Most `scikit-learn` models allows you to estimate how each feature contributes to the final prediction. For example, for tree-based models, both `DecisionTreeRegressor` and `RandomForestRegressor` in `scikit-learn` store a `feature_importances_` attribute after fitting. This gives an estimate of how much each feature contributed to reducing the split criterion (e.g., MSE).

5.6 Scikit-learn Pipelines

Pipelines in `scikit-learn` let us combine data preprocessing (e.g., imputation, scaling) and model training (e.g., a random forest) into a single workflow object. Below is a simple example:

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.impute import SimpleImputer
```

```
example_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("regressor", LinearRegression())
])
```

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.ensemble import RandomForestRegressor

class AddConstantTransformer(BaseEstimator, TransformerMixin):
    """
    Custom transformer that adds a constant value to all features.
    """
    def __init__(self, constant=0.0):
        self.constant = constant

    def fit(self, X, y=None):
        return self

    def transform(self, X, y=None):
        return X + self.constant
```

```
custom_pipeline = Pipeline([
```

(continues on next page)

(continued from previous page)

```

("imputer", SimpleImputer(strategy="mean")),
("custom_add_constant", AddConstantTransformer(constant=1.0)),
("scaler", StandardScaler()),
("regressor", RandomForestRegressor(n_estimators=10, random_state=42))
])

```

5.7 Exercises

5.7.1 California Housing

1. Load the well-known *California Housing* regression dataset using the following function:

```
from sklearn.datasets import fetch_california_housing
```

2. Explore the dataset (distribution, summary statistics).
3. Check for missing values. If the dataset does not contain missing values, artificially introduce some for the purpose of this exercise. Then, handle missing values by:
 - Dropping them (simple approach).
 - Imputing them (mean or median).
 - Using a more sophisticated method (e.g., `KNNImputer`).
4. Normalize the features (using e.g., `StandardScaler`). Remember that, as all processing steps, scaling is fit on the training set (not on the entire dataset) to avoid biasing the test set!
5. Preserve original vs. processed versions of the data for comparison.

Solution

Dataset and exploration

For this exercise, we will use the California Housing dataset from `sklearn.datasets.fetch_california_housing`. It contains features related to California housing data, and the target is the average house value.

Alternatively, if you would like test your solution on other datasets, `scikit-learn` provides other built-in datasets, such as:

- Diabetes (a small dataset with 10 features for a regression task on disease progression).
- Linnerud (exercise/physiological data, a small multi-output regression problem). If you want a dataset with even more features or complexity, you could consider Ames Housing or Kaggle House Price, both of which have many categorical and numeric features but may require additional steps to download or preprocess. You can also explore the UCI Machine Learning Repository for a wide range of tabular datasets.

```

import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing

cal_housing = fetch_california_housing()
X_original = pd.DataFrame(cal_housing.data, columns=cal_housing.feature_names)

```

(continues on next page)

(continued from previous page)

```

y_original = pd.Series(cal_housing.target, name='Target')

print("Shape of the feature matrix:", X_original.shape)
print("Shape of the target vector:", y_original.shape)

X_original.info()
display(X_original.describe())

```

```

Shape of the feature matrix: (20640, 8)
Shape of the target vector: (20640,)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 8 columns):
#   Column          Non-Null Count  Dtype
---  -
0   MedInc          20640 non-null  float64
1   HouseAge        20640 non-null  float64
2   AveRooms        20640 non-null  float64
3   AveBedrms       20640 non-null  float64
4   Population      20640 non-null  float64
5   AveOccup        20640 non-null  float64
6   Latitude        20640 non-null  float64
7   Longitude       20640 non-null  float64
dtypes: float64(8)
memory usage: 1.3 MB

```

	MedInc	HouseAge	AveRooms	AveBedrms	Population \
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.870671	28.639486	5.429000	1.096675	1425.476744
std	1.899822	12.585558	2.474173	0.473911	1132.462122
min	0.499900	1.000000	0.846154	0.333333	3.000000
25%	2.563400	18.000000	4.440716	1.006079	787.000000
50%	3.534800	29.000000	5.229129	1.048780	1166.000000
75%	4.743250	37.000000	6.052381	1.099526	1725.000000
max	15.000100	52.000000	141.909091	34.066667	35682.000000

	AveOccup	Latitude	Longitude
count	20640.000000	20640.000000	20640.000000
mean	3.070655	35.631861	-119.569704
std	10.386050	2.135952	2.003532
min	0.692308	32.540000	-124.350000
25%	2.429741	33.930000	-121.800000
50%	2.818116	34.260000	-118.490000
75%	3.282261	37.710000	-118.010000
max	1243.333333	41.950000	-114.310000

#####. Handling Missing Values Although the California Housing dataset typically does not contain missing values, let's artificially introduce some missing values for demonstration. We will then show three approaches:

1. Dropping rows with missing values.
2. Imputing with mean or median.
3. `KNNImputer` or another advanced method.

An advanced method: `KNNImputer` `KNNImputer` uses a K-Nearest Neighbors approach to fill missing entries: for each row that has missing features, it finds the nearest k rows (in feature space) that do not have missing values in those

columns, and uses their values (e.g., average) to impute the missing ones. This can often be more accurate than a simple mean or median if the data has local structure—rows that are similar in some features are likely also similar in the others.

```
from sklearn.model_selection import train_test_split
from sklearn.impute import KNNImputer

missing_mask = np.random.rand(*X_original.shape) < 0.01
X_missing = X_original.copy()
X_missing[missing_mask] = np.nan

print("--- Number of missing values (artificial) ---")
print(X_missing.isnull().sum())

X_train_full, X_test_full, y_train_full, y_test_full = train_test_split(
    X_missing, y_original, test_size=0.2, random_state=42
)

X_train_drop = X_train_full.dropna(axis=0)
y_train_drop = y_train_full.loc[X_train_drop.index]
X_test_drop = X_test_full.dropna(axis=0)
y_test_drop = y_test_full.loc[X_test_drop.index]

print("--- After Dropping Missing Values ---")
print("Training set size:", X_train_drop.shape)
print("Test set size:", X_test_drop.shape)

from sklearn.impute import SimpleImputer

mean_imputer = SimpleImputer(strategy='mean')
X_train_mean = mean_imputer.fit_transform(X_train_full)
X_test_mean = mean_imputer.transform(X_test_full)

median_imputer = SimpleImputer(strategy='median')
X_train_median = median_imputer.fit_transform(X_train_full)
X_test_median = median_imputer.transform(X_test_full)

knn_imputer = KNNImputer(n_neighbors=3)
X_train_knn = knn_imputer.fit_transform(X_train_full)
X_test_knn = knn_imputer.transform(X_test_full)
```

```
--- Number of missing values (artificial) ---
MedInc      220
HouseAge    212
AveRooms    216
AveBedrms   206
Population  196
AveOccup    195
Latitude    208
Longitude   202
dtype: int64
--- After Dropping Missing Values ---
Training set size: (15236, 8)
Test set size: (3816, 8)
```

Dealing with Categorical Values

In this dataset, we have only numerical features. However, real-world datasets often include categorical variables. Two popular encoding strategies are:

1. Label Encoding: Each category is assigned an integer.
2. One-Hot Encoding (Dummy Encoding): Create a new binary column for each category.

Below is a small example using a synthetic dataframe containing a categorical column.

```
from sklearn.preprocessing import LabelEncoder

example_df = pd.DataFrame({
    "Size": ["Small", "Medium", "Large", "Medium", "Small"],
    "Weight": [1.0, 2.3, 5.5, 2.0, 1.1]
})

label_encoder = LabelEncoder()
example_df["Size_label"] = label_encoder.fit_transform(example_df["Size"])

one_hot_df = pd.get_dummies(example_df, columns=["Size"], prefix="Size")
```

Original DataFrame with Label Encoding:

```
display(example_df)
```

	Size	Weight	Size_label
0	Small	1.0	2
1	Medium	2.3	1
2	Large	5.5	0
3	Medium	2.0	1
4	Small	1.1	2

DataFrame with One-Hot Encoding:

```
display(one_hot_df)
```

	Weight	Size_label	Size_Large	Size_Medium	Size_Small
0	1.0	2	False	False	True
1	2.3	1	False	True	False
2	5.5	0	True	False	False
3	2.0	1	False	True	False
4	1.1	2	False	False	True

We'll stick to numeric data for the main practical, but this is how you'd handle categorical features.

Normalization/Scaling

Scaling is critical for distance-based methods. We will use `StandardScaler`, which transforms each feature to have zero mean and unit variance, but note that `MinMaxScaler` is also a standard.

Remember: Always fit the scaler on the training set, then apply the same transformation to the test set. This prevents data leakage and preserves the integrity of the test data.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_mean)
X_test_scaled = scaler.transform(X_test_mean)

X_train_final = X_train_scaled
X_test_final = X_test_scaled
y_train_final = y_train_full
y_test_final = y_test_full
```

5.7.2 Vanilla KNN Implementation

1. Split your data into training and test sets (we should have already done that in the previous exercise).
2. Implement a vanilla KNN regressor from scratch (no `scikit-learn` for this part).
3. Choose a value of k (e.g., 3, 5, 7).
4. Compare predictions with the true values using Mean Squared Error (MSE).

Solution

On this graph, we clearly see that there is an optimal value for the number of neighbors to use: a value too small increases the variance of the predictions while a value too high increases the bias. The optimal value, of course, depends on the problem definition and on the training data.

```
import matplotlib.pyplot as plt
plt.style.use(['seaborn-v0_8-muted', 'practicals.mplstyle'])
from sklearn.metrics import mean_squared_error
from tqdm.auto import tqdm

def euclidean_distance(x1, x2):
    return np.sqrt(np.sum((x1 - x2) ** 2))

def knn_regressor(X_train, y_train, X_test, k=3):
    """
    A simple KNN regressor (manual implementation).
    - X_train: training features (numpy array)
    - y_train: training targets (numpy array)
    - X_test: test features (numpy array)
    - k: number of neighbors to consider
    """
```

(continues on next page)

(continued from previous page)

```

predictions = []
for test_point in tqdm(X_test):
    distances = [euclidean_distance(test_point, x_train) for x_train in X_train]
    k_indices = np.argsort(distances)[:k]
    prediction = np.mean(y_train[k_indices])
    predictions.append(prediction)
return np.array(predictions)

from scipy.spatial import distance

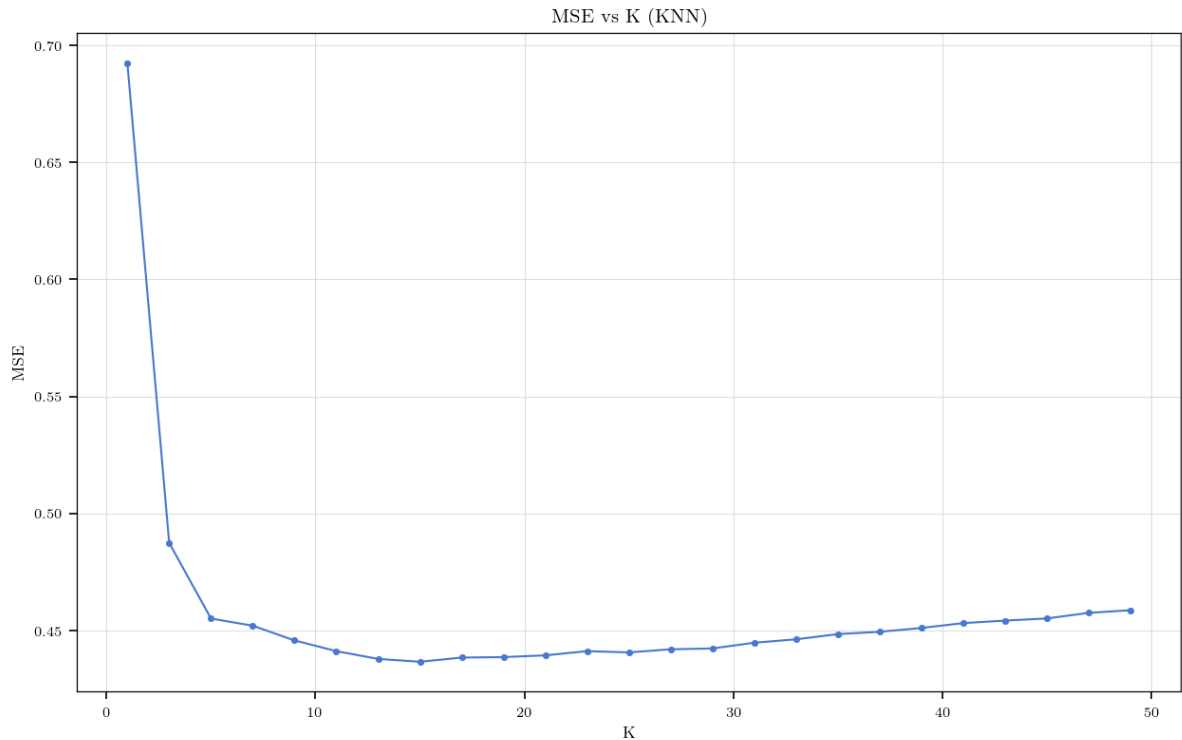
def knn_regressor_fast(X_train, y_train, X_test, k=3):
    """
    A faster KNN regressor using vectorized distance computation.
    """
    dists = distance.cdist(X_test, X_train, metric='euclidean')
    k_indices = np.argpartition(dists, kth=k, axis=1)[:k]
    predictions = np.mean(y_train[k_indices], axis=1)
    return predictions

mses_knn = []
for k in tqdm(range(1, 51, 2)):
    y_pred_knn = knn_regressor_fast(X_train_final, y_train_final.values, X_test_final,
    ↪k=k)
    mses_knn.append(mean_squared_error(y_test_final, y_pred_knn))

plt.figure(figsize=(10, 6))
plt.plot(list(range(1, 51, 2)), mses_knn, marker='o')
plt.title("MSE vs K (KNN)")
plt.xlabel("K")
plt.ylabel("MSE")
plt.grid("on")
plt.show()

```

0% | 0/25 [00:00<?, ?it/s]



5.7.3 Regression Trees and Random Forests

1. Train a regression tree on the training set.
2. Train a random forest on the training set.
3. Evaluate both models using MSE.
4. Compare their performance with your KNN model.
5. Determine how modifying the `max_depth` (for the regression trees) and `n_estimators` (for the random forests) impacts the performances of the model. Plot these results graphically.
6. Plot the features importance of the best model with the optimal parameters

Solution: Implementation with `scikit-learn`

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

tree_reg = DecisionTreeRegressor(max_depth=9, random_state=42)
tree_reg.fit(X_train_final, y_train_final)
y_pred_tree = tree_reg.predict(X_test_final)
mse_tree = mean_squared_error(y_test_final, y_pred_tree)

forest_reg = RandomForestRegressor(n_estimators=60, random_state=42)
forest_reg.fit(X_train_final, y_train_final)
y_pred_forest = forest_reg.predict(X_test_final)
mse_forest = mean_squared_error(y_test_final, y_pred_forest)
```

(continues on next page)

(continued from previous page)

```
print("Comparison of MSE:")
print(f"KNN (k=13): {mean_squared_error(y_test_final, knn_regressor_fast(X_train_
    ↪final, y_train_final.values, X_test_final, k=13)):.4f}")
print(f"Decision Tree: {mse_tree:.4f}")
print(f"Random Forest: {mse_forest:.4f}")
```

Comparison of MSE:

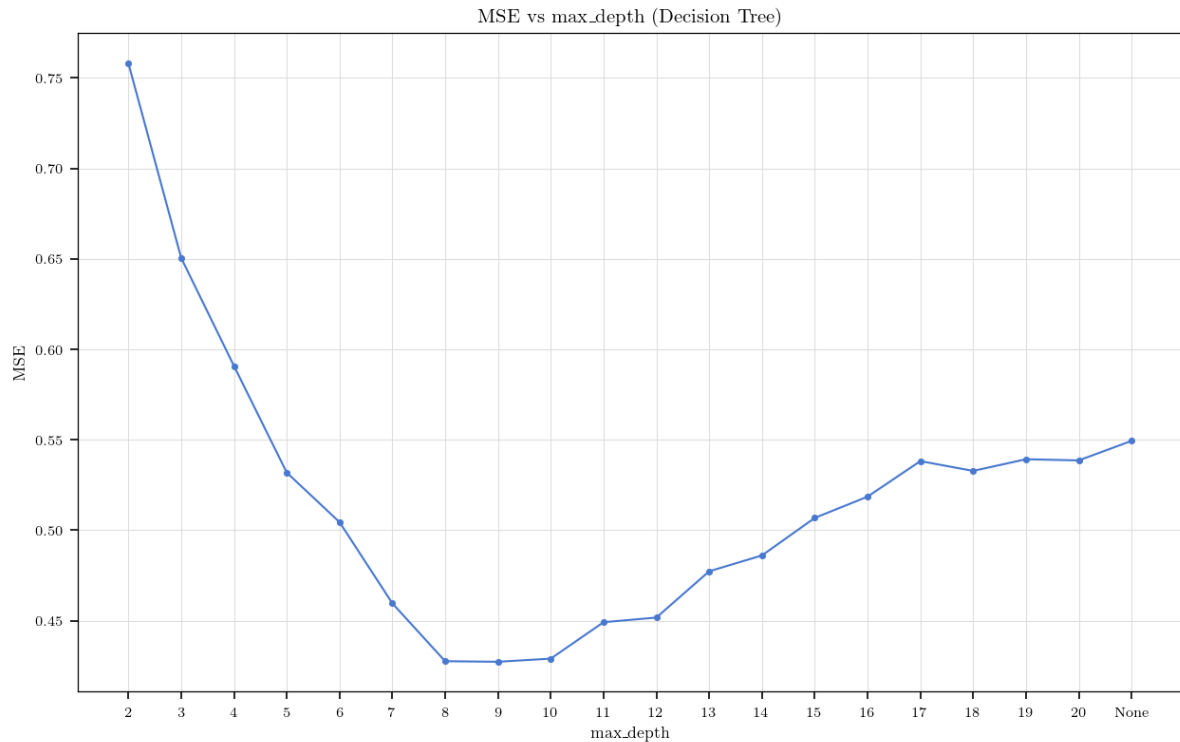
```
KNN (k=13): 0.4378
Decision Tree: 0.4273
Random Forest: 0.2698
```

We explore the different hyper-parameters. We can examine how `max_depth` (the maximum number of splits from the root to the leaf) affects performance.

```
depths = list(range(2, 21)) + [None]
mse_values_tree = []

for d in depths:
    dt = DecisionTreeRegressor(max_depth=d, random_state=42)
    dt.fit(X_train_final, y_train_final)
    y_pred_dt = dt.predict(X_test_final)
    mse_values_tree.append(mean_squared_error(y_test_final, y_pred_dt))

plt.figure(figsize=(10, 6))
plt.plot([str(d) for d in depths], mse_values_tree, marker='o')
plt.title("MSE vs max_depth (Decision Tree)")
plt.xlabel("max_depth")
plt.ylabel("MSE")
plt.grid("on")
plt.show()
```



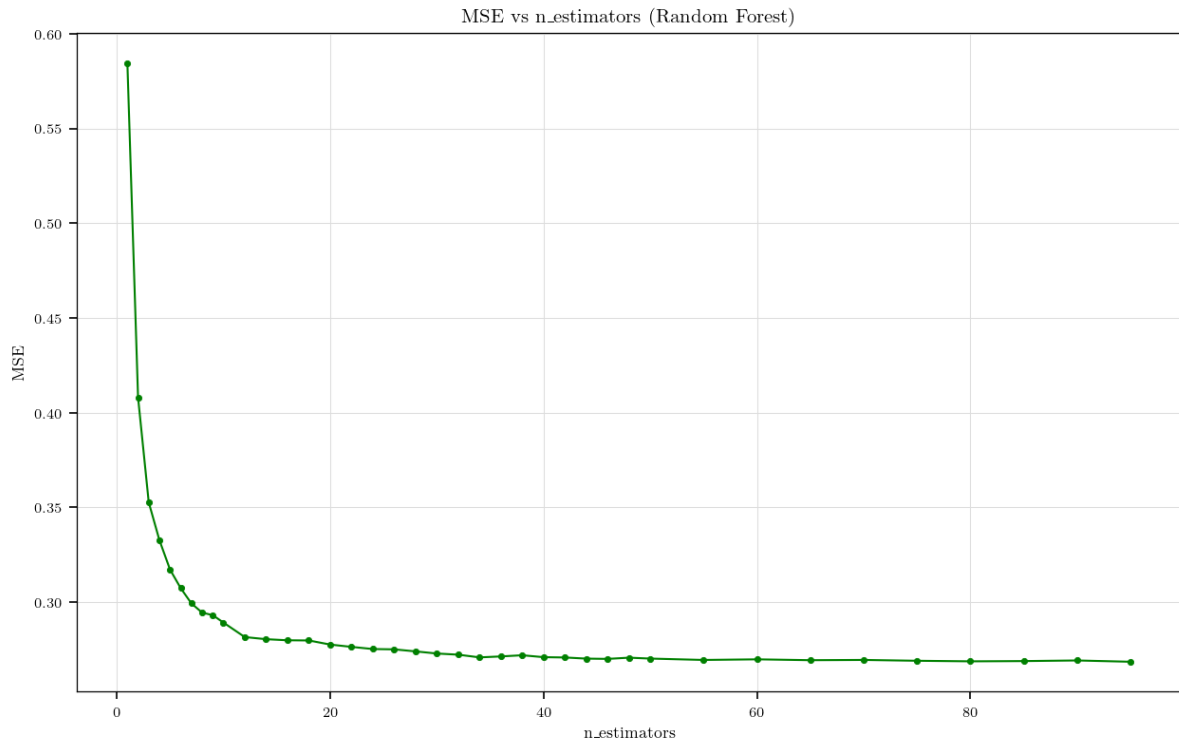
Similarly, we can see how `n_estimators` (the number of trees in the forest) impacts the MSE.

```
estimators_range = list(range(1, 10))+list(range(10, 50, 2))+list(range(50, 100, 5))
mse_values_rf = []

for n in tqdm(estimators_range):
    rf = RandomForestRegressor(n_estimators=n, random_state=42)
    rf.fit(X_train_final, y_train_final)
    y_pred_rf = rf.predict(X_test_final)
    mse_values_rf.append(mean_squared_error(y_test_final, y_pred_rf))

plt.figure(figsize=(10, 6))
plt.plot(estimators_range, mse_values_rf, marker='o', color='green')
plt.title("MSE vs n_estimators (Random Forest)")
plt.xlabel("n_estimators")
plt.ylabel("MSE")
plt.grid("on")
plt.show()
```

0% | 0/39 [00:00<?, ?it/s]



Feature importance with the best models

Based on the experiments above, let's define user-chosen parameters for the *best* (or at least improved) Decision Tree and Random Forest, then compute and display their feature importances.

Feel free to update `max_depth` and `n_estimators` according to the results you obtained above. Later, we will see how this can be used as a *feature selection* method.

```
max_depth = 9
n_estimators = 60

best_tree = DecisionTreeRegressor(max_depth=max_depth, random_state=42)
best_tree.fit(X_train_final, y_train_final)

best_forest = RandomForestRegressor(n_estimators=n_estimators, random_state=42)
best_forest.fit(X_train_final, y_train_final)

feature_names = X_original.columns

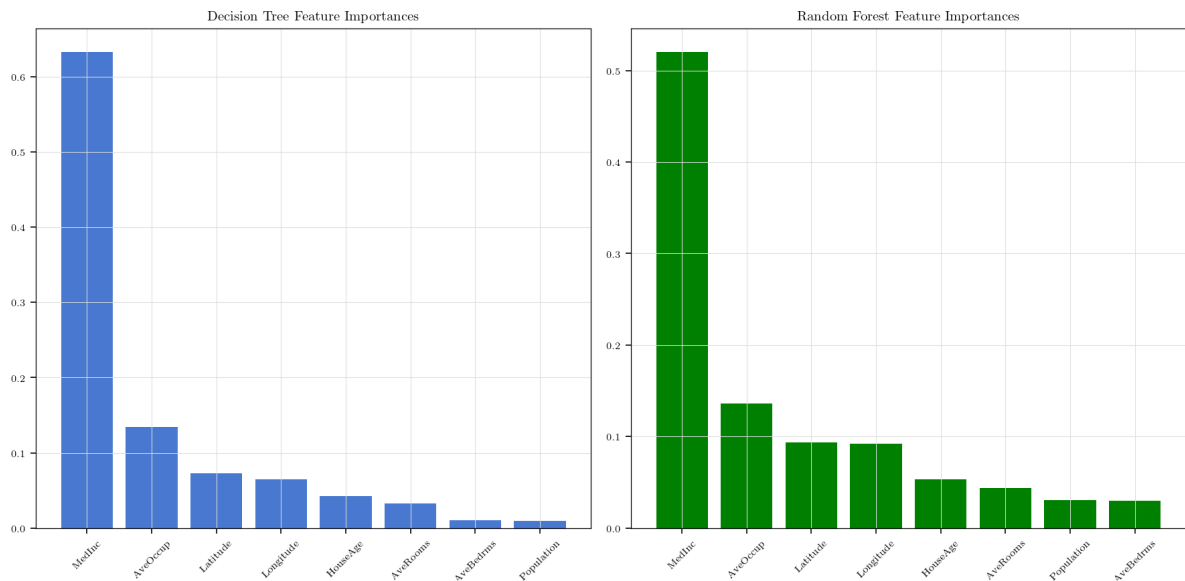
tree_importances = best_tree.feature_importances_
forest_importances = best_forest.feature_importances_

tree_sorted_idx = np.argsort(tree_importances)[::-1]
forest_sorted_idx = np.argsort(forest_importances)[::-1]
```

(continues on next page)

(continued from previous page)

```
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Decision Tree Feature Importances")
plt.bar(range(len(tree_importances)), tree_importances[tree_sorted_idx], align='center',
        color='blue')
plt.xticks(range(len(tree_importances)), feature_names[tree_sorted_idx], rotation=45)
plt.grid("on")
plt.subplot(1, 2, 2)
plt.title("Random Forest Feature Importances")
plt.bar(range(len(forest_importances)), forest_importances[forest_sorted_idx], align='center',
        color='green')
plt.xticks(range(len(forest_importances)), feature_names[forest_sorted_idx], rotation=45)
plt.grid("on")
plt.tight_layout()
plt.show()
```



5.7.4 Building a Pipeline

1. Define a pipeline that applies the preprocessing steps (e.g., imputation, scaling) and the best-performing model (from Exercise 3).
2. Fit this pipeline on the training data and evaluate on the test data.
3. Observe how `scikit-learn` handles transformations during `fit` and `predict`.

Solution

Below, we define a pipeline that:

1. Imputes missing values with mean.
2. Scales the features.
3. Trains a random forest regressor.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error

pipeline_rf = Pipeline([
    ("imputer", SimpleImputer(strategy="mean")),
    ("scaler", StandardScaler()),
    ("regressor", RandomForestRegressor(n_estimators=60, random_state=42))
])

pipeline_rf.fit(X_train_full, y_train_full)
y_pred_pipe = pipeline_rf.predict(X_test_full)
mse_pipeline_rf = mean_squared_error(y_test_full, y_pred_pipe)
print(f"Pipeline (Random Forest) MSE: {mse_pipeline_rf:.4f}")
```

```
Pipeline (Random Forest) MSE: 0.2698
```

5.7.5 Additional exercise: bias–variance trade-off in regression with decision trees

In this exercise, you will investigate the bias–variance trade-off for decision tree regressors of varying complexity. Specifically, you will perform simulations to estimate how the depth of a decision tree influences its performance.

1. Select a nonlinear, moderately complex target function e.g.:

$$f(x) = \sin(4\pi x) + w$$

or

$$f(x) = \sin(2\pi x_1 x_2 x_3) + w$$

2. Evaluate three decision tree regressors, each with a different complexity level:

- Shallow tree: depth = 2
- Medium-depth tree: depth = 6
- Deep tree: depth = 20

3. Generate training datasets composed of $n_{\text{train}} = 400$ random samples:

$$X_{\text{train}} \sim U(0, 1), \quad y_{\text{train}} = f(X_{\text{train}}) + w$$

Here, noise w follows a Gaussian distribution with mean 0 and standard deviation 0.3:

$$w \sim \mathcal{N}(0, 0.3^2)$$

5. Repeat the entire training and evaluation process $n_{\text{runs}} = 100$ times. For each run, compute and store the mean squared error (MSE) using 10-fold cross-validation on the training set.

6. Evaluate your models on a test set X_{test} evenly spaced in the interval $[0, 1]$ (e.g., 400 points).
 - For each test point, compute predictions across all 100 simulation runs.
 - Using these predictions, compute
 - MSE (CV): The average cross-validation mean squared error (which we will call $\widehat{\text{MISE}}_{\text{kfold}}$) obtained across the 100 runs.
 - Bias²: Based on how far the *average prediction* per sample is from the sample's true label.
 - Variance: Based on the variability of predictions per sample around that sample's mean prediction.
 - MSE (true): the approximation $\text{MSE} \approx \text{Bias}^2 + \text{Variance}$.
7. Discussion

Analyze your results to illustrate how model complexity (tree depth) affects the balance between bias and variance:

 - Identify which depth leads to underfitting (high bias, low variance).
 - Identify which depth leads to overfitting (low bias, high variance).

Solution

```
import pandas as pd
from sklearn.model_selection import cross_val_score, KFold

n_train = 400
n_runs = 100
noise_std = 0.3
depths = [2, 6, 20]
K = 10

def f_func(x):
    return np.sin(4 * np.pi * x).ravel()

X_test = np.linspace(0, 1, 400)[: , None]
y_true = f_func(X_test)

predictions = {depth: np.zeros((n_runs, X_test.shape[0])) for depth in depths}
cv_mse_values = {depth: [] for depth in depths}

for i in range(n_runs):
    X_train = np.random.rand(n_train, 1)
    y_train = f_func(X_train) + np.random.normal(0, noise_std, size=n_train)

    for depth in depths:
        model = DecisionTreeRegressor(max_depth=depth)
        model.fit(X_train, y_train)

        predictions[depth][i, :] = model.predict(X_test)

    cv_scores = cross_val_score(
        DecisionTreeRegressor(max_depth=depth),
        X_train, y_train,
        scoring='neg_mean_squared_error',
        cv=KFold(n_splits=K, shuffle=True, random_state=1)
    )
    cv_mse = -cv_scores.mean()
    cv_mse_values[depth].append(cv_mse)
```

(continues on next page)

(continued from previous page)

```

noise_var = noise_std**2
metrics = {}

for depth in depths:
    mean_pred = np.mean(predictions[depth], axis=0)
    bias_sq = np.mean((mean_pred - y_true)**2)
    variance = np.mean(np.var(predictions[depth], axis=0))
    true_mse = bias_sq + variance + noise_var
    cv_mse_mean = np.mean(cv_mse_values[depth])

    metrics[depth] = {
        "Bias^2": bias_sq,
        "Variance": variance,
        "True MSE": true_mse,
        "CV MSE": cv_mse_mean
    }

metrics_df = pd.DataFrame(metrics).T
metrics_df.index.name = "Tree Depth"
metrics_df = metrics_df.round(4)
print(metrics_df)

```

Tree Depth	Bias^2	Variance	True MSE	CV MSE
2	0.1799	0.1083	0.3783	0.3839
6	0.0009	0.0325	0.1235	0.1259
20	0.0009	0.0902	0.1812	0.1807

NEURAL NETWORKS FOR REGRESSION

6.1 Introduction

This practical has not as intention to reintroduce entirely the notion of Neural Network. We redirect the student to Chapter 8 of [the handbook](#) for a formal introduction to those tools.

However, we aim at giving a practical sight on what Neural Networks actually are. Concretely, Neural Networks form a family of approximations of functions. Generally speaking, most Neural Networks make the hypothesis of non-linearity in the relationship between the inputs and the outputs. The design of the Neural Network involves, non-exhaustively, the definition of the input and output shape, the size and the relationships between the hidden layers, and the non-linear activation functions for each of the layers.

One main difference between *tensors* and *matrices*, mathematically speaking, is the fact that the tensors do not depend on the chosen basis, while matrices do. Matrices can serve as good representations of tensors. Note that tensors can be seen members of vector spaces. However, this is not important for the covered subject, nor for the practical tensorial algebra that is used in Neural Networks design and optimization.

In most modern Neural Networks frameworks, the design of a Neural Network also implies the definition of the optimization algorithm and the definition of some hyperparameters. In most of the existing libraries, the data needs to be transformed to *tensors*, which, despite the fact that they have another mathematical definition, behave really closely to the NumPy arrays that we have presented earlier. Modern computing architectures take into account the usual operations computed on tensors in order to optimize them, which, like it was the case for the arrays and the GPU's (Graphical Processing Units), gives birth to the TPU's (Tensor Processing Units).

In this practical, we will present one of the most used framework for Neural Networks design and optimization, PyTorch, to introduce two famous Neural Network families: the *Multi-Layers Perceptrons* (MLP), where layers are densely connected with the next one, and *Convolutional Neural Networks* (CNN), which take advantage of spatial (or temporal) proximity of the data's components.

As usual, this line can be skipped if the required libraries are already installed on your system.

```
#!/pip install torch torchsummary
```

Let us define some basic imports and a pprint function.

```

import numpy as np
import torch
from IPython.display import display, Math
from numpyarray_to_latex.jupyter import to_ltx
import matplotlib.pyplot as plt
plt.style.use(['seaborn-v0_8-muted', 'practicals.mplstyle'])
def pprint(*args):
    res = ""
    for i in args:
        if type(i) == np.ndarray:
            res += to_ltx(i, brackets='[]')
        elif type(i) == torch.Tensor:
            res += to_ltx(i.detach().cpu().numpy(), brackets='[]',)
        elif type(i) == str:
            res += i
    display(Math(res))

```

6.2 Data Preprocessing for Neural Networks

6.2.1 Normalization and Standardization

Gradient-based optimization of neural networks often becomes unstable if different features vary widely in scale. Hence, **Sec. 8.1** of the handbook (and earlier practicals) reiterate that scaling inputs to zero mean and unit variance—or to a small bounded range—is essential. This ensures the gradients have more balanced magnitudes across parameters, which helps the optimizer converge more reliably. Typically, normalization and standardization follow those steps:

1. Compute the empirical mean and standard deviation of each input dimension:

$$\mu_j = \frac{1}{N} \sum_{i=1}^N x_{ij}, \quad \sigma_j = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_j)^2}.$$

2. Transform (standardise) the data as

$$x_{ij}^{(std)} = \frac{x_{ij} - \mu_j}{\sigma_j}.$$

3. Use the same μ_j and σ_j for the validation and test sets to maintain consistency.

6.2.2 Input Data Shaping

- **For MLPs:** Usually arrange training data into a matrix of shape (batch, n), where n is the number of input features.
- **For 2D-CNNs:** Reshape data into a 4D tensor of shape (batch, channels, height, width). Convolutional layers exploit the spatial dimensions for images or other grid-like inputs.

6.3 Feed-Forward Neural Networks (MLPs)

6.3.1 Architecture and Notation

An MLP (multi-layer perceptron, or feed-forward network) with L layers (including hidden and output layers) maps an input vector $\mathbf{x} \in \mathbb{R}^n$ to an output (for regression, typically $\hat{y} \in \mathbb{R}$ or $\mathbb{R}^m$) through successive affine transformations and nonlinear activations. In the notation of **Eqs.** (8.1.2)–(8.1.5):

1. **Layer 1:**

$$A^{(1)} = Z^{(0)}W^{(1)} + b^{(1)}, \quad Z^{(1)} = g^{(1)}(A^{(1)}),$$

where $Z^{(0)} = \mathbf{x}$ is the input and $W^{(1)}$ is an $(n \times h_1)$ weight matrix if the first hidden layer has h_1 units.

2. **Layer 2** (or subsequent hidden layers):

$$A^{(2)} = Z^{(1)}W^{(2)} + b^{(2)}, \quad Z^{(2)} = g^{(2)}(A^{(2)}),$$

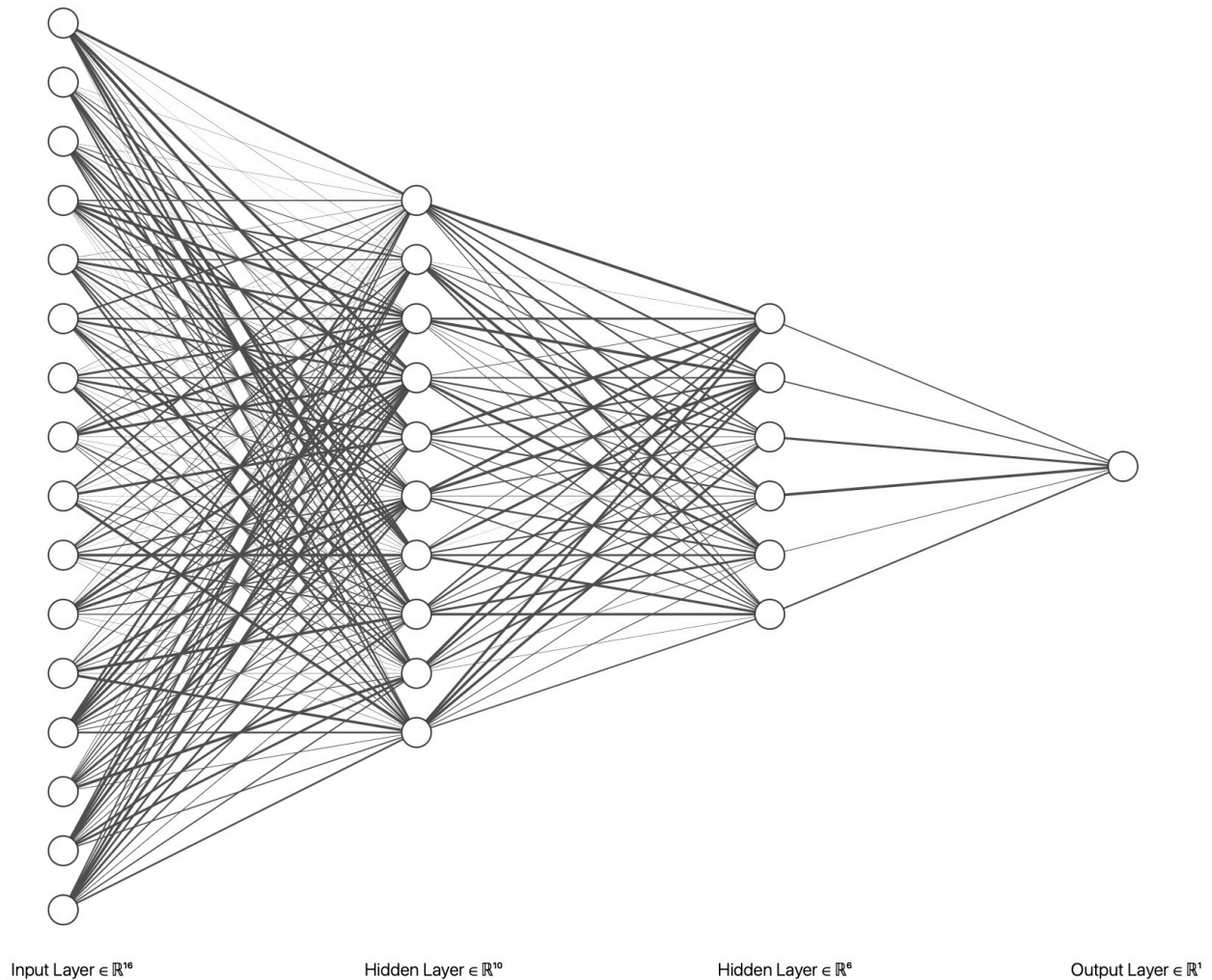
continuing similarly up to layer $L - 1$.

3. **Output Layer** ($l = L$): For regression with a linear output,

$$A^{(L)} = Z^{(L-1)}W^{(L)} + b^{(L)}, \quad Z^{(L)} = A^{(L)}.$$

If the output dimension is 1, $W^{(L)}$ is $(h_{L-1} \times 1)$. For vector outputs, adjust accordingly.

An example of representation of a MLP made of layers of size (16, 10, 6, 1), made with [this tool](#), is given here:



6.4 Convolutional Neural Networks (CNNs)

Though the practical focuses on regression, CNNs (p. 198–199) illustrate deep architectures that handle grid-structured data (e.g., images). Main ideas:

1. **Local Connectivity:** Each neuron in a convolutional layer only connects to a local patch of the input.
2. **Shared Weights:** A small kernel (filter) is “convolved” across the entire image or feature map.
3. **Translation Equivariance:** The same filter is applied at all spatial locations.

6.4.1 Convolution Operation

A single 2D filter:

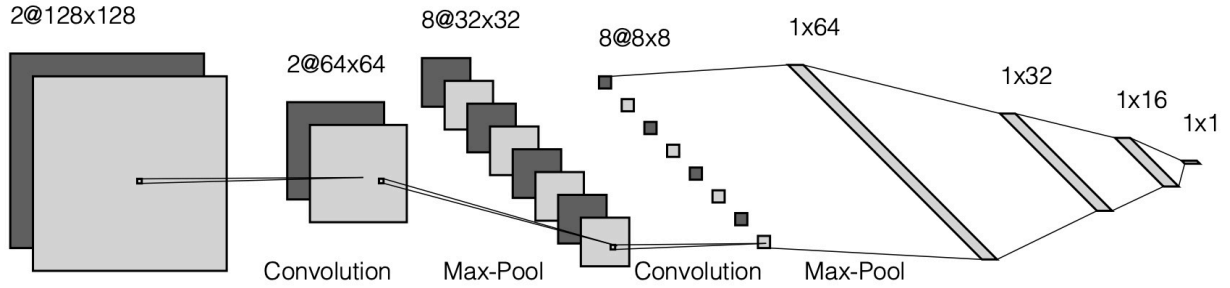
$$y_{ij} = \sum_{m=1}^{k_h} \sum_{n=1}^{k_w} W_{mn} x_{(i+m-1, j+n-1)} + b,$$

where (k_h, k_w) is the kernel size. ReLU or another activation is then applied.

6.4.2 Pooling Layers

Often, a **pooling** layer follows one or more convolutional layers to reduce the spatial resolution. For instance, **max pooling** with a 2×2 window and stride 2 halves the height and width.

An example of representation of a CNN is given here:



6.5 Activation Functions

Nonlinearities $g^{(l)}(\cdot)$ are critical. Common examples in the course:

- **Sigmoid:** $\sigma(z) = 1/(1 + e^{-z})$, which saturates for large $|z|$.
- **Tanh:** $\tanh(z) \in (-1, 1)$, similar saturation behavior.
- **ReLU:** $\max(0, z)$, which alleviates vanishing gradients on the positive side and is widely used in deep networks.

Without a nonlinear $g^{(l)}$, the stacked linear transformations collapse to an effective single linear mapping. Nonlinearity ensures the network can approximate complex functions (Universal Approximation property).

6.6 Cost function and matrix-based Back-Propagation

6.6.1 Sum of Squared Errors (SSE)

For regression tasks, the *empirical sum of squared errors*:

$$\text{SSE}_{\text{emp}}(\alpha_N) = \sum_{i=1}^N (y_i - \hat{y}(x_i))^2,$$

where $\hat{y}(x_i)$ is the network's prediction for input x_i , and y_i is the true target. The parameter vector α_N includes $\{W^{(l)}, b^{(l)}\}$ for $l = 1, \dots, L$.

6.6.2 Gradient Descent Update

Using the notation from **Algorithm 2 (p. 194)**, we iteratively update each parameter in the direction that reduces the SSE cost:

$$\alpha_N(k+1) = \alpha_N(k) - \eta \frac{\partial \text{SSE}_{\text{emp}}}{\partial \alpha_N}(k),$$

where η is the learning rate.

6.6.3 Back-Propagation (Algorithm 2)

Back-propagation is the application of the chain rule to compute $\nabla \text{SSE}_{\text{emp}}$ with respect to each weight matrix $W^{(l)}$ and bias $b^{(l)}$.

1. Output-Layer Delta:

$$\delta^{(L)} = \frac{\partial \text{SSE}_{\text{emp}}}{\partial A^{(L)}} \odot g'^{(L)}(A^{(L)}).$$

- In linear output regression, $g'^{(L)}(\cdot) = 1$, and the cost derivative is $\delta^{(L)} = (\hat{y} - y)$.

2. Backward Recursion:

$$\delta^{(l)} = (W^{(l+1)} \delta^{(l+1)}) \odot g'^{(l)}(A^{(l)}), \quad l = L-1, \dots, 1.$$

This “error” $\delta^{(l)}$ is used to compute partial derivatives for $W^{(l)}$ and $b^{(l)}$.

3. Gradient w.r.t. the weights:

$$\frac{\partial \text{SSE}_{\text{emp}}}{\partial W^{(l)}} = (Z^{(l-1)})^\top \delta^{(l)}, \quad \frac{\partial \text{SSE}_{\text{emp}}}{\partial b^{(l)}} = \sum_{i=1}^N \delta_i^{(l)}.$$

using the matrix-based form.

6.7 Example: Back-Propagation in a MLP

```
import torch.nn as nn

class Simple2LayerMLP(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim=1):
        super().__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = self.fc1(x)
        x = torch.relu(x)
        x = self.fc2(x)
        return x

model = Simple2LayerMLP(input_dim=10, hidden_dim=15, output_dim=1)
criterion = nn.MSELoss(reduction='sum')
eta = 1e-2

X_sample = torch.randn(5, 10)
```

(continues on next page)

(continued from previous page)

```

y_sample = torch.randn(5, 1)

y_pred = model(X_sample)
loss = criterion(y_pred, y_sample)

loss.backward()

with torch.no_grad():
    for param in model.parameters():
        param -= eta * param.grad
        param.grad.zero_()

```

6.8 Overfitting and Cross-Validation

6.8.1 Overfitting Phenomenon

In the handbook's example on p. 197, a network with 15 hidden units achieves extremely low training SSE but high test SSE, illustrating **overfitting**. Overfitting arises when a model fits training data “too well,” capturing noise rather than the underlying trend.

6.8.2 Detection and Mitigation

- **Validation Set:** Split data into training and validation. Monitor the validation error (e.g., MSE). If it increases while training error decreases, the model is overfitting.
- **Cross-Validation:** Alternatively, use k -fold cross-validation.
- **Regularization:**
 - **Dropout:** Temporarily “drop” a fraction of hidden units during training.
 - **Weight Decay (L2):** Penalizes large weights.
 - **Early Stopping:** Halt training when validation performance stops improving.

6.9 Model Persistence: Saving and Loading Models in PyTorch

For safe and reproducible experiments, one typically needs to **save**:

1. **Model Weights (state_dict):** The learned parameters $\{W^{(l)}, b^{(l)}\}$.
2. **Optimizer State (if resuming):** Internal states like momentum buffers in gradient-based optimizers.

A typical code pattern:

```

import torch.optim as optim

model = Simple2LayerMLP(10, 15, 1)
optimizer = optim.SGD(model.parameters(), lr=0.01)

torch.save({
    'model_state_dict': model.state_dict(),

```

(continues on next page)

(continued from previous page)

```

    'optimizer_state_dict': optimizer.state_dict(),
    'epoch': 10
}, "checkpoint.pth")

checkpoint = torch.load("checkpoint.pth", weights_only=False)
model.load_state_dict(checkpoint['model_state_dict'])
optimizer.load_state_dict(checkpoint['optimizer_state_dict'])
start_epoch = checkpoint['epoch'] + 1

# For inference, only model_state_dict is strictly required.

```

6.10 Example: CNN for Regression

Below is a more detailed illustration of an implementation of a CNN in PyTorch for regression highlighting many of the points presented above:

```

import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader, TensorDataset

class SimpleCNNRegression(torch.nn.Module):
    def __init__(self):
        super().__init__()

        self.conv1 = nn.Conv2d(in_channels=1, out_channels=8, kernel_size=3,
padding=1)
        self.conv2 = nn.Conv2d(in_channels=8, out_channels=16, kernel_size=3,
padding=1)
        self.fc1 = nn.Linear(16 * 7 * 7, 64)
        self.fc2 = nn.Linear(64, 1)

        self.dropout = nn.Dropout(p=0.25)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

    def forward(self, x):
        # First conv: conv1 -> ReLU -> pool => shape: (N, 8, 14, 14)
        x = self.conv1(x)
        x = F.relu(x)
        x = self.pool(x)

        # Second conv: conv2 -> ReLU -> pool => shape: (N, 16, 7, 7)
        x = self.conv2(x)
        x = F.relu(x)
        x = self.pool(x)

        # Flatten => shape: (N, 16*7*7 = 784)
        x = x.view(x.size(0), -1)

        # First fully connected => ReLU => Dropout => shape: (N, 64)
        x = self.fc1(x)
        x = F.relu(x)
        x = self.dropout(x)

        # Final fully connected => shape: (N, 1)
        x = self.fc2(x)

```

(continues on next page)

(continued from previous page)

```

        return x

N_train = 1000
N_val   = 200
X_train = torch.randn(N_train, 1, 28, 28)
y_train = torch.randn(N_train, 1)
X_val   = torch.randn(N_val, 1, 28, 28)
y_val   = torch.randn(N_val, 1)

train_dataset = TensorDataset(X_train, y_train)
val_dataset   = TensorDataset(X_val, y_val)
batch_size    = 50
train_loader  = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader    = DataLoader(val_dataset,   batch_size=batch_size, shuffle=False)

model = SimpleCNNRegression()
criterion = nn.MSELoss()

optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

max_epochs      = 20
patience        = 3
best_val_loss    = float('inf')
epochs_no_improve = 0
best_model_path  = "best_model.pth"

for epoch in range(max_epochs):
    model.train()
    running_train_loss = 0.0
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        preds = model(X_batch)
        loss = criterion(preds, y_batch)
        loss.backward()
        optimizer.step()
        running_train_loss += loss.item()
    avg_train_loss = running_train_loss / len(train_loader)

    model.eval()
    running_val_loss = 0.0
    with torch.no_grad():
        for Xv_batch, yv_batch in val_loader:
            val_preds = model(Xv_batch)
            val_loss = criterion(val_preds, yv_batch)
            running_val_loss += val_loss.item()

    avg_val_loss = running_val_loss / len(val_loader)

    print(f"Epoch [{epoch+1}/{max_epochs}]   "
          f"Train MSE: {avg_train_loss:.4f}   "
          f"Val MSE: {avg_val_loss:.4f}")

    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        epochs_no_improve = 0
        torch.save(model.state_dict(), best_model_path)
    print("  Best model saved (improved validation MSE).")

```

(continues on next page)

(continued from previous page)

```

else:
    epochs_no_improve += 1
    if epochs_no_improve >= patience:
        print("Early stopping triggered (no improvement).")
        break

print("\nTraining complete. Loading best model for final checks.")
best_model = SimpleCNNRegression()
best_model.load_state_dict(torch.load(best_model_path, weights_only=False))
best_model.eval()

final_val_loss = 0.0
with torch.no_grad():
    for Xv_batch, yv_batch in val_loader:
        val_preds = best_model(Xv_batch)
        val_loss = criterion(val_preds, yv_batch)
        final_val_loss += val_loss.item()
print(f"Final validation MSE (best model): {final_val_loss / len(val_loader):.4f}")

```

```

Epoch [1/20]  Train MSE: 1.0111  Val MSE: 1.0975
Best model saved (improved validation MSE).
Epoch [2/20]  Train MSE: 0.9949  Val MSE: 1.1007

```

```

Epoch [3/20]  Train MSE: 0.9863  Val MSE: 1.1014
Epoch [4/20]  Train MSE: 0.9760  Val MSE: 1.1093
Early stopping triggered (no improvement).

```

```

Training complete. Loading best model for final checks.
Final validation MSE (best model): 1.0975

```

6.11 Introduction to PyTorch tensors

PyTorch is a popular open-source deep learning framework, and **tensors** are its core data structure. In PyTorch, tensors are the central data abstraction – a specialized data structure very similar to arrays or matrices, used to represent data (inputs, outputs, model parameters) in deep learning models. Tensors are conceptually like NumPy's multi-dimensional arrays, but with two key advantages: they can run on GPUs (for accelerated computing), and they support automatic differentiation (through PyTorch's autograd engine) for computing gradients. These features make tensors fundamental for building and training neural networks.

In this introduction, we'll explore PyTorch tensors from the ground up. We assume you are comfortable with Python and NumPy, but completely new to PyTorch. We'll cover what tensors are and how they compare to NumPy arrays, how to create and manipulate them, basic operations, how PyTorch's autograd works for automatic differentiation, and some additional important tensor functionalities like broadcasting, using GPUs, and in-place operations. Along the way, you'll find hands-on code examples you can run and modify to solidify your understanding.

Let's get started by importing PyTorch (the `torch` library) and seeing how to create tensors in various ways.

PyTorch (`torch`) provides the `Tensor` class, which, as mentioned, is similar to NumPy's `ndarrays` but with GPU support and gradient computation. If you're familiar with NumPy arrays, you'll find the tensor API quite intuitive. In fact, many operations (creation, indexing, math, etc.) have a very similar syntax.

One important difference is that PyTorch operations can be automatically differentiated (if required), which is essential for tasks like training neural networks.

6.11.1 Creating Tensors

There are many ways to create a tensor in PyTorch. Here are some of the most common methods:

1. **From Python data:** You can create a tensor directly from a Python list or sequence of numbers using `torch.tensor()`. PyTorch will automatically infer the data type (dtype) if not specified.
2. **From a NumPy array:** PyTorch can convert NumPy ndarrays into tensors with `torch.from_numpy()`, and vice versa using the Tensor's `.numpy()` method. This makes it easy to move data between NumPy and PyTorch. (Notably, if the NumPy array is on CPU, the tensor and the array will share the same memory buffer, so changing one will also change the other.)
3. **Using factory functions:** PyTorch provides factory methods to create tensors with specific values or distributions. For example, `torch.zeros(shape)` creates a tensor filled with zeros, `torch.ones(shape)` creates one filled with ones, and `torch.rand(shape)` creates one with random values uniformly sampled between 0 and 1. You can also create tensors based on an existing tensor's properties with functions like `torch.ones_like(existing_tensor)` or `torch.rand_like(existing_tensor)`.
4. **Uninitialized tensors:** If you need an uninitialized tensor (just allocating memory without setting values), you can use `torch.empty(shape)`.
5. **Specifying data types and devices:** All the above functions accept a `dtype` parameter to specify the tensor's data type (e.g. `torch.float32`, `torch.int64`, etc.), and a `device` parameter to specify whether the tensor should reside on the CPU or GPU. By default, PyTorch tensors are created with `dtype=torch.float32` (for floating point numbers) and on the CPU.

Let's go through some examples of creating tensors.

```
# Creating a tensor from a Python list
data_list = [[1, 2, 3], [4, 5, 6]]
x_data = torch.tensor(data_list)
pprint("\text{Tensor from list:}", x_data)

# Creating a tensor from a NumPy array
np_array = np.array(data_list)
x_np = torch.from_numpy(np_array)
pprint("\text{Tensor from NumPy array:}", x_np)

# Verify that modifying the NumPy array changes the tensor (shared memory)
np_array[0, 0] = 99
pprint("\text{After modifying the NumPy array, the Tensor becomes:}", x_np)

# Creating tensors using factory functions
shape = (2, 3)
x_zeros = torch.zeros(shape)
x_ones = torch.ones(shape)
x_rand = torch.rand(shape)
pprint("\text{Zero Tensor:}", x_zeros)
pprint("\text{Ones Tensor:}", x_ones)
pprint("\text{Random Tensor:}", x_rand)

# Creating tensors based on existing ones
x_like = torch.ones_like(x_rand) # tensor of ones with same shape as x_rand
x_like_float = torch.rand_like(x_data, dtype=torch.float32) # random tensor with
↳ same shape as x_data, but forced float type
pprint("\text{Ones-like Tensor:}", x_like)
pprint("\text{Rand-like Tensor with float32 dtype:}", x_like_float)

# Checking tensor attributes
```

(continues on next page)

(continued from previous page)

```
print("Tensor shape:", x_data.shape)
print("Tensor data type:", x_data.dtype)
print("Tensor device:", x_data.device)
```

Tensor from list:

$$\begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \end{bmatrix}$$

Tensor from NumPy array:

$$\begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \end{bmatrix}$$

After modifying the NumPy array, the Tensor becomes:

$$\begin{bmatrix} 99.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \end{bmatrix}$$

Zero Tensor:

$$\begin{bmatrix} 0.0000 \\ 0.0000 \\ 0.0000 \\ 0.0000 \\ 0.0000 \\ 0.0000 \end{bmatrix}$$

Ones Tensor:

$$\begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$$

Random Tensor:

$$\begin{bmatrix} 0.1390 \\ 0.8934 \\ 0.4520 \\ 0.2259 \\ 0.5928 \\ 0.8223 \end{bmatrix}$$

Ones-like Tensor:

$$\begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$$

Rand-like Tensor with float32 dtype:

$$\begin{bmatrix} 0.2000 \\ 0.6103 \\ 0.5210 \\ 0.0526 \\ 0.0352 \\ 0.0515 \end{bmatrix}$$

```
Tensor shape: torch.Size([2, 3])
Tensor data type: torch.float32
Tensor device: cpu
```

6.11.2 Basic Tensor Operations

Once you have tensors, PyTorch provides a rich set of operations to manipulate them. In fact, there are over 100 tensor operations available, including arithmetic, linear algebra, slicing, reshaping, and more. Many of these mirror NumPy operations in syntax and function. Let's go through some fundamental operations.

Arithmetic operations

Arithmetic on tensors is typically element-wise (just like with NumPy arrays). You can use Python arithmetic operators (+, -, *, /) or the equivalent `torch` functions (`torch.add`, `torch.sub`, etc.). For example:

```
# Element-wise arithmetic operations
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])

pprint("a=", a)
pprint("b=", b)
pprint("a + b =", a + b)          # element-wise addition
pprint("a * b =", a * b)          # element-wise multiplication
pprint("a - b =", a - b)          # element-wise subtraction
pprint("a / b =", a / b)          # element-wise division
```

$$a = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \end{bmatrix}$$

$$b = \begin{bmatrix} 4.0000 \\ 5.0000 \\ 6.0000 \end{bmatrix}$$

$$a + b = \begin{bmatrix} 5.0000 \\ 7.0000 \\ 9.0000 \end{bmatrix}$$

$$a \times b = \begin{bmatrix} 4.0000 \\ 10.0000 \\ 18.0000 \end{bmatrix}$$

$$a - b = \begin{bmatrix} -3.0000 \\ -3.0000 \\ -3.0000 \end{bmatrix}$$

$$\frac{a}{b} = \begin{bmatrix} 0.2500 \\ 0.4000 \\ 0.5000 \end{bmatrix}$$

The results should be as expected:

- $a + b$ gives $[5.0, 7.0, 9.0]$
- $a * b$ gives $[4.0, 10.0, 18.0]$
- etc.

If you want to do **matrix multiplication** (dot products, etc.), you should use the `@` operator or `torch.matmul`. For example, $A @ B$ does matrix multiply if A and B are 2-D matrices (or higher-dim, following broadcast batch rules). For 1-D vectors, $a @ b$ will give a scalar dot product. Let's see a quick example:

```
# Matrix multiplication vs elementwise
A = torch.tensor([[1, 2],
                  [3, 4]])
B = torch.tensor([[5, 6],
                  [7, 8]])

pprint("A=", A)
pprint("B=", B)
pprint("\text{Element-wise multiply:}", A * B)    # same shape -> elementwise product
pprint("\text{Matrix multiply:}", A @ B)         # 2x2 @ 2x2 -> 2x2 matrix result
```

$$A = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \end{bmatrix}$$

$$B = \begin{bmatrix} 5.0000 \\ 6.0000 \\ 7.0000 \\ 8.0000 \end{bmatrix}$$

$$\text{Element-wise multiply:} \begin{bmatrix} 5.0000 \\ 12.0000 \\ 21.0000 \\ 32.0000 \end{bmatrix}$$

$$\text{Matrix multiply:} \begin{bmatrix} 19.0000 \\ 22.0000 \\ 43.0000 \\ 50.0000 \end{bmatrix}$$

Here, $A * B$ will do element-wise multiplication (resulting in $\begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$), whereas $A @ B$ will compute the matrix product (result $\begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$).

Indexing and slicing

You can access and modify parts of tensors using indexing and slicing, just like with NumPy. PyTorch uses 0-based indexing. You can use indices, ranges, colons, and even Boolean masks or index tensors (though advanced indexing is outside our scope here). Some examples:

```
# Create a 2-D tensor (matrix) for demonstration
x = torch.tensor([[10, 11, 12],
                  [20, 21, 22],
                  [30, 31, 32]])
pprint("\\text{Original tensor: }", x)

# Indexing a single element
pprint("\\text{Element at (0, 1): }", str(x[0, 1].item())) # .item() to get Python
↳ scalar

# Slicing rows and columns
pprint("\\text{First row: }", x[0]) # equivalent to x[0, :]
pprint("\\text{Last column: }", x[:, -1]) # all rows, last column

# Slice a submatrix (e.g., top-left 2x2 block)
sub_x = x[0:2, 0:2] # rows 0-1 and cols 0-1
pprint("\\text{Top-left 2x2 sub-tensor: }", sub_x)

# Modify part of the tensor
x[1:, 1:] = torch.tensor([[ -1, -2],
                           [ -3, -4]]) # set bottom-right 2x2 block to new values
pprint("\\text{Tensor after modification: }", x)
```

Original tensor:
$$\begin{bmatrix} 10.0000 \\ 11.0000 \\ 12.0000 \\ 20.0000 \\ 21.0000 \\ 22.0000 \\ 30.0000 \\ 31.0000 \\ 32.0000 \end{bmatrix}$$

Element at (0, 1): 11

First row:
$$\begin{bmatrix} 10.0000 \\ 11.0000 \\ 12.0000 \end{bmatrix}$$

Last column:
$$\begin{bmatrix} 12.0000 \\ 22.0000 \\ 32.0000 \end{bmatrix}$$

Top-left 2x2 sub-tensor:
$$\begin{bmatrix} 10.0000 \\ 11.0000 \\ 20.0000 \\ 21.0000 \end{bmatrix}$$

Tensor after modification:

$$\begin{bmatrix} 10.0000 \\ 11.0000 \\ 12.0000 \\ 20.0000 \\ -1.0000 \\ -2.0000 \\ 30.0000 \\ -3.0000 \\ -4.0000 \end{bmatrix}$$

In this code:

- We created a 3×3 tensor `x` with some distinct values.
- We accessed a single element `x[0, 1]` (which should be 11 in the original tensor). Using `.item()` converts the 0-dim tensor to a Python number.
- We took the first row `x[0]` (which yields `[10, 11, 12]`) and the last column `x[:, -1]` (which yields `[12, 22, 32]`).
- We sliced a 2×2 sub-tensor (the upper-left corner).
- We then modified a sub-section of the tensor (`x[1:, 1:]`) by assigning a new tensor of the same shape. The original tensor `x` is changed in place.

Note: Slicing a tensor (e.g., `y = x[0:2, 0:2]`) will result in a **view** of the original tensor whenever possible (i.e., it doesn't copy the data). So modifying `y` would also modify `x`. If you need to explicitly copy a tensor or sub-tensor, you can use `.clone()`.

Reshaping and concatenation

Often you'll want to change the shape or dimensionality of a tensor without changing its data. This is done with operations like `tensor.view()` or `tensor.reshape()`. Both do similar things (though `.view()` only works on contiguous tensors). For simplicity, we'll use `reshape` here.

You can also join tensors together (concatenate) along a given dimension using `torch.cat` or stack them with `torch.stack` (which adds a new dimension). Examples:

```
# Reshape (view) a tensor
t = torch.arange(16)          # 1-D tensor [0, 1, 2, ..., 15]
pprint("t=", t)

t_matrix = t.reshape(4, 4)    # reshape to 4x4 matrix
pprint("\text{t reshaped to 4x4: }", t_matrix)

# Transpose the matrix (swap dimensions)
pprint("t^T:\n", t_matrix.T)
```

$t =$

0.0000
1.0000
2.0000
3.0000
4.0000
5.0000
6.0000
7.0000
8.0000
9.0000
10.0000
11.0000
12.0000
13.0000
14.0000
15.0000

t reshaped to 4x4:

0.0000
1.0000
2.0000
3.0000
4.0000
5.0000
6.0000
7.0000
8.0000
9.0000
10.0000
11.0000
12.0000
13.0000
14.0000
15.0000

$t^T :$

0.0000
4.0000
8.0000
12.0000
1.0000
5.0000
9.0000
13.0000
2.0000
6.0000
10.0000
14.0000
3.0000
7.0000
11.0000
15.0000

```
# Concatenate two tensors
t1 = torch.tensor([[1, 2, 3],
```

(continues on next page)

(continued from previous page)

```

        [4, 5, 6]])
t2 = torch.tensor([[7, 8, 9],
                  [10, 11, 12]])
t_cat0 = torch.cat([t1, t2], dim=0)  # concatenate along rows (dim=0)
t_cat1 = torch.cat([t1, t2], dim=1)  # concatenate along columns (dim=1)

pprint("t_1=", t1)
pprint("t_2:=", t2)
pprint("t_1\\text{ and }t_2\\text{ concatenated along dim=0: }", t_cat0)
pprint("t_1\\text{ and }t_2\\text{ concatenated along dim=1: }", t_cat1)

```

$$t_1 = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \end{bmatrix}$$

$$t_2 := \begin{bmatrix} 7.0000 \\ 8.0000 \\ 9.0000 \\ 10.0000 \\ 11.0000 \\ 12.0000 \end{bmatrix}$$

t_1 and t_2 concatenated along dim=0:

$$\begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \\ 7.0000 \\ 8.0000 \\ 9.0000 \\ 10.0000 \\ 11.0000 \\ 12.0000 \end{bmatrix}$$

t_1 and t_2 concatenated along dim=1:

$$\begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 7.0000 \\ 8.0000 \\ 9.0000 \\ 4.0000 \\ 5.0000 \\ 6.0000 \\ 10.0000 \\ 11.0000 \\ 12.0000 \end{bmatrix}$$

Here:

- We created a 1-D tensor $\mathbf{t}$ with 16 sequential elements. We reshaped it into a 4×4 tensor $\mathbf{t_matrix}$.

- We transposed `t_matrix` with `.T`.
- We took two 2×3 tensors `t1` and `t2` and concatenated them. Along `dim=0` (rows), we got a 4×3 tensor by stacking `t2` below `t1`. Along `dim=1` (columns), we got a 2×6 tensor by stacking `t2` to the right of `t1`.

These basic operations (arithmetic, indexing, reshaping, concatenation, etc.) allow you to manipulate tensor data in flexible ways. PyTorch's API is designed to be highly compatible with NumPy's where it makes sense.

6.11.3 Autograd and Gradients

One of the most powerful features of PyTorch is its **autograd** system, which allows automatic computation of gradients for tensor operations. This is what enables training of neural networks using gradient descent: you define a forward computation, and PyTorch can compute the backward gradients for you.

In PyTorch, the `torch.autograd` engine tracks all operations on tensors that have `requires_grad=True`. By default, tensors do **not** track gradients. You need to specify which tensors require gradient computation (usually the learnable parameters of your model, or any input for which you want to compute a derivative) by setting `requires_grad=True` when creating the tensor or by calling `.requires_grad_()` on an existing tensor.

When you perform operations on such tensors, PyTorch builds a computational graph behind the scenes: nodes are tensors, and edges are functions that produce output tensors from input tensors. Once you have a result (typically a scalar loss value), you can call `.backward()` on it, and PyTorch will automatically traverse the graph to compute the gradient of that result with respect to every tensor that has `requires_grad=True` (using the chain rule of calculus). Let's see a simple example to illustrate this:

```
# A simple autograd example: y = f(x) where f(x) = x^2 + 2x + 1
x = torch.tensor(3.0, requires_grad=True) # define a tensor x with gradient_
    ↳tracking
print("x:", x)

# Define a function of x
y = x**2 + 2*x + 1
print("y:", y)

# Compute the gradient dy/dx by calling backward on y
y.backward() # computes gradient of y w.r.t. x
print("dy/dx at x=3:", x.grad)
```

```
x: tensor(3., requires_grad=True)
y: tensor(16., grad_fn=<AddBackward0>)
dy/dx at x=3: tensor(8.)
```

In this example:

- We created a tensor `x` with value 3.0 and `requires_grad=True`. This means PyTorch will track operations on `x`.
- We computed `y = x**2 + 2*x + 1`. Because `x` requires grad, `y` will also by default require grad and have a grad function associated.
- We called `y.backward()`. Since `y` is a scalar (single value), we can call `backward` directly. After this call, the gradient of `y` with respect to `x` is computed and stored in `x.grad`.
- We printed `x.grad`. The derivative $y = x^2 + 2x + 1$ is $dy/dx = 2x + 2$. At $x = 3$, that is 8.

Important notes about autograd:

1. You can disable gradient tracking by wrapping code in with `torch.no_grad()` : or by using `.detach()` on a tensor to get a new tensor that shares the same data but with no grad tracking.

2. If y is not a scalar, you need to specify a gradient argument in `y.backward(gradient=...)` which is the tensor of same shape as y containing the gradient of the final result w.r.t. each element of y .
3. By default, gradients are **accumulated** into the `.grad` property. So typically you want to zero out gradients (e.g., `x.grad.zero_()`) between independent backward passes.

Autograd is what powers training routines. If you had a multi-parameter function (like a neural network with many weights), you would set `requires_grad=True` for all those parameters. After computing a loss (scalar), you'd call `loss.backward()`, and each parameter tensor's `.grad` will be filled with its gradient.

To reinforce understanding, let's do a slightly more complex autograd example with two inputs:

```
# Autograd with two inputs: z = x * y + y^2
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(-3.0, requires_grad=True)

z = x * y + y**2    # z is a function of both x and y
print("z:", z)

z.backward()        # Compute gradients dz/dx and dz/dy
print("dz/dx:", x.grad)    # should be y
print("dz/dy:", y.grad)    # should be x + 2y
```

```
z: tensor(3., grad_fn=<AddBackward0>)
dz/dx: tensor(-3.)
dz/dy: tensor(-4.)
```

Here $z = x \times y + y^2$. We expect $dz/dx = y$ and $dz/dy = x + 2y$. With $x = 2.0$ and $y = -3.0$, those values are $dz/dx = -3$ and $dz/dy = 2 + 2(-3) = -4$. The autograd should produce the same results in `x.grad` and `y.grad`.

6.11.4 Advanced Tensor Functions

Broadcasting

Like NumPy, PyTorch supports **broadcasting** – a mechanism that allows arithmetic operations on tensors of different shapes by automatically expanding one of them to match the shape of the other. Broadcasting rules in PyTorch are the same as NumPy's. In short, two tensors are compatible for an elementwise operation if their shapes are equal or align in such a way that one of them can be expanded (a dimension of size 1 can be expanded to match the other's size, and missing dimensions are treated as size 1).

For example, if you have a tensor of shape (4, 3) and another of shape (3,), you can add them because the second tensor can be treated as if it were (1, 3) and then expanded to (4, 3) to match the first tensor's shape.

In code, we can demonstrate broadcasting with a simple example:

```
# Broadcasting example: add a vector to each row of a matrix
mat = torch.tensor([[0, 0, 0],
                    [10, 10, 10],
                    [20, 20, 20],
                    [30, 30, 30]])    # shape (4,3)
vec = torch.tensor([1, 2, 3])        # shape (3,)

result = mat + vec    # vec is broadcast to shape (4,3)
pprint("M=", mat)
pprint("V=", vec)
pprint("M+V", result)
```

$$M = \begin{bmatrix} 0.0000 \\ 0.0000 \\ 0.0000 \\ 10.0000 \\ 10.0000 \\ 10.0000 \\ 20.0000 \\ 20.0000 \\ 20.0000 \\ 30.0000 \\ 30.0000 \\ 30.0000 \end{bmatrix}$$

$$V = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \end{bmatrix}$$

$$M + V = \begin{bmatrix} 1.0000 \\ 2.0000 \\ 3.0000 \\ 11.0000 \\ 12.0000 \\ 13.0000 \\ 21.0000 \\ 22.0000 \\ 23.0000 \\ 31.0000 \\ 32.0000 \\ 33.0000 \end{bmatrix}$$

Here, `vec` is added to each row of `mat`. The expected result would be each row of `mat` incremented by `[1, 2, 3]`. Broadcasting is a powerful feature that makes your code more concise and often more efficient.

Device Management (CPU vs GPU)

One of PyTorch's strengths is the ability to perform computations on a **GPU** for speed. Tensors can reside on either the CPU or the GPU. By default, tensors are created on the CPU. You can check where a tensor is located by looking at its `.device` attribute. To leverage a GPU (if you have one and PyTorch is installed with CUDA support), you need to explicitly move tensors to the GPU. There are a few ways to do this:

1. Using the `.to(device)` method on a tensor, where `device` is something like `torch.device("cuda")` or the shorthand string `"cuda"`.
2. Using `.cuda()` method on the tensor.
3. Creating the tensor directly on the GPU by specifying `device=torch.device("cuda")` (or `device="cuda"`) in the creation function.

It's common to write code that is device-agnostic (runs on CPU if no GPU is available, otherwise uses the GPU). For example:

```
# Device-agnostic code: choose CPU or CUDA
device = torch.device("cuda" if torch.cuda.is_available() else ("mps" if torch.
    backends.mps.is_available() else "cpu"))
```

(continues on next page)

(continued from previous page)

```

print("Using device:", device)

# Create a tensor on the chosen device
x = torch.tensor([1.0, 2.0, 3.0], device=device)
print("x device:", x.device)

# Alternatively, move an existing tensor to the device
y = torch.ones(3)          # defaults to CPU
y = y.to(device)           # move to GPU if available
print("y device:", y.device)

```

```

Using device: mps
x device: mps:0
y device: mps:0

```

If you actually run this on a system with a GPU and CUDA installed, you should see device printed as `cuda:0` and the tensor devices as such. On a CPU-only system, it will say `cpu`.

A quick verification of CPU vs GPU performance can be done by timing a large matrix multiplication. (Note: if you actually run this, avoid printing the giant result; just measure the time.)

```

import time

# Large tensor operation on CPU vs GPU (if available)
large_cpu = torch.randn(10000, 10000) # 10000x10000 matrix on CPU
start = time.time()
res_cpu = large_cpu @ large_cpu        # matrix multiply on CPU
end = time.time()
print("CPU computation time:", end - start, "seconds")

if torch.cuda.is_available():
    large_gpu = large_cpu.to("cuda")
    torch.cuda.synchronize() # ensure data is transferred
    start = time.time()
    res_gpu = large_gpu @ large_gpu # matrix multiply on GPU
    torch.cuda.synchronize() # wait for GPU to finish computation
    end = time.time()
    print("GPU computation time:", end - start, "seconds")

if torch.backends.mps.is_available():
    large_gpu = large_cpu.to("mps")
    start = time.time()
    res_gpu = large_gpu @ large_gpu
    end = time.time()
    print("MPS computation time:", end - start, "seconds")

```

```
CPU computation time: 2.039803981781006 seconds
```

```
MPS computation time: 0.07256889343261719 seconds
```

This snippet will likely show that the GPU computation is significantly faster for large matrix operations (depending on hardware). Remember to move your **model parameters** and **data** to the same device before doing operations.

For example, the code below moves a tensor to GPU if available:

```

tensor = torch.rand(3)
print("Before moving, device:", tensor.device)
if torch.cuda.is_available():
    tensor = tensor.to('cuda')
    print("After moving, device:", tensor.device)
if torch.backends.mps.is_available():
    tensor = tensor.to('mps')
    print("After moving, device:", tensor.device)

```

```
Before moving, device: cpu
```

```
After moving, device: mps:0
```

Running this will print:

```
Before moving, device: cpu
After moving, device: cuda:0
```

if you have a GPU, or

```
Before moving, device: cpu
After moving, device: mps:0
```

on M-chip Apple devices.

```
torch.set_default_device(device)
```

In-place Operations

PyTorch differentiates between operations that create new tensors and those that modify an existing tensor **in-place**. In-place operations are identified by an underscore suffix in the method name. For example, `tensor.add_(5)` will add 5 to every element of the tensor **in-place** (mutating the tensor), whereas `tensor.add(5)` returns a new tensor with the result and leaves the original unchanged. All functions ending in `_` modify the tensor in-place.

Why use in-place ops? They can save memory and sometimes a bit of compute, because you don't allocate a new tensor for the result. However, be careful: in-place operations can sometimes interfere with autograd (if you modify values required to compute gradients). As a rule of thumb, avoid using in-place ops on tensors that require grad unless you know what you're doing.

Let's demonstrate the difference between an in-place operation and its out-of-place counterpart:

```

a = torch.ones(5)
pprint("a=", a)

# Out-of-place addition
b = a.add(5)
print("After b = a.add(5):")
pprint("a=", a)  # a should remain unchanged
pprint("b=", b)  # b is the new tensor

# In-place addition
a.add_(5)
print("\nAfter a.add_(5):")
pprint("a=", a)  # a is now changed in-place

```

$$a = \begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$$

```
After b = a.add(5):
```

$$a = \begin{bmatrix} 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \\ 1.0000 \end{bmatrix}$$

$$b = \begin{bmatrix} 6.0000 \\ 6.0000 \\ 6.0000 \\ 6.0000 \\ 6.0000 \end{bmatrix}$$

```
After a.add_(5):
```

$$a = \begin{bmatrix} 6.0000 \\ 6.0000 \\ 6.0000 \\ 6.0000 \\ 6.0000 \end{bmatrix}$$

Output should show:

1. Initially, a is [1, 1, 1, 1, 1].
2. After `b = a.add(5)`, a stays [1, 1, 1, 1, 1] but b is [6, 6, 6, 6, 6].
3. After `a.add_(5)`, now a has become [6, 6, 6, 6, 6] in-place.

Another example with multiplication:

```
x = torch.ones(3)
y = x.mul_(2)  # in-place multiply x by 2
pprint("x=", x)
pprint("y=", y)

pprint("\text{x after in-place mul_: }", x)
pprint("\text{y (result of x.mul_(2)): }", y)

# Now x is [2,2,2], let's do out-of-place multiply
z = x.mul(3)
pprint("\text{x after out-of-place mul(3): }", x)  # x remains [2,2,2]
pprint("\text{z (result of x.mul(3)): }", z)      # z is [6,6,6]
```

$$x = \begin{bmatrix} 2.0000 \\ 2.0000 \\ 2.0000 \end{bmatrix}$$

$$y = \begin{bmatrix} 2.0000 \\ 2.0000 \\ 2.0000 \end{bmatrix}$$

$$x \text{ after in-place mul_}: \begin{bmatrix} 2.0000 \\ 2.0000 \\ 2.0000 \end{bmatrix}$$

$$y \text{ (result of } x.\text{mul_}(2)\text{)}: \begin{bmatrix} 2.0000 \\ 2.0000 \\ 2.0000 \end{bmatrix}$$

$$x \text{ after out-of-place mul}(3): \begin{bmatrix} 2.0000 \\ 2.0000 \\ 2.0000 \end{bmatrix}$$

$$z \text{ (result of } x.\text{mul}(3)\text{)}: \begin{bmatrix} 6.0000 \\ 6.0000 \\ 6.0000 \end{bmatrix}$$

This shows that after the in-place multiply, x (and y) have doubled their values, while the out-of-place multiply by 3 produces a new tensor z and leaves x the same.

6.12 Exercises

6.12.1 Comparison of symbolic and automatic differentiation in a simple feedforward neural network

In this exercise, we will illustrate how **automatic differentiation** (as implemented in PyTorch) matches with the result of **symbolic differentiation** for a simple two-hidden-neuron feedforward neural network. You should:

1. **Define** a simple feedforward neural network (FNN) with two hidden neurons z_1 and z_2 using the values and notation from the figure below.
2. **Compute** the network output and the loss function (a squared error).
3. **Derive** the gradient of the loss function with respect to each weight **symbolically**.
4. **Use** PyTorch's **automatic differentiation** to compute the same gradients **numerically**.
5. **Compare** the two results to verify they match.

The feedforward neural network architecture that we will use in this exercise is the following:

Problem setup

Let's consider the following simplified network:

- **Input:** x , a single scalar input.
- **Hidden layer:** two neurons z_1 and z_2
- **Output:** $\hat{y}$

For a single input x , the hidden neurons are given by:

$$\begin{aligned} \text{Neuron } z_1 : \quad a_1^{(1)} &= w_{11}^{(1)} \cdot x, & z_1 &= g(a_1^{(1)}) = \frac{1}{1 + e^{-a_1^{(1)}}}, \\ \text{Neuron } z_2 : \quad a_2^{(1)} &= w_{12}^{(1)} \cdot x, & z_2 &= g(a_2^{(1)}) = \frac{1}{1 + e^{-a_2^{(1)}}}. \end{aligned}$$

Then the output neuron combines these two hidden neurons:

$$\begin{aligned} a_1^{(2)} &= w_{11}^{(2)} z_1 + w_{21}^{(2)} z_2, \\ \hat{y} &= g(a_1^{(2)}) = \frac{1}{1 + e^{-a_1^{(2)}}}. \end{aligned}$$

We consider a **loss** (squared error w.r.t. a true label y):

$$L = (y - \hat{y})^2.$$

Our goal is to **differentiate** L with respect to the parameters:

$$w_{11}^{(1)}, \quad w_{12}^{(1)}, \quad w_{11}^{(2)}, \quad w_{21}^{(2)}.$$

We will compute these derivatives in two ways:

1. **Symbolic Differentiation** (analytical formulas)
2. **Automatic Differentiation** (via PyTorch `backward()`)

We will show that these two results match.

PyTorch implementation

Activation function and its derivative

We will use the logistic sigmoid $g(a) = 1/(1 + e^{-a})$ as the activation function. Its derivative with respect to a is:

$$\frac{d}{da} g(a) = g(a)(1 - g(a)).$$

However, for clarity, we can directly implement it as given in the example code:

```
def g(a):
    """
    Sigmoid function g(a) = 1 / (1 + e^-a)
    """
    return 1.0 / (1.0 + torch.exp(-a))

def gp(a):
    """
    Derivative of the sigmoid function g'(a)
    = exp(-a) / (1 + exp(-a))^2
    """
    return torch.exp(-a) / (1 + torch.exp(-a))**2
```

Define the working variables

We will define:

- A target value y
- An input x

- Four weights we wish to learn/differentiate:

- $w_{11}^{(1)}$ and $w_{12}^{(1)}$ for the hidden layer
- $w_{11}^{(2)}$ and $w_{21}^{(2)}$ for the output layer

By setting `requires_grad=True`, PyTorch will keep track of operations involving these tensors, allowing automatic differentiation via the `backward()` call.

```
# Input
x = torch.tensor([-1.7]) # A single scalar input

# True target
y = torch.tensor([-4.5]) # A single scalar target

# Hidden layer weights
w11_1 = torch.tensor([0.13], requires_grad=True) # w_{11}^{(1)}
w12_1 = torch.tensor([-2.0], requires_grad=True) # w_{12}^{(1)}

# Output layer weights
w11_2 = torch.tensor([1.0], requires_grad=True) # w_{11}^{(2)}
w21_2 = torch.tensor([0.5], requires_grad=True) # w_{21}^{(2)}
```

Forward pass

We compute:

$$a_1^{(1)} = w_{11}^{(1)}x, \quad a_2^{(1)} = w_{12}^{(1)}x, \quad z_1 = g(a_1^{(1)}), \quad z_2 = g(a_2^{(1)}), \quad a_1^{(2)} = w_{11}^{(2)}z_1 + w_{21}^{(2)}z_2, \quad \hat{y} = g(a_1^{(2)})$$

and finally the loss $L = (y - \hat{y})^2$.

```
# Hidden neuron 1
a1_1 = w11_1 * x
z1 = g(a1_1)

# Hidden neuron 2
a2_1 = w12_1 * x
z2 = g(a2_1)

# Output neuron
a1_2 = w11_2 * z1 + w21_2 * z2
yhat = g(a1_2)

# Loss
L = (y - yhat)**2
```

Symbolic gradients

From the slides and from our direct symbolic differentiation, we have:

$$\begin{aligned} \frac{\partial L}{\partial w_{11}^{(1)}} &= -2(y - \hat{y}) g'(a_1^{(2)}) w_{11}^{(2)} g'(a_1^{(1)}) x, \\ \frac{\partial L}{\partial w_{12}^{(1)}} &= -2(y - \hat{y}) g'(a_1^{(2)}) w_{21}^{(2)} g'(a_2^{(1)}) x, \\ \frac{\partial L}{\partial w_{11}^{(2)}} &= -2(y - \hat{y}) g'(a_1^{(2)}) z_1, \\ \frac{\partial L}{\partial w_{21}^{(2)}} &= -2(y - \hat{y}) g'(a_1^{(2)}) z_2. \end{aligned}$$

Let's compute these values explicitly in PyTorch (still using the same gp function for $g'(a)$).

```
# Symbolic gradients
Sgrad_L_w11_1 = -2 * (y - yhat) * gp(a1_2) * w11_2 * gp(a1_1) * x
Sgrad_L_w12_1 = -2 * (y - yhat) * gp(a1_2) * w21_2 * gp(a2_1) * x
Sgrad_L_w11_2 = -2 * (y - yhat) * gp(a1_2) * z1
Sgrad_L_w21_2 = -2 * (y - yhat) * gp(a1_2) * z2

Sgrad_L_w11_1, Sgrad_L_w12_1, Sgrad_L_w11_2, Sgrad_L_w21_2
```

```
(tensor([-0.8892], device='mps:0', grad_fn=<MulBackward0>),
 tensor([-0.0563], device='mps:0', grad_fn=<MulBackward0>),
 tensor([0.9424], device='mps:0', grad_fn=<MulBackward0>),
 tensor([2.0494], device='mps:0', grad_fn=<MulBackward0>))
```

Automatic differentiation

We can then rely on PyTorch's **autograd** engine to compute the gradients for us. Simply call:

```
L.backward()
```

and then retrieve the `.grad` attribute of each parameter of interest.

```
# Reset gradients (in case this cell is run multiple times)
w11_1.grad = None
w12_1.grad = None
w11_2.grad = None
w21_2.grad = None

# Perform backprop
L.backward()

# Automatic gradients
grad_L_w11_1 = w11_1.grad
grad_L_w12_1 = w12_1.grad
grad_L_w11_2 = w11_2.grad
grad_L_w21_2 = w21_2.grad

grad_L_w11_1, grad_L_w12_1, grad_L_w11_2, grad_L_w21_2
```

```
(tensor([-0.8892], device='mps:0'),
 tensor([-0.0563], device='mps:0'),
 tensor([0.9424], device='mps:0'),
 tensor([2.0494], device='mps:0'))
```

Comparison of symbolic and automatic differentiation

Finally, let's print them side by side to verify they match (or are very close, up to floating-point rounding).

```
print("Loss L =", L.item())

print("\n--- dL/dw11^2 ---")
print("Auto-grad:", grad_L_w11_2.item(), "\t Symbolic:", Sgrad_L_w11_2.item())

print("\n--- dL/dw21^2 ---")
print("Auto-grad:", grad_L_w21_2.item(), "\t Symbolic:", Sgrad_L_w21_2.item())

print("\n--- dL/dw11^1 ---")
```

(continues on next page)

(continued from previous page)

```
print("Auto-grad:", grad_L_w11_1.item(), "\t Symbolic:", Sgrad_L_w11_1.item())

print("\n--- dL/dw12^(1) ---")
print("Auto-grad:", grad_L_w12_1.item(), "\t Symbolic:", Sgrad_L_w12_1.item())
```

```
Loss L = 27.21538734436035

--- dL/dw11^(2) ---
Auto-grad: 0.9423852562904358      Symbolic: 0.9423853754997253

--- dL/dw21^(2) ---
Auto-grad: 2.049448013305664      Symbolic: 2.049448251724243

--- dL/dw11^(1) ---
Auto-grad: -0.8891826272010803    Symbolic: -0.8891826272010803

--- dL/dw12^(1) ---
Auto-grad: -0.05625969544053078   Symbolic: -0.056259699165821075
```

In this exercise, we demonstrated that for our simple 2-hidden-neuron feedforward network:

1. **Symbolic differentiation** (i.e., deriving the gradient expressions by hand)
2. **Automatic differentiation** (i.e., via PyTorch's `backward()`)

yield the **same** results, up to numerical precision. This is a powerful illustration of why automatic differentiation is so valuable: it **automates** what would otherwise be a lengthy (and error-prone) symbolic derivation process.

Feel free to change the values of x , y , and the weights w to see how the gradients match up in other situations. You can now expand this example to more complex network architectures or different activation and loss functions.

6.12.2 Facial keypoints detection with PyTorch FCN and CNN architectures

Facial keypoint detection is the task of identifying important landmarks on a human face (e.g., corners of the eyes, nose tip, mouth corners) in an image. These keypoints are crucial for many applications in computer vision:

- Aligning faces for recognition or verification.
- Tracking facial expressions for emotion detection.
- Applying augmented reality filters or effects accurately on a face.

In this exercise, we will build fully connected neural networks and convolutional neural networks to detect **15 keypoints** on **96x96 grayscale face images**. The network will take an image as input and output the (x, y) coordinates of the **15 facial keypoints**.

Source: danielnouri.org

Dataset overview

We will use the **Kaggle Facial Keypoints Detection** dataset ([Kaggle challenge webpage](#)). This dataset consists of a training set of **7,049** face images (96x96 pixels, grayscale). Each image has up to **15 keypoint coordinates** labeled (30 values: x and y for each of 15 facial landmarks). For example, the keypoints include the centers of the eyes, the tip of the nose, and the corners of the mouth.

Data format

The training data is provided as a CSV file (`training.csv`). Each row contains:

- **30 columns** of keypoint coordinates: `left_eye_center_x`, `left_eye_center_y`, `right_eye_center_x`, ... etc. (15 keypoints $\times$ 2). If a keypoint was not labeled for a given image, its value is missing.
- An **Image** column containing 96x96 pixel intensity values (9216 numbers) in a single string, separated by spaces.

If you downloaded this exercise from the Université Virtuelle, you should have a `training.csv` file in the `data/` directory. Now, let's load the training data using pandas and examine its structure.

Missing values

Not all images have all 15 keypoints annotated. In fact, only about 30% of the images have all 15 keypoints; the rest have some missing keypoint labels. Nearly all images have at least a few keypoints (e.g., eyes, nose, mouth) labeled, but many did not have the more peripheral points. In this tutorial, for simplicity, we will use only images with **complete keypoint data** (all 15 points). This means we'll discard images with any missing keypoint values. (In practice, one could use data augmentation or multi-stage training to leverage partially labeled images, but we will not cover that here.)

In this exercise, you should:

1. Explore the dataset
2. Preprocess the dataset to use it with neural networks including:
 - handling missing values
 - converting the image data from strings to numeric arrays
 - normalizing the pixel values
 - reshaping the images
 - preparing the keypoint coordinates labels
3. Implement a fully connected neural network and compute the validated MSE
4. Implement a convolutional neural network and compute the validated MSE
5. Visualize the predictions of both models with respect to the true keypoints coordinates on images

```
import pandas as pd

# Load the training data
df = pd.read_csv('facial-keypoints-detection/training.csv')
print("Training data shape:", df.shape)
df.head(5)  # show first 5 rows as an example
```

```
Training data shape: (7049, 31)
```

	left_eye_center_x	left_eye_center_y	right_eye_center_x	\
0	66.033564	39.002274	30.227008	
1	64.332936	34.970077	29.949277	
2	65.057053	34.909642	30.903789	

(continues on next page)

(continued from previous page)

```

3      65.225739      37.261774      32.023096
4      66.725301      39.621261      32.244810

    right_eye_center_y  left_eye_inner_corner_x  left_eye_inner_corner_y  \
0      36.421678      59.582075      39.647423
1      33.448715      58.856170      35.274349
2      34.909642      59.412000      36.320968
3      37.261774      60.003339      39.127179
4      38.042032      58.565890      39.621261

    left_eye_outer_corner_x  left_eye_outer_corner_y  right_eye_inner_corner_x  \
0      73.130346      39.969997      36.356571
1      70.722723      36.187166      36.034723
2      70.984421      36.320968      37.678105
3      72.314713      38.380967      37.618643
4      72.515926      39.884466      36.982380

    right_eye_inner_corner_y  ...  nose_tip_y  mouth_left_corner_x  \
0      37.389402  ...  57.066803      61.195308
1      34.361532  ...  55.660936      56.421447
2      36.320968  ...  53.538947      60.822947
3      38.754115  ...  54.166539      65.598887
4      39.094852  ...  64.889521      60.671411

    mouth_left_corner_y  mouth_right_corner_x  mouth_right_corner_y  \
0      79.970165      28.614496      77.388992
1      76.352000      35.122383      76.047660
2      73.014316      33.726316      72.732000
3      72.703722      37.245496      74.195478
4      77.523239      31.191755      76.997301

    mouth_center_top_lip_x  mouth_center_top_lip_y  mouth_center_bottom_lip_x  \
0      43.312602      72.935459      43.130707
1      46.684596      70.266553      45.467915
2      47.274947      70.191789      47.274947
3      50.303165      70.091687      51.561183
4      44.962748      73.707387      44.227141

    mouth_center_bottom_lip_y  \
0      84.485774
1      85.480170
2      78.659368
3      78.268383
4      86.871166

                                Image
0  238 236 237 238 240 240 239 241 241 243 240 23...
1  219 215 204 196 204 211 212 200 180 168 178 19...
2  144 142 159 180 188 188 184 180 167 132 84 59 ...
3  193 192 193 194 194 194 193 192 168 111 50 12 ...
4  147 148 160 196 215 214 216 217 219 220 206 18...

[5 rows x 31 columns]
```

This will show the DataFrame with columns for each keypoint and the “Image” column containing pixel values. Let’s check how many values are present in each column and how many are missing:

```
# Count non-null values in each column
print(df.count())
```

```
left_eye_center_x      7039
left_eye_center_y      7039
right_eye_center_x     7036
right_eye_center_y     7036
left_eye_inner_corner_x 2271
left_eye_inner_corner_y 2271
left_eye_outer_corner_x 2267
left_eye_outer_corner_y 2267
right_eye_inner_corner_x 2268
right_eye_inner_corner_y 2268
right_eye_outer_corner_x 2268
right_eye_outer_corner_y 2268
left_eyebrow_inner_end_x 2270
left_eyebrow_inner_end_y 2270
left_eyebrow_outer_end_x 2225
left_eyebrow_outer_end_y 2225
right_eyebrow_inner_end_x 2270
right_eyebrow_inner_end_y 2270
right_eyebrow_outer_end_x 2236
right_eyebrow_outer_end_y 2236
nose_tip_x             7049
nose_tip_y             7049
mouth_left_corner_x    2269
mouth_left_corner_y    2269
mouth_right_corner_x   2270
mouth_right_corner_y   2270
mouth_center_top_lip_x  2275
mouth_center_top_lip_y  2275
mouth_center_bottom_lip_x 7016
mouth_center_bottom_lip_y 7016
Image                  7049
dtype: int64
```

From this, we can see which keypoint columns have missing values. We expect some columns to have 7049 values (no missing data for those keypoints) and others to have fewer (indicating missing labels). If a column has fewer than 7049 non-null entries, the difference is the number of missing values.

Data exploration

Before training a model, it's important to understand the dataset. We will:

1. Identify how many keypoint annotations are missing.
2. Visualize a few images with their annotated keypoints to get an intuition of the task.

Missing Values

We already counted non-null values. Let's quantify missing data more directly:

```
# Check total missing values per column
missing_counts = df.isnull().sum()
print("Missing values per column:")
print(missing_counts)
```

```

Missing values per column:
left_eye_center_x      10
left_eye_center_y      10
right_eye_center_x     13
right_eye_center_y     13
left_eye_inner_corner_x 4778
left_eye_inner_corner_y 4778
left_eye_outer_corner_x 4782
left_eye_outer_corner_y 4782
right_eye_inner_corner_x 4781
right_eye_inner_corner_y 4781
right_eye_outer_corner_x 4781
right_eye_outer_corner_y 4781
left_eyebrow_inner_end_x 4779
left_eyebrow_inner_end_y 4779
left_eyebrow_outer_end_x 4824
left_eyebrow_outer_end_y 4824
right_eyebrow_inner_end_x 4779
right_eyebrow_inner_end_y 4779
right_eyebrow_outer_end_x 4813
right_eyebrow_outer_end_y 4813
nose_tip_x             0
nose_tip_y             0
mouth_left_corner_x    4780
mouth_left_corner_y    4780
mouth_right_corner_x   4779
mouth_right_corner_y   4779
mouth_center_top_lip_x 4774
mouth_center_top_lip_y 4774
mouth_center_bottom_lip_x 33
mouth_center_bottom_lip_y 33
Image                  0
dtype: int64

```

You should see that some keypoint columns (e.g., `left_eye_center_x`) have very few missing values, while others (e.g., some mouth or eyebrow points) have thousands of missing entries. In fact, only 2,140 images have **all** 15 keypoints labeled, meaning the remainder have some missing points. We will handle this in preprocessing by removing incomplete rows.

Sample visualization

Let's visualize some of the images with their keypoints to understand the data. We will pick a few examples (ensuring they have no missing keypoints) and plot them in a grid.

```

df_complete = df.dropna().reset_index(drop=True)
print("Number of complete cases (no missing keypoints):", len(df_complete))

sample_images = df_complete['Image'].iloc[:9].values
sample_images = [np.fromstring(img_str, sep=' ').reshape(96, 96) for img_str in
                 sample_images]
sample_keypoints = df_complete.iloc[:9, :-1].values # first 30 columns are keypoints

fig, axes = plt.subplots(3, 3, figsize=(10, 10))
axes = axes.flatten()
for i in range(9):
    axes[i].imshow(sample_images[i], cmap='gray')
    # Plot keypoints as red dots
    kp = sample_keypoints[i]

```

(continues on next page)

(continued from previous page)

```
axes[i].scatter(kp[0::2], kp[1::2], c='red', s=15)
axes[i].axis('off')
plt.tight_layout()
plt.show()
```

Number of complete cases (no missing keypoints): 2140



We see 9 face images with red dots marking the locations of eyes, nose, mouth, etc. This visual check helps verify that the keypoints line up with facial features (for instance, dots around the eyes, nose, and mouth).

Data preprocessing

Next, we need to prepare the data for training a PyTorch model. This involves:

1. **Handling missing keypoint values:** We'll simplify by using only the images that have all 15 keypoints labeled.
2. **Converting the image data** from strings to numeric arrays.
3. **Normalizing** the pixel values (scaling them to $[0, 1]$) by dividing by 255.
4. **Reshaping** the images to the proper format $((N, 1, 96, 96))$ for a single-channel image).
5. **Preparing the keypoint coordinates** array of shape $(N, 30)$. Each sample has 30 target values: 15 x -coordinates and 15 y -coordinates.

We encourage you to try at home to use more sophisticated data augmentation strategies or partial labeling approaches, but we'll keep things simple here.

```
# 1. Drop rows with missing values
df = df.dropna().reset_index(drop=True)
print("Remaining training samples after dropping missing:", len(df))

# 2. Convert the 'Image' column from strings to numpy arrays of shape (96,96)
df['Image'] = df['Image'].apply(lambda img: np.fromstring(img, sep=' ', dtype=np.
    float32))

# 3. Normalize pixel values to [0,1]
df['Image'] = df['Image'].apply(lambda img: img/255.0)

# 4. Convert list of image arrays to a 4D numpy array for model input
X_fcn = np.vstack(df['Image'].values) # shape (N, 9216)
X_cnn = np.vstack(df['Image'].values).reshape(-1, 1, 96, 96).astype(np.float32) # (N,
    1, 96, 96)

# 5. Prepare keypoint targets as a numpy array
y = df[df.columns[:-1]].values # all columns except the 'Image' column
y = y.astype(np.float32)
print("X_fcn shape:", X_fcn.shape, "y shape:", y.shape)
print("X_cnn shape:", X_cnn.shape, "y shape:", y.shape)
```

```
Remaining training samples after dropping missing: 2140
```

```
X_fcn shape: (2140, 9216) y shape: (2140, 30)
X_cnn shape: (2140, 1, 96, 96) y shape: (2140, 30)
```

Fully connected neural network (FCN)

FCN architecture

We create a **Fully Connected Neural Network (FCN)** using PyTorch to regress the 30 keypoint coordinates (x, y) directly from flattened grayscale images (of size $96 \times 96 = 9216$ pixels). Feel free to adjust the hidden layer sizes and dropout probabilities based on your experimentation.

The architecture of our FCN is as follows:

- **Fully Connected Layer 1:**
 - Input: 9216 features (flattened image)

- Output: 1024 neurons
- Activation: ReLU
- Dropout: 0.3 probability (to reduce overfitting)

- **Fully Connected Layer 2:**

- Input: 1024 neurons
- Output: 512 neurons
- Activation: ReLU
- Dropout: 0.3 probability

- **Output Layer:**

- Input: 512 neurons
- Output: 30 neurons (coordinates for 15 keypoints)
- Linear activation (no final nonlinearity, since this is a regression output)

This FCN is trained using the **Mean Squared Error (MSE)** loss function to predict the (x, y) coordinates of 15 facial keypoints from the flattened images.

```
class KeypointFCN(nn.Module):
    def __init__(self):
        super(KeypointFCN, self).__init__()
        # We have 96*96 = 9216 input features per image
        self.fc1 = nn.Linear(96*96, 1024)
        self.dropout1 = nn.Dropout(0.3)
        self.fc2 = nn.Linear(1024, 512)
        self.dropout2 = nn.Dropout(0.3)
        self.fc3 = nn.Linear(512, 30) # 30 = (x,y) for 15 keypoints

    def forward(self, x):
        # x shape: (batch_size, 9216)
        # (If x is not already flattened, do x = x.view(x.size(0), -1))
        x = F.relu(self.fc1(x))
        x = self.dropout1(x)
        x = F.relu(self.fc2(x))
        x = self.dropout2(x)
        x = self.fc3(x)
        return x
```

Split the Flattened Dataset for Training/Validation

We already have X_{fcn} of shape $(N, 9216)$ and y of shape $(N, 30)$. We split them into training and validation sets:

```
from sklearn.model_selection import train_test_split

# 90/10 train-validation split using X_fcn
X_train_fcn, X_val_fcn, y_train_fcn, y_val_fcn = train_test_split(
    X_fcn, y, test_size=0.1, random_state=42
)

print("Training samples (FCN):", len(X_train_fcn),
      "Validation samples (FCN):", len(X_val_fcn))
```

```
Training samples (FCN): 1926 Validation samples (FCN): 214
```

Create PyTorch Datasets and DataLoaders

We'll create TensorDatasets from the NumPy arrays, then wrap them in DataLoaders for batching and shuffling. Notice that we **did not** reshape the input for the FCN version.

```
train_dataset_fcn = TensorDataset(
    torch.from_numpy(X_train_fcn).to(device), # shape: (batch_size, 9216)
    torch.from_numpy(y_train_fcn).to(device) # shape: (batch_size, 30)
)
val_dataset_fcn = TensorDataset(
    torch.from_numpy(X_val_fcn).to(device),
    torch.from_numpy(y_val_fcn).to(device)
)

g = torch.Generator(device=device)
train_loader_fcn = DataLoader(train_dataset_fcn, batch_size=64, shuffle=True,
                               generator=g)
val_loader_fcn = DataLoader(val_dataset_fcn, batch_size=64, shuffle=False)
```

Instantiate the FCN Model, define criterion and the optimizer

We use an MSE loss since we're doing keypoint regression. We use the Adam optimiser in this exercise but you can experiment with any optimizer you like (Adam, SGD, etc.):

```
from torchsummary import summary
fcn_model = KeypointFCN().to(device)
summary(fcn_model.cpu(), X_train_fcn[0].shape)
fcn_model = fcn_model.to(device)
criterion_fcn = nn.MSELoss()
optimizer_fcn = optim.Adam(fcn_model.parameters(), lr=0.001)
```

```
-----
Layer (type)          Output Shape          Param #
=====
Linear-1              [-1, 1024]            9,438,208
Dropout-2             [-1, 1024]             0
Linear-3              [-1, 512]             524,800
Dropout-4             [-1, 512]             0
Linear-5              [-1, 30]              15,390
=====
Total params: 9,978,398
Trainable params: 9,978,398
Non-trainable params: 0
-----
Input size (MB): 0.04
Forward/backward pass size (MB): 0.02
Params size (MB): 38.06
Estimated Total Size (MB): 38.12
-----
```

Training with ongoing evaluation of the validation error

```
from tqdm.auto import tqdm
epochs = 20
for epoch in (pbar:=tqdm(range(1, epochs+1))):
    # --- Training ---
    fcn_model.train()
    train_loss = 0.0
```

(continues on next page)

(continued from previous page)

```

for batch_images, batch_keypoints in train_loader_fcn:
    outputs = fcn_model(batch_images)
    loss = criterion_fcn(outputs, batch_keypoints)

    optimizer_fcn.zero_grad()
    loss.backward()
    optimizer_fcn.step()

    train_loss += loss.item() * batch_images.size(0)
train_loss /= len(train_loader_fcn.dataset)

# --- Validation ---
fcn_model.eval()
val_loss = 0.0
with torch.no_grad():
    for batch_images, batch_keypoints in val_loader_fcn:
        preds = fcn_model(batch_images)
        loss = criterion_fcn(preds, batch_keypoints)
        val_loss += loss.item() * batch_images.size(0)
val_loss /= len(val_loader_fcn.dataset)

pbar.set_description(f"Epoch {epoch}/{epochs} - Train Loss: {train_loss:.4f} - \
↪Val Loss: {val_loss:.4f}")

```

```
0%|          | 0/20 [00:00<?, ?it/s]
```

Evaluate on validation set

After the training completes, we can compute the validation set predictions and compute appropriate metrics to estimate the performance of the fully connected neural network model:

```

fcn_model.eval()
with torch.no_grad():
    val_preds_fcn = fcn_model(torch.Tensor(X_val_fcn).to(device)).cpu().numpy()

val_truth_fcn = y_val_fcn
mse_val_fcn = np.mean((val_preds_fcn - val_truth_fcn)**2)
rmse_val_fcn = np.sqrt(mse_val_fcn)

print(f"Validation RMSE (FCN): {rmse_val_fcn:.3f} pixels")

```

```
Validation RMSE (FCN): 10.077 pixels
```

Convolutional neural network

We will create a Convolutional Neural Network (CNN) to **regress** the 30 keypoint coordinates from the input image. CNNs are highly effective in image analysis as they can automatically learn spatial hierarchies of features from images – from low-level edges to high-level shapes – which makes them well-suited for detecting patterns like facial features.

The architecture of our CNN will be as follows:

1. **Conv Layer 1:** 1 input channel (grayscale) -> 32 filters, kernel size 3x3, activation ReLU, followed by Max Pooling (2x2).
2. **Conv Layer 2:** 32 -> 64 filters, 3x3, ReLU, followed by Max Pooling (2x2).

3. **Conv Layer 3:** 64 $\rightarrow$ 128 filters, 3x3, ReLU, followed by Max Pooling (2x2).
4. **Flatten:** Flatten the feature maps into a vector.
5. **Fully Connected 1:** 256 neurons, ReLU, with Dropout (to reduce overfitting).
6. **Fully Connected 2:** 128 neurons, ReLU, with Dropout.
7. **Output Layer:** 30 outputs (the x, y coordinates for 15 keypoints), linear activation (no final nonlinearity, since this is a regression output).

Dropout layers will help regularize the network by randomly dropping a fraction of neurons during training, which can improve generalization. We expect the CNN to learn to detect facial features through the convolutional layers (which capture local patterns like edges, corners of eyes/mouth, etc.), and the fully connected layers will combine these to predict the precise coordinates.

```
class KeypointCNN(nn.Module):
    def __init__(self):
        super(KeypointCNN, self).__init__()
        # Convolutional layers
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3) # 96x96 -> 94x94 (then 47x47_
        # after pool
        self.conv2 = nn.Conv2d(32, 64, kernel_size=3) # 47x47 -> 45x45 (then 22x22_
        # after pool
        self.conv3 = nn.Conv2d(64, 128, kernel_size=3) # 22x22 -> 20x20 (then 10x10_
        # after pool
        self.pool = nn.MaxPool2d(2, 2)

        # Dropout layers
        self.dropout1 = nn.Dropout(0.1)
        self.dropout2 = nn.Dropout(0.2)

        # Fully connected layers
        self.fc1 = nn.Linear(128 * 10 * 10, 256) # 128 feature maps * 10 * 10
        self.fc2 = nn.Linear(256, 128)
        self.fc3 = nn.Linear(128, 30) # 30 outputs (x,y for 15 keypoints)

    def forward(self, x):
        # Three conv layers with ReLU and pooling
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))

        # Flatten
        x = x.view(x.size(0), -1)

        # Fully connected layers with dropout and ReLU
        x = F.relu(self.fc1(x))
        x = self.dropout1(x)
        x = F.relu(self.fc2(x))
        x = self.dropout2(x)
        x = self.fc3(x)
        return x
```

Training the model

Now, let's train the model on our training data and monitor its performance on the validation set.

- **Data Loaders:** We'll use PyTorch `DataLoader` to batch and shuffle the data for training and to batch the validation data.

- **Loss Function:** This is a regression problem (predicting continuous coordinate values), so a common choice is **Mean Squared Error** (`nn.MSELoss`) in PyTorch.
- **Optimizer:** We'll use **Adam** for efficient stochastic gradient descent.
- **Training Loop:** We'll train for a number of epochs (iterations over the whole training set). In each epoch:
 1. Set the model to training mode.
 2. Loop over mini-batches from the training loader.
 3. For each batch, do a forward pass to get predictions, compute the loss, do a backward pass to compute gradients, and update weights.
 4. Evaluate the model on the validation set after each epoch to track performance.

```
from sklearn.model_selection import train_test_split

# We'll do a 90/10 train-validation split
X_train_cnn, X_val_cnn, y_train_cnn, y_val_cnn = train_test_split(X_cnn, y, test_
    size=0.1, random_state=42)
print("Training samples:", len(X_train_cnn), "Validation samples:", len(X_val_cnn))
```

Training samples: 1926 Validation samples: 214

```
# Create TensorDatasets for train and validation
train_dataset = TensorDataset(torch.from_numpy(X_train_cnn).to(device), torch.from_
    numpy(y_train_cnn).to(device))
val_dataset = TensorDataset(torch.from_numpy(X_val_cnn).to(device), torch.from_
    numpy(y_val_cnn).to(device))

# DataLoader for batching
g = torch.Generator(device=device)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, generator=g)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False)
```

```
cnn_model = KeypointCNN()
summary(cnn_model.cpu(), X_train_cnn[0].shape)
cnn_model = cnn_model.to(device)
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 32, 94, 94]	320
MaxPool2d-2	[-1, 32, 47, 47]	0
Conv2d-3	[-1, 64, 45, 45]	18,496
MaxPool2d-4	[-1, 64, 22, 22]	0
Conv2d-5	[-1, 128, 20, 20]	73,856
MaxPool2d-6	[-1, 128, 10, 10]	0
Linear-7	[-1, 256]	3,277,056
Dropout-8	[-1, 256]	0
Linear-9	[-1, 128]	32,896
Dropout-10	[-1, 128]	0
Linear-11	[-1, 30]	3,870
Total params: 3,406,494		
Trainable params: 3,406,494		
Non-trainable params: 0		

(continues on next page)

(continued from previous page)

```
-----
Input size (MB): 0.04
Forward/backward pass size (MB): 4.42
Params size (MB): 12.99
Estimated Total Size (MB): 17.45
-----
```

```
criterion = nn.MSELoss() # Mean Squared Error loss
optimizer = optim.Adam(cnn_model.parameters(), lr=0.001)

epochs = 20
for epoch in (pbar:=tqdm(range(1, epochs+1))):
    cnn_model.train()
    train_loss = 0.0
    for batch_images, batch_keypoints in train_loader:
        images = batch_images
        keypoints = batch_keypoints

        outputs = cnn_model(images)
        loss = criterion(outputs, keypoints)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        train_loss += loss.item() * images.size(0)
    train_loss /= len(train_loader.dataset)

    # Validation phase
    cnn_model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for batch_images, batch_keypoints in val_loader:
            images = batch_images
            keypoints = batch_keypoints
            preds = cnn_model(images)
            loss = criterion(preds, keypoints)
            val_loss += loss.item() * images.size(0)
        val_loss /= len(val_loader.dataset)

    pbar.set_description(f"Epoch {epoch}/{epochs} - Training Loss: {train_loss:.4f} - \n
    Validation Loss: {val_loss:.4f}")
```

```
0%|          | 0/20 [00:00<?, ?it/s]
```

Evaluation and results

After training, we'll evaluate the model's performance. One common metric for this task is **Root Mean Squared Error (RMSE)** of the keypoint predictions. RMSE is the square root of MSE, giving an error in the same units as the coordinates (pixels). We will calculate RMSE on the validation set.

For instance, an RMSE of 4.0 would mean on average the predictions are about 4 pixels away from the true keypoint positions.

```
# Switch to evaluation mode
cnn_model.eval()
```

(continues on next page)

(continued from previous page)

```
# Predict on the entire validation set
with torch.no_grad():
    val_preds = cnn_model(torch.Tensor(X_val_cnn).to(device)).cpu().numpy()

# Compute RMSE on validation set
val_truth = y_val_cnn # actual keypoints
mse_val = np.mean((val_preds - val_truth)**2)
rmse_val = np.sqrt(mse_val)
print(f"Validation RMSE: {rmse_val:.3f} pixels")
```

```
Validation RMSE: 4.997 pixels
```

Visualizing predictions

Finally, let's visualize some predictions to see how the model is performing. We'll take a few images from the validation set (which the model hasn't seen during training) and plot the image with:

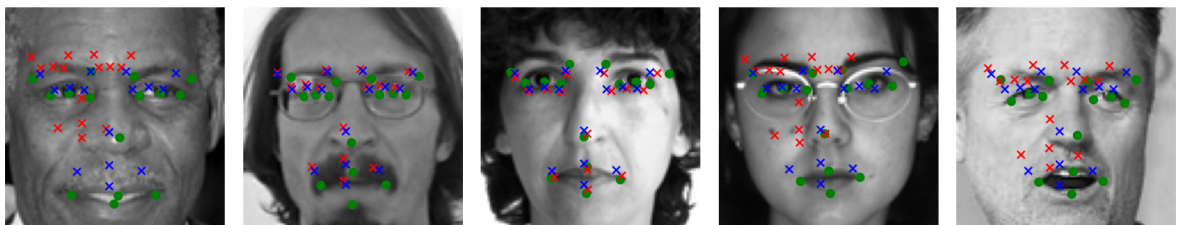
- The **ground truth** keypoints (in green)
- The **predicted** keypoints (in red)

```
# Choose a few samples from validation to visualize
num_samples_to_show = 5
indices = np.random.choice(len(X_val), size=num_samples_to_show, replace=False)

fig, axes = plt.subplots(1, num_samples_to_show, figsize=(10, 6))
for i, idx in enumerate(indices):
    img = X_val_fcn[idx].reshape(96, 96)
    true_kp = y_val_fcn[idx]
    cnn_pred_kp = val_preds[idx]
    fcn_pred_kp = fcn_model(torch.tensor(X_val_fcn[idx].reshape(1, 96*96)).to(device)).cpu().detach().numpy()[0]

    axes[i].imshow(img, cmap='gray')
    # Plot true keypoints in green
    axes[i].scatter(true_kp[0::2], true_kp[1::2], c='green', s=20, label='True')
    # Plot predicted keypoints in red
    axes[i].scatter(fcn_pred_kp[0::2], fcn_pred_kp[1::2], c='red', s=20, marker='x',
    label='Pred')
    # Plot predicted keypoints in red
    axes[i].scatter(cnn_pred_kp[0::2], cnn_pred_kp[1::2], c='blue', s=20, marker='x',
    label='Pred')
    axes[i].axis('off')

plt.tight_layout()
plt.show()
```



In the displayed images, **green** dots should mark the actual keypoint positions and **red** x's mark the model's predicted

positions. Ideally, they will be close for most points. You might notice the model does well on prominent features like the eye centers or nose tip, but could be less accurate on some others, especially if the network hasn't fully converged or if data is limited. A deeper network or additional data augmentation could further improve performance.

ENSEMBLES OF MODELS AND FEATURE SELECTION

Let us define some basic imports and a pprint function.

```
import numpy as np
import pandas as pd
import sklearn
import warnings
warnings.filterwarnings('ignore')
from tqdm.auto import tqdm
from IPython.display import display, Math
from numpyarray_to_latex.jupyter import to_ltx
import matplotlib.pyplot as plt
plt.style.use(['seaborn-v0_8-muted', 'practicals.mplstyle'])
import seaborn as sns
def pprint(*args):
    res = ""
    for i in args:
        if type(i) == np.ndarray:
            res += to_ltx(i, brackets='[]')
        elif type(i) == str:
            res += i
    display(Math(res))
```

7.1 Reminder: Supervised learning

Supervised learning aims to model the (unknown) stochastic dependence between a set of n inputs x (also called features) and an output y on the basis of a training set D_N of size N . Supervised learning tasks are decomposed into *regression* and *classification* tasks.

7.1.1 Regression

In regression the stochastic dependence is given by

$$\mathbf{y} = f(x) + \mathbf{w}$$

where:

- $\mathbf{y} \in \mathbb{R}$ represents the output variable (also called target)
- $x \in \mathbb{R}^n$ represents the vector of inputs (also called features)
- $\mathbf{w}$ denotes the noise, and is typically assumed that $E[\mathbf{w}] = 0$ and the variance $\text{Var}[\mathbf{w}] = \sigma_{\mathbf{w}}^2$ is constant

- $f(x) = E[y|x]$ is the (unknown) mapping between input and outputs, also known as the *regression function*.

In regression, the goal of learning is to return an estimator

$$h(x, \alpha)$$

of the regression function $f(x)$, where α denotes the set of parameters of the model h .

7.1.2 Error measurement

We already have used the *mean squared error* as an evaluation tool for the quality of the predictions of a model. Unfortunately, this tool does not take into account how “variable” the expected outputs are, and therefore two models could become undistinguishable when using the MSE. We can normalize the MSE by dividing it by the (measured) variance of the output data. This leads to the *normalized mean squared error*:

$$\text{NMSE} = \frac{\sum (y - \hat{y})^2}{N\sigma_y^2}$$

We can see that the most naive model which outputs the mean of the observed outputs should give:

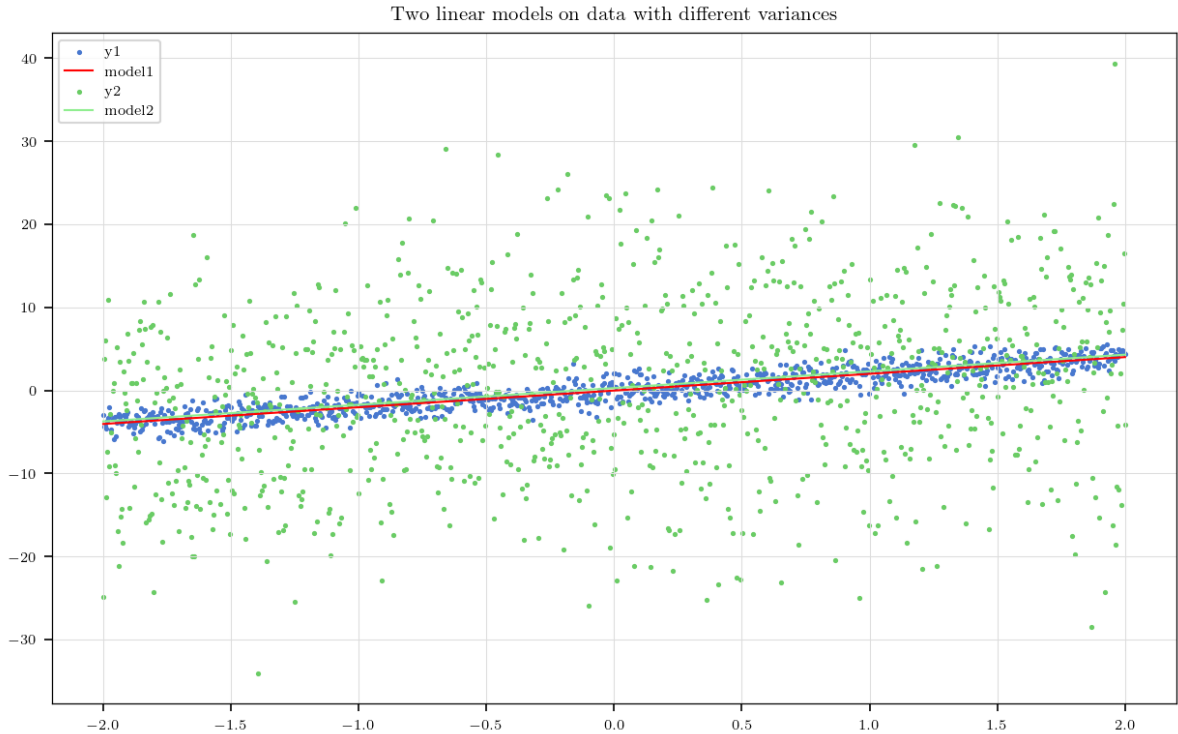
$$\frac{\sum (y - \mu_y)^2}{N\sigma_y^2} = \frac{\sigma_y^2}{\sigma_y^2} = 1$$

Any model that has a NMSE greater than 1 is therefore performing worse than the most naive model.

```
from sklearn.linear_model import LinearRegression
x = np.linspace(-2, 2, 1000).reshape(1000,-1)
B1, B2 = np.random.normal(0, 1, size=(1000,1)), np.random.normal(0, 10, size=(1000,1))
A = 2
y1 = A*x+B1
y2 = A*x+B2

model1 = LinearRegression().fit(x, y1)
model2 = LinearRegression().fit(x, y2)

plt.figure(figsize=(10, 6))
plt.scatter(x, y1, marker='.', label='y1')
plt.plot(x, model1.predict(x), color='red', label='model1')
plt.scatter(x, y2, marker='.', label='y2')
plt.plot(x, model2.predict(x), color='lightgreen', label='model2')
plt.title("Two linear models on data with different variances")
plt.legend()
plt.grid("on")
plt.show()
```



```
def mse(y, y_hat):
    return np.mean((y-y_hat)**2)

def nmse(y, y_hat):
    return np.mean((y-y_hat)**2)/np.std(y)**2
```

```
print(f"MSEs: {mse(y1, model1.predict(x))}, {mse(y2, model2.predict(x))}")
print(f"NMSEs: {nmse(y1, model1.predict(x))}, {nmse(y2, model2.predict(x))}")
```

```
MSEs: 0.9950393395460394, 103.5326025188694
NMSEs: 0.1552969623290508, 0.9504494009358256
```

7.2 Feature selection

Feature selection and ensembles of models are two techniques which can be used to improve the accuracy of predictions.

Feature selection aims at reducing the dimensionality of the problem, and is useful when input variables contain redundant or irrelevant (noisy) information. Benefits are twofold: it decreases the training time by simplifying the problem, and it decreases the complexity of the predictive model. This in turn usually improves the prediction accuracy, since high-dimensionality makes predictive models more prone to overfitting, and estimates of parameters more variant.

There are three main approaches to feature selection:

- **Filter methods:** These methods rely solely on the data and their intrinsic properties, without considering the impact of the selected features on the learning algorithm performance. For this reason, they are often used as preprocessing techniques.
- **Wrapper methods:** These methods assess subsets of variables according to their usefulness to a given predictor. The feature selection is performed using an evaluation function that includes the predictive performance of the

considered learning algorithm as a selection criterion.

- **Embedded methods:** These methods are specific to given learning machines, and usually built-in in the learning procedure (e.g. random forest, regularization based techniques).

Ensembles of models consist in building several predictive models using resampled subsets of the original training set. The method works particularly well for predictive models with high variance (for example, decision trees or neural networks). The average prediction of the resulting models usually strongly decreases the variance component of the error, and as a consequence improves the prediction accuracy.

In this session, we will use a synthetic dataset, made of 30 continuous variables. 24 of them will be useless, and 6 will serve in the output. The output will be a function of the useful input variables.

7.2.1 Data overview and preprocessing

```
def compute(data):
    data['y'] = np.zeros(data.shape[0])
    for i in range(6):
        if i%3 == 0:
            data['y'] += np.random.uniform(-1, 1) * data[f'x_useful_{i+1}'] + np.
random.uniform(-5, 5)
        elif i%3 == 1:
            data['y'] += np.random.uniform(-1, 1) * data[f'x_useful_{i+1}']**2 + np.
random.uniform(-5, 5)
        elif i%3 == 2:
            data['y'] += np.random.uniform(-1, 1) * np.sin(data[f'x_useful_{i+1}']) +
np.random.uniform(-5, 5)
    return data
```

```
n_x_useful = 6
n_x_useless = 24
n_samples = 1000
d = {}
np.random.seed(0)

for i in range(n_x_useful):
    m, s = np.random.uniform(-1, 1), np.random.uniform(0.5, 2.5)
    d[f'x_useful_{i+1}'] = np.random.normal(m, s, size=(n_samples,))
for i in range(n_x_useless):
    m, s = np.random.uniform(-1, 1), np.random.uniform(0.5, 2.5)
    d[f'x_useless_{i+1}'] = np.random.normal(m, s, size=(n_samples,))

data = pd.DataFrame(d)

data = compute(data)
```

```
data.describe(include='all')
```

	x_useful_1	x_useful_2	x_useful_3	x_useful_4	x_useful_5	\
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	
mean	0.008883	0.787868	-0.862362	-0.220739	0.900991	
std	1.904117	2.362233	1.466222	0.881817	1.535395	
min	-5.782583	-6.520188	-5.587836	-3.444757	-3.751236	
25%	-1.250588	-0.839990	-1.910921	-0.821390	-0.111143	
50%	-0.014389	0.822298	-0.868373	-0.229933	0.883910	
75%	1.269272	2.284124	0.175776	0.372973	1.957939	

(continues on next page)

(continued from previous page)

max	5.424227	8.481613	3.712036	3.085683	5.435128	
	x_useful_6	x_useless_1	x_useless_2	x_useless_3	x_useless_4	...
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	...
mean	0.048531	0.848947	0.019645	0.655122	-0.189598	...
std	0.512844	1.362313	1.729953	1.665836	0.771923	...
min	-1.380297	-3.305645	-5.469208	-4.426479	-2.355184	...
25%	-0.293415	-0.091018	-1.090598	-0.432532	-0.738497	...
50%	0.050555	0.825699	0.014043	0.655699	-0.199494	...
75%	0.386908	1.786537	1.128925	1.744190	0.350852	...
max	1.539755	4.825280	5.912388	5.919659	2.026439	...
	x_useless_16	x_useless_17	x_useless_18	x_useless_19	x_useless_20	...
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	...
mean	-0.421467	0.821257	-0.011411	0.441756	-0.958489	...
std	0.922338	0.862039	2.287231	1.402679	1.452495	...
min	-3.443265	-1.763982	-7.985982	-4.070035	-5.422766	...
25%	-1.025653	0.250196	-1.464619	-0.527079	-1.967275	...
50%	-0.423784	0.844540	0.021261	0.423711	-1.010335	...
75%	0.211144	1.361641	1.522601	1.368873	0.002406	...
max	3.010346	4.035696	7.689106	5.682900	3.504355	...
	x_useless_21	x_useless_22	x_useless_23	x_useless_24	y	
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000	
mean	-0.384471	-0.152625	-0.518022	0.051116	-7.379040	
std	0.787796	0.610938	2.501076	1.985946	1.890504	
min	-2.947481	-1.917751	-11.958636	-6.453296	-13.989287	
25%	-0.890462	-0.564771	-2.155428	-1.329528	-8.514752	
50%	-0.403122	-0.147460	-0.412912	0.054168	-7.414389	
75%	0.173736	0.252462	1.202683	1.373122	-6.287151	
max	2.146540	1.745356	8.160750	5.685927	2.202998	

[8 rows x 31 columns]

7.2.2 Input and output variables

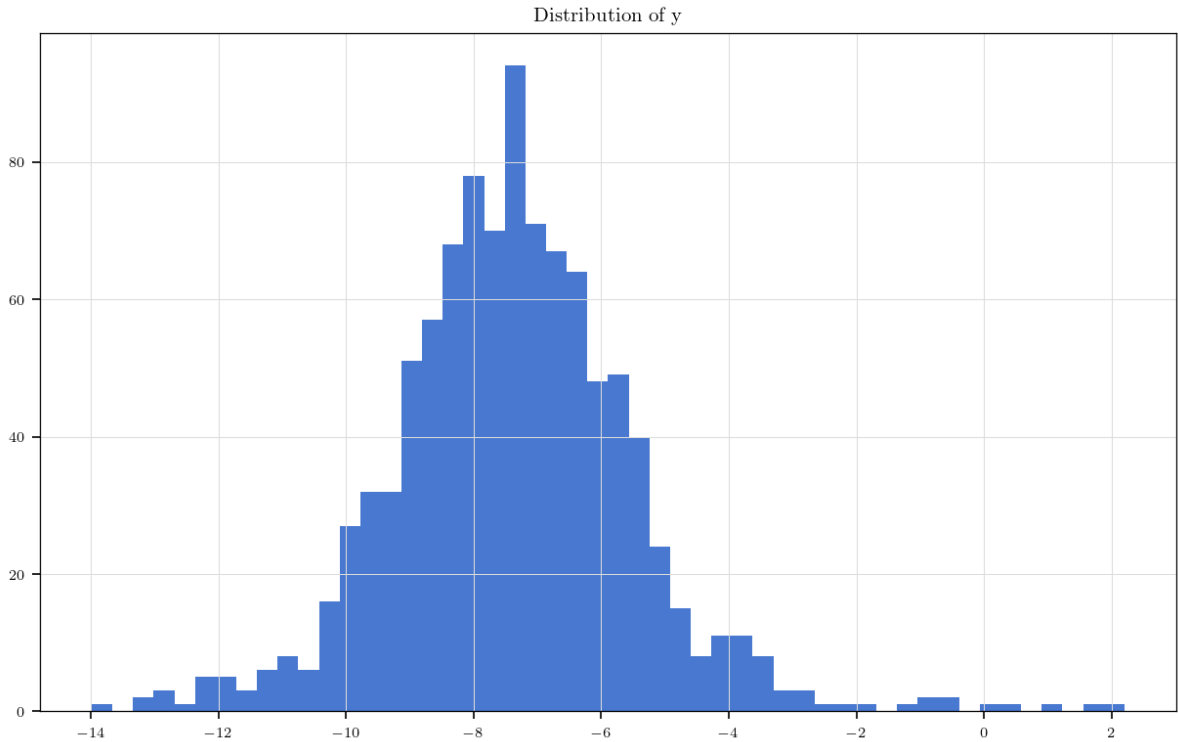
The output variable (Y) is y, and all other variables (X) are considered as inputs.

```
Y = data['y'].values
X = data.drop(columns=['y'])

N = X.shape[0]
n = X.shape[1]

plt.figure(figsize=(10, 6))
plt.hist(Y, bins=50)
plt.title("Distribution of y")
plt.grid("on")
plt.show()

print("Mean of Y:", np.mean(Y))
print("Variance of Y:", np.var(Y))
```



Mean of Y: -7.379040070934719
 Variance of Y: 3.5704323990926374

7.2.3 Modelling with linear and decision tree models

Linear model

- Create a linear model $h(x)$ for predicting the IMDB score on the basis of the input variables, and compute its empirical (or training) mean square error.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, Y)
Y_hat = model.predict(X)

empirical_error = nmse(Y, Y_hat)
print(f"Empirical error: {empirical_error:.5f}")
```

Empirical error: 0.55375

- Which input variables are statistically correlated with the output?

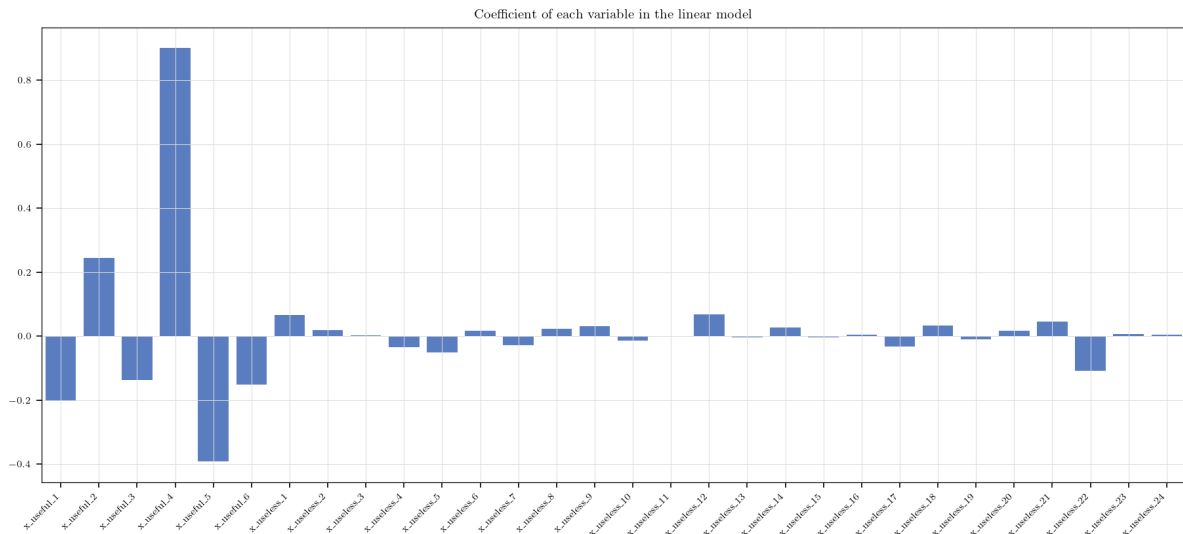
In Python, we can check the coefficients of the linear model.

```
coefs = pd.Series(model.coef_, index=X.columns)
plt.figure(figsize=(15, 6))
sns.barplot(coefs)
plt.title("Coefficient of each variable in the linear model")
```

(continues on next page)

(continued from previous page)

```
plt.xticks(rotation=45, ha='right')
plt.grid("on")
plt.show()
```



- Compute the validation error with a 10-fold cross-validation

```
from sklearn.model_selection import KFold

kf = KFold(n_splits=10, shuffle=True, random_state=3)
CV_err_lm_single_model = []

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    model = LinearRegression()
    model.fit(X_tr, Y_tr)
    Y_hat_ts = model.predict(X_ts)
    CV_err_lm_single_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_lm_single_model):.5f}, std dev: {np.std(CV_err_lm_
↵single_model):.5f}")
```

CV error: 0.61060, std dev: 0.08403

Decision tree

- Modify the previous code to compute the empirical error using a decision tree model. Use sklearn's DecisionTreeRegressor.

```
from sklearn.tree import DecisionTreeRegressor

model = DecisionTreeRegressor(random_state=3)
model.fit(X, Y)
Y_hat = model.predict(X)
```

(continues on next page)

(continued from previous page)

```
empirical_error = nmse(Y, Y_hat)
print(f"Empirical error: {empirical_error:.5f}")
```

```
Empirical error: 0.00000
```

- Plot the resulting tree is more complicated in Python due to size, but we can just visualize its structure or get the depth.

```
print(f"Depth: {model.get_depth()}, number of leaves: {model.get_n_leaves()}")
```

```
Depth: 22, number of leaves: 1000
```

- What is the 10-fold cross-validation error using a decision tree model?

```
CV_err_rpart_single_model = []

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    model = DecisionTreeRegressor(random_state=3)
    model.fit(X_tr, Y_tr)
    Y_hat_ts = model.predict(X_ts)
    CV_err_rpart_single_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_rpart_single_model):.5f}, std dev: {np.std(CV_err_
    rpart_single_model):.5f}")
```

```
CV error: 0.32932, std dev: 0.08640
```

Why is the result so different using the 10-fold cross-validation?

Ridge regression with LOO-CV

Recall: Ridge regression looks like a linear regression but includes a penalty term for large coefficients:

$$\min_{\beta} \|Y - X\beta\|^2 + \lambda\|\beta\|^2.$$

Ridge also has a known closed-form:

$$\hat{\beta}_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T Y.$$

- Leave-one out cross-validation is a method that consists in evaluating a given model by training it on the entire training set, except for one sample used for validation. By performing this validation, eliminating each of the samples at a time, we get an estimation of the quality of the model.
- Then, the coefficients can give an idea of the importance of each of the features in the prediction task.

Here, we limit the dataset size to 50 items, to avoid too long computations.

```
X_np = X.to_numpy()[:50]
N = X_np.shape[0]
```

(continues on next page)

(continued from previous page)

```

lambdas = np.linspace(0.01, 0.5, 10)
E = np.empty((N, len(lambdas)))
E[:] = np.nan

for i in tqdm(range(N)):
    Xtr = np.column_stack((np.ones(N - 1), np.delete(X_np, i, axis=0)))
    Ytr = np.delete(Y[:50], i)

    Xts = np.concatenate(([1], X_np[i,]))

    cnt = 0
    for l in lambdas:
        A = Xtr.T @ Xtr + l * np.eye(n + 1)
        b = Xtr.T @ Ytr
        betahat = np.linalg.inv(A) @ b
        Yhati = Xts @ betahat
        E[i, cnt] = ((Y[i] - Yhati) ** 2)
        cnt += 1

mseloo = np.mean(E, axis=0)/np.std(Y)**2

bestLambda = lambdas[np.argmin(np.mean(E, axis=0))]
print(f"Best lambda: {bestLambda}")

XX = np.column_stack((np.ones(N), X_np))
A_final = np.dot(XX.T, XX) + bestLambda * np.eye(n + 1)
b_final = np.dot(XX.T, Y[:50])
betahat = np.linalg.inv(A_final) @ XX.T @ Y[:50]

abs_betahat = np.abs(betahat)
sorted_indices = np.argsort(-abs_betahat)
top4 = sorted_indices[:4]
print(f"Greatest coefficients indexes: {top4}")
print(f"Greatest coefficients: {betahat[top4]}")
plt.figure(figsize=(10, 6))
plt.plot(lambdas, mseloo)
plt.title('NMSE.1oo vs Lambda')
plt.grid("on")
plt.show()

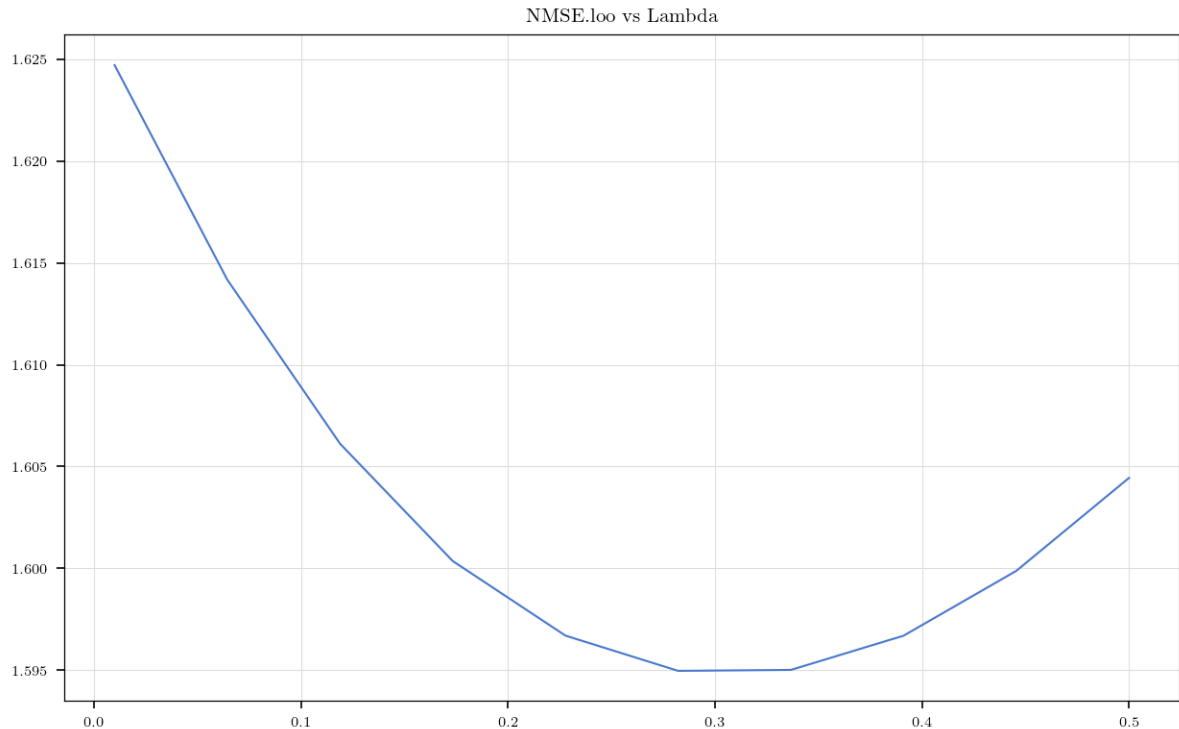
```

0% | 0/50 [00:00<?, ?it/s]

```

Best lambda: 0.2822222222222222
Greatest coefficients indexes: [ 0  4 27 23]
Greatest coefficients: [-7.1367055  0.89720319 -0.34672033 -0.33732339]

```



Selection bias

Take care that if the features are selected using the error on the training set as an importance criterium leads to a flaw: the features may be over-fittingly selected and the performances then do not reflect on the testing performances dataset.

```
def pred(X_train, Y_train, X_test):
    reg = sklearn.tree.DecisionTreeRegressor().fit(X_train, Y_train)
    return reg.predict(X_test)

X_tr = X.to_numpy()[:200]
Y_tr = Y[:200]
X_ts = X.to_numpy()[200:250]
Y_ts = Y[200:250]

N_tr = X_tr.shape[0]

fset = []
best = []
bests = []

for it in tqdm(range(15)):
    NMSEf = {}
    candidates = [f for f in range(n) if f not in fset]
    for f in candidates:
        e = np.empty(N_tr)
        for i in range(N_tr):
            X_train_all = np.delete(X_tr, i, axis=0)
            Y_train_all = np.delete(Y_tr, i)
            cols = [j - 1 for j in (fset + [f])]
            X_train = X_train_all[:, cols]
```

(continues on next page)

(continued from previous page)

```

        X_test_sample = X_tr[i, cols]
        Yhati = pred(X_train, Y_train_all, X_test_sample.reshape(1, -1))
        e[i] = (Y[i] - Yhati)**2/np.std(Y_tr)**2
    NMSEf[f] = np.mean(e)
    f_best = min(NMSEf, key=NMSEf.get)
    fset.append(f_best)
    bestr.append(NMSEf[f_best])
    cols = [j - 1 for j in fset]
    Y_hats = pred(X_tr[:, cols], Y_tr, X_ts[:, cols])
    nmse_test = nmse(Y_hats, Y_ts)
    bests.append(nmse_test)

pprint(fset)
pprint(np.array(bests) - np.array(bestr))

plt.figure(figsize=(10, 6))
plt.plot(bests, color="red", label="Test NMSE")
plt.plot(bestr, linestyle="--", linewidth=2, color="black", label="Internal NMSE LOO")
plt.ylim(0, max(bests))
plt.title("Selection bias and feature selection")
plt.xlabel("Feature set size")
plt.ylabel("")
plt.legend(loc="upper left")
plt.grid("on")
plt.show()

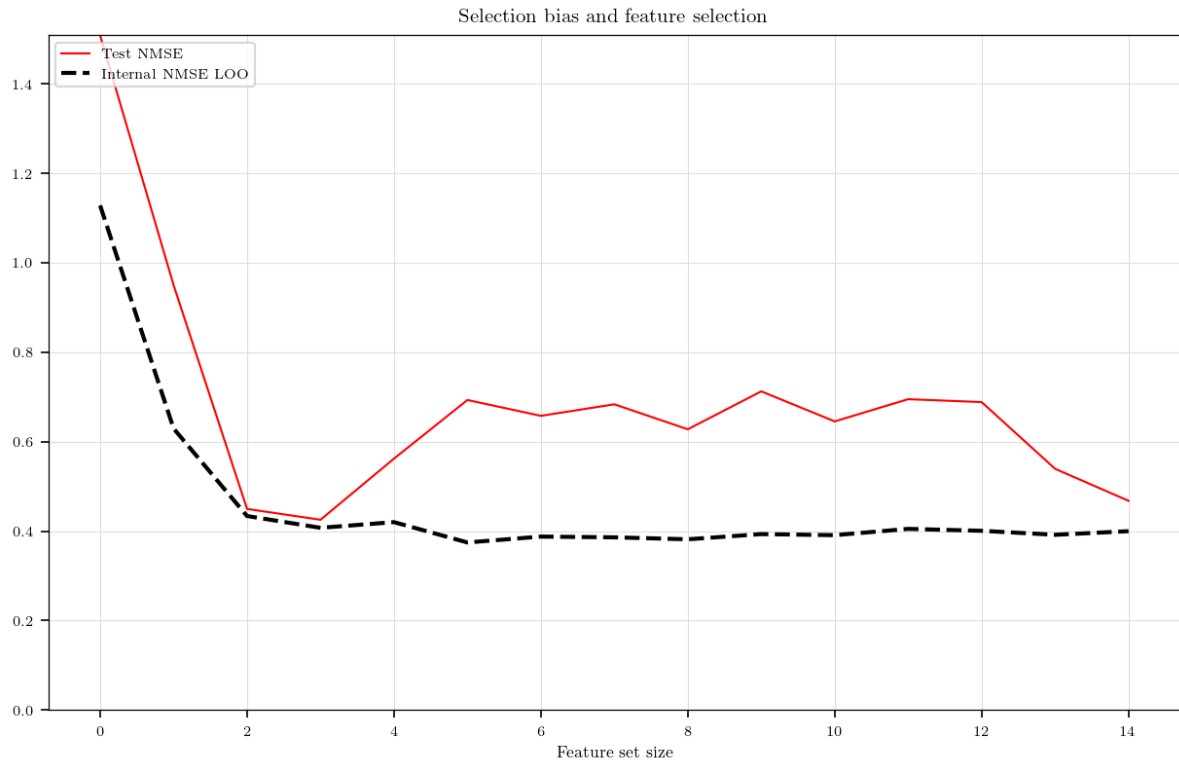
```

0% | 0/15 [00:00<?, ?it/s]

```

[0.3802
 0.3206
 0.0159
 0.0180
 0.1419
 0.3184
 0.2698
 0.2973
 0.2459
 0.3190
 0.2543
 0.2898
 0.2876
 0.1475
 0.0678]

```



7.2.4 Ensemble of models

Let us now create an ensemble of $R=20$ linear models to make predictions.

- Use a linear model as the base model.

```
R = 20
CV_err_lm_ensemble_model = []

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    Y_hat_ts_ensemble = np.zeros((X_ts.shape[0], R))
    for r in range(R):
        idx_tr_resample = np.random.choice(train_index, size=len(train_index),
        ↪replace=True)
        X_tr_res = X.iloc[idx_tr_resample]
        Y_tr_res = Y[idx_tr_resample]

        model = LinearRegression()
        model.fit(X_tr_res, Y_tr_res)
        Y_hat_ts_ensemble[:, r] = model.predict(X_ts)

    Y_hat_ts = np.mean(Y_hat_ts_ensemble, axis=1)
    CV_err_lm_ensemble_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_lm_ensemble_model):.5f}, std dev: {np.std(CV_err_lm_
↪ensemble_model):.5f}")
```

(continues on next page)

(continued from previous page)

```
print("Is ensemble error lower than single model?", np.mean(CV_err_lm_ensemble_model)
      < np.mean(CV_err_lm_single_model))
```

```
CV error: 0.60855, std dev: 0.08307
Is ensemble error lower than single model? True
```

- Use a decision tree as the base model. Is the CV error lower?

```
R = 20
CV_err_rpart_ensemble_model = []

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    Y_hat_ts_ensemble = np.zeros((X_ts.shape[0], R))
    for r in range(R):
        idx_tr_resample = np.random.choice(train_index, size=len(train_index),
      replace=True)
        X_tr_res = X.iloc[idx_tr_resample]
        Y_tr_res = Y[idx_tr_resample]

        model = DecisionTreeRegressor(random_state=r)
        model.fit(X_tr_res, Y_tr_res)
        Y_hat_ts_ensemble[:, r] = model.predict(X_ts)

    Y_hat_ts = np.mean(Y_hat_ts_ensemble, axis=1)
    CV_err_rpart_ensemble_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_rpart_ensemble_model):.5f}, std dev: {np.std(CV_err_
rpart_ensemble_model):.5f}")
print("Is ensemble error lower than single model?", np.mean(CV_err_rpart_ensemble_
model) < np.mean(CV_err_rpart_single_model))
```

```
CV error: 0.16061, std dev: 0.05035
Is ensemble error lower than single model? True
```

7.3 Feature selection

7.3.1 Filter methods

Correlation with the output

The following code performs feature selection by keeping the most correlated variables with the output. Compare the results for linear models and decision trees. What are the smallest CV errors, and how many features were needed?

```
correlations = np.abs(X.corrwith(pd.Series(Y)))
ranking_corr_idx = correlations.sort_values(ascending=False).index

CV_err = np.zeros((n, 10))

fold_id = 0
```

(continues on next page)

(continued from previous page)

```

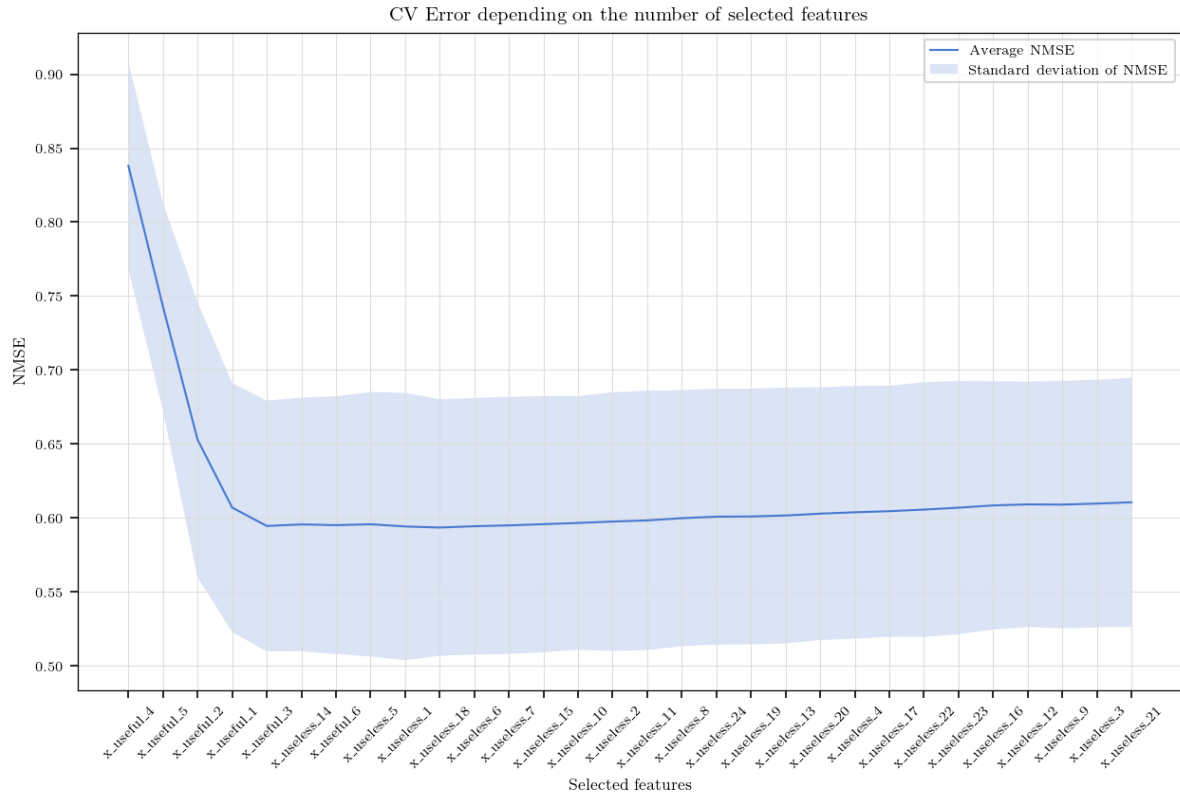
for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    for nb_features in range(1, n+1):
        selected_features = ranking_corr_idx[:nb_features]
        model = LinearRegression()
        model.fit(X_tr[selected_features], Y_tr)
        Y_hat_ts = model.predict(X_ts[selected_features])
        CV_err[nb_features-1, fold_id] = nmse(Y_ts, Y_hat_ts)
    fold_id += 1

x = list(range(1, n+1))
errors = np.array([np.mean(CV_err[i, :]) for i in range(n)])
stds = np.array([np.std(CV_err[i, :]) for i in range(n)])

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation of NMSE")
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Selected features")
ax.set_ylabel("NMSE")
ax.set_xticks(ticks=x, labels=ranking_corr_idx.tolist(), rotation=45)
plt.legend()
plt.grid("on")
plt.show()

```



mRMR

We will implement a simple mRMR feature selection. mRMR uses mutual information. We will approximate mutual information via the correlation-based formula provided.

```
def mutual_info_corr(X, Y):
    c = np.corrcoef(X, Y)[0,1]
    # Avoid invalid value if correlation == 1 or == -1
    if abs(c)==1:
        c = 0.999999
    return -0.5 * np.log(1 - c**2)

def compute_mi_vector(X_tr, Y_tr):
    mis = []
    for col in X_tr.columns:
        mi = mutual_info_corr(X_tr[col].values, Y_tr)
        mis.append(mi)
    return np.array(mis)

CV_err = np.zeros((n,10))

fold_id = 0
for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    mutual_info_values = compute_mi_vector(X_tr, Y_tr)
    selected = []
    candidates = list(range(n))

    for j in range(n):
        redundancy_score = np.zeros(len(candidates))
        if len(selected)>0:
            # Compute pairwise mi between selected and candidates
            mi_sc = []
            for cidx in candidates:
                col_c = X_tr.iloc[:, cidx]
                mis_c = []
                for sidx in selected:
                    col_s = X_tr.iloc[:, sidx]
                    # Compute mutual info between col_s and col_c
                    cc = np.corrcoef(col_s, col_c)[0,1]
                    if abs(cc)==1:
                        cc=0.999999
                    mis_c.append(-0.5*np.log(1-cc**2))
                redundancy_score[candidates.index(cidx)] = np.mean(mis_c)
            mRMR_score = mutual_info_values[candidates] - redundancy_score
            best_idx = candidates[np.argmax(mRMR_score)]
            selected.append(best_idx)
            candidates.remove(best_idx)

    # selected is the ranking
    for nb_features in range(1, n+1):
        features_to_use = [X.columns[i] for i in selected[:nb_features]]
        model = LinearRegression()
        model.fit(X_tr[features_to_use], Y_tr)
        Y_hat_ts = model.predict(X_ts[features_to_use])
        CV_err[nb_features-1, fold_id] = nmse(Y_ts, Y_hat_ts)
```

(continues on next page)

(continued from previous page)

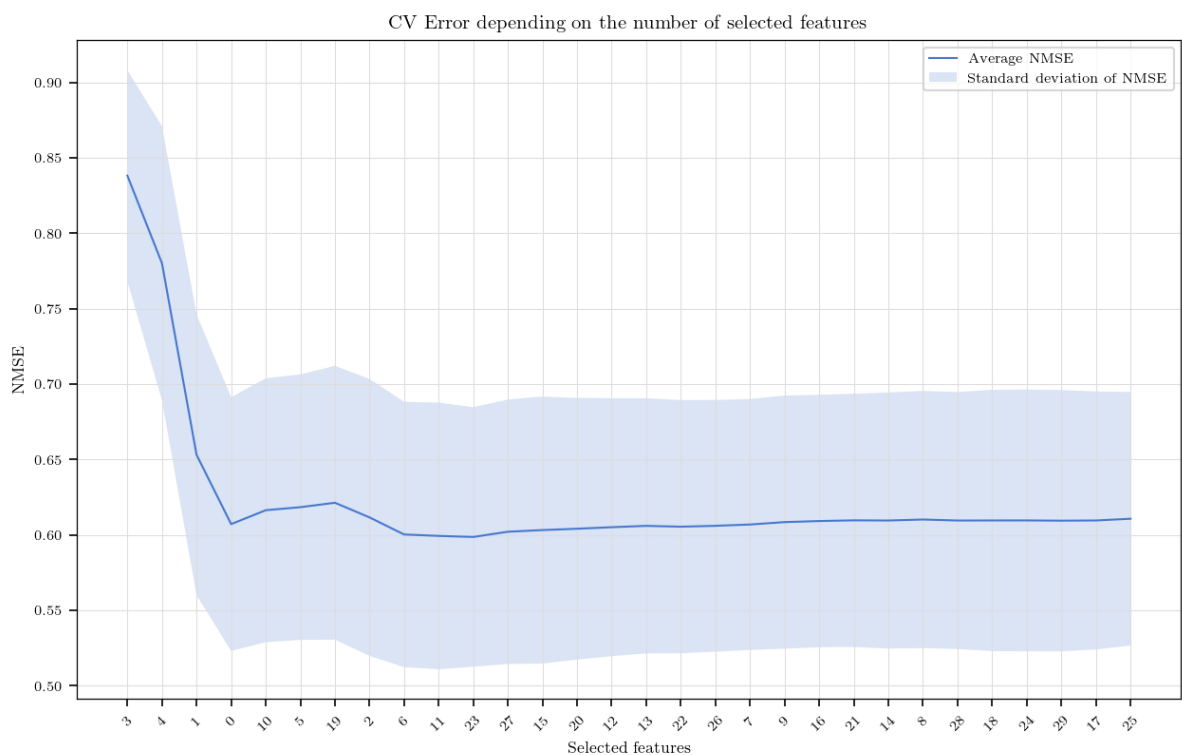
```

fold_id += 1

x = list(range(1, n+1))
errors = np.array([np.mean(CV_err[i, :]) for i in range(n)])
stds = np.array([np.std(CV_err[i, :]) for i in range(n)])

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation of NMSE")
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Selected features")
ax.set_ylabel("NMSE")
ax.set_xticks(ticks=x, labels=selected, rotation=45)
plt.legend()
plt.grid("on")
plt.show()

```



PCA

The following code performs features selection by first transforming the inputs using PCA, and then keeping the most relevant principal components in the model.

```

from sklearn.decomposition import PCA

pca = PCA()
X_pca = pca.fit_transform(X)

```

(continues on next page)

(continued from previous page)

```

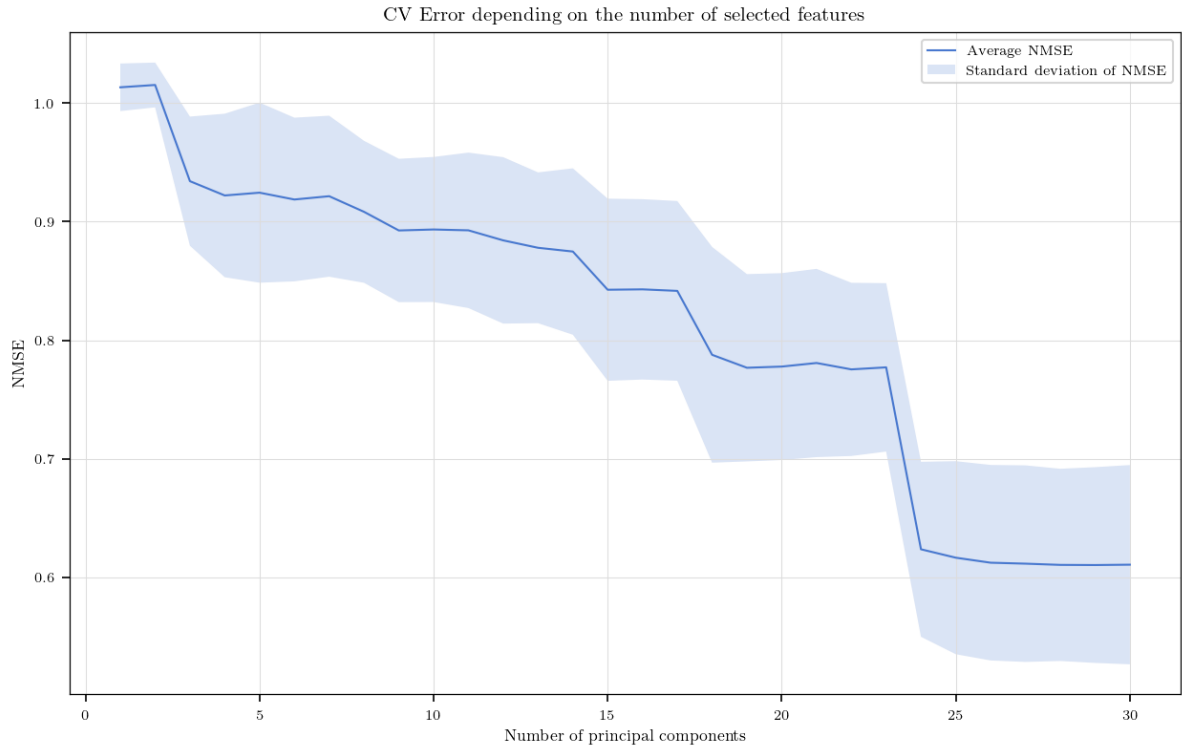
CV_err = np.zeros((n,10))
fold_id = 0
for train_index, test_index in kf.split(X_pca):
    X_tr, X_ts = X_pca[train_index], X_pca[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    for nb_components in range(1, n+1):
        model = LinearRegression()
        model.fit(X_tr[:, :nb_components], Y_tr)
        Y_hat_ts = model.predict(X_ts[:, :nb_components])
        CV_err[nb_components-1, fold_id] = nmse(Y_ts, Y_hat_ts)
    fold_id += 1

x = list(range(1, n+1))
errors = np.array([np.mean(CV_err[i, :]) for i in range(n)])
stds = np.array([np.std(CV_err[i, :]) for i in range(n)])

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation of NMSE")
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Number of principal components")
ax.set_ylabel("NMSE")
plt.legend()
plt.grid("on")
plt.show()

```



7.3.2 Wrapper methods

```

selected = []
CV_errs = []
std_errs = []
for round_i in tqdm(range(n)):
    candidates = list(set(range(n)) - set(selected))
    CV_err_temp = []
    for c in candidates:
        features_to_include = selected + [c]
        fold_errors = []
        for train_index, test_index in kf.split(X):
            X_tr, X_ts = X.iloc[train_index, features_to_include], X.iloc[test_index,
            features_to_include]
            Y_tr, Y_ts = Y[train_index], Y[test_index]

            model = LinearRegression()
            model.fit(X_tr, Y_tr)
            Y_hat_ts = model.predict(X_ts)
            fold_errors.append(nmse(Y_ts, Y_hat_ts))
        CV_err_temp.append(np.mean(fold_errors))

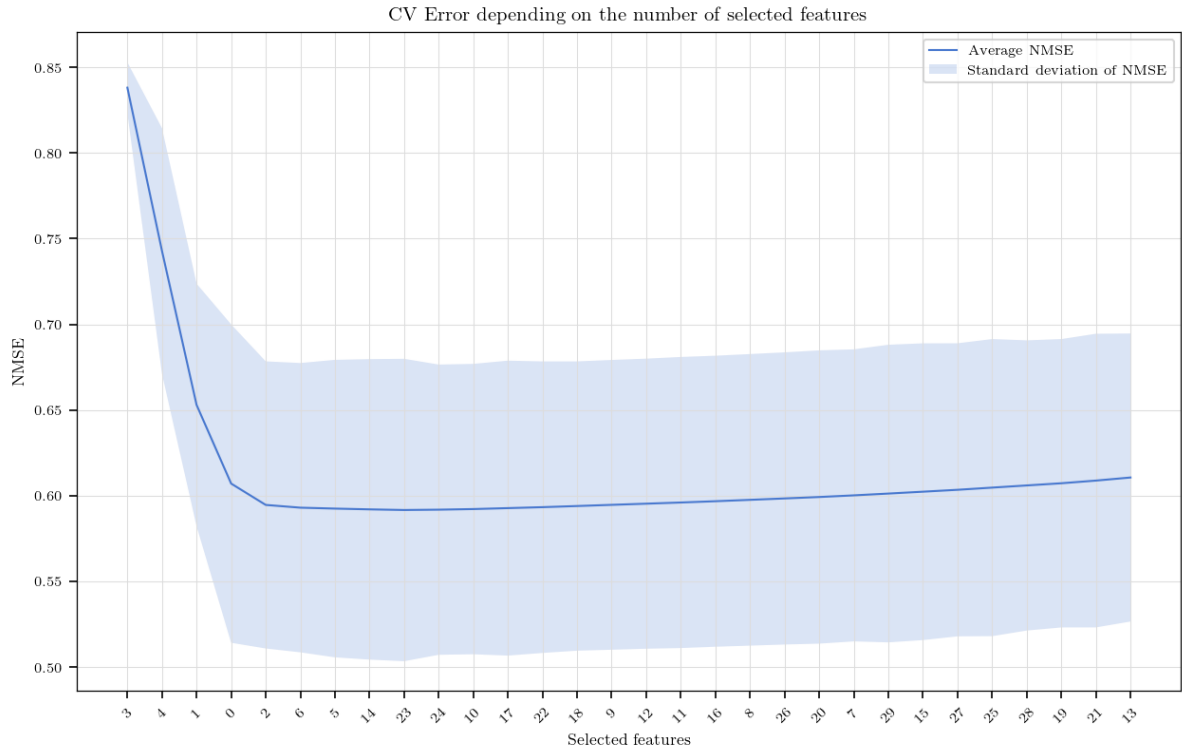
    best_candidate = candidates[np.argmin(CV_err_temp)]
    selected.append(best_candidate)
    CV_errs.append(min(CV_err_temp))
    std_errs.append(np.std(fold_errors))

x = list(range(1, n+1))
errors = np.array(CV_errs)
stds = np.array(std_errs)

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation
of NMSE")
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Selected features")
ax.set_ylabel("NMSE")
ax.set_xticks(ticks=x, labels=selected, rotation=45)
plt.legend()
plt.grid("on")
plt.show()

```

0% | 0/30 [00:00<?, ?it/s]



7.4 Using other predictive models

We can try other models like SVM (SVR), Neural Networks (MLPRegressor), K-Nearest Neighbors (KNeighborsRegressor) and see if performance improves.

- Using SVR, show the error using all the features.

```
from sklearn.svm import SVR

CV_err_svm_single_model = []
for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    model = SVR()
    model.fit(X_tr, Y_tr)
    Y_hat_ts = model.predict(X_ts)
    CV_err_svm_single_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_svm_single_model):.5f}, std dev: {np.std(CV_err_svm_single_model):.5f}")
```

CV error: 0.30621, std dev: 0.03799

- Then, using cross-validation, use mRMR.

```
# Feature selection with mRMR for SVM
```

(continues on next page)

(continued from previous page)

```

n_variables = 10

CV_err_svm_single_model_fs = np.zeros((n_variables,10))
fold_id = 0

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    mutual_info_values = compute_mi_vector(X_tr, Y_tr)
    selected = []
    candidates = list(range(n))

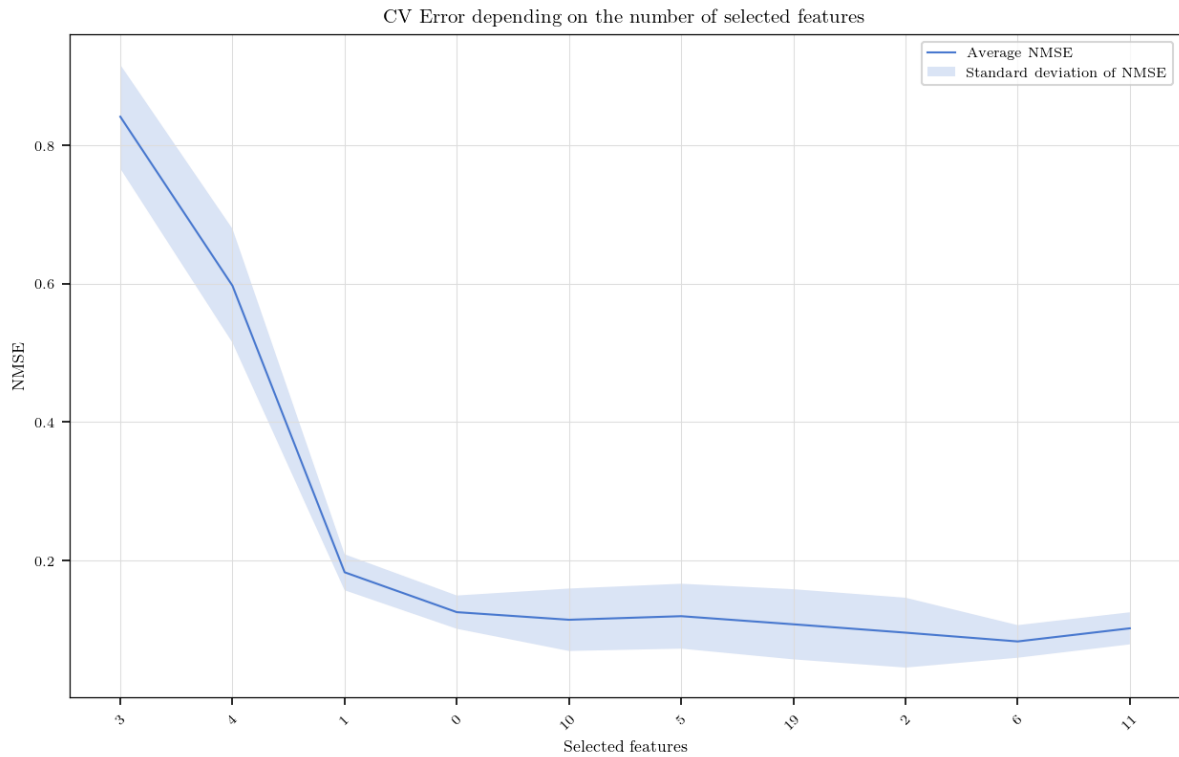
    for j in range(n_variables):
        redundancy_score = np.zeros(len(candidates))
        if len(selected)>0:
            for ci, cidx in enumerate(candidates):
                col_c = X_tr.iloc[:, cidx].values
                mis_c = []
                for sidx in selected:
                    col_s = X_tr.iloc[:, sidx].values
                    cc = np.corrcoef(col_s, col_c)[0,1]
                    if abs(cc)==1:
                        cc=0.999999
                    mis_c.append(-0.5*np.log(1-cc**2))
                redundancy_score[ci] = np.mean(mis_c)
            mRMR_score = mutual_info_values[candidates] - redundancy_score
            best_idx = candidates[np.argmax(mRMR_score)]
            selected.append(best_idx)
            candidates.remove(best_idx)

        # Evaluate performance with subsets of selected features
        for nb_features in range(1, n_variables+1):
            feats = [X.columns[i] for i in selected[:nb_features]]
            model = SVR()
            model.fit(X_tr[feats], Y_tr)
            Y_hat_ts = model.predict(X_ts[feats])
            CV_err_svm_single_model_fs[nb_features-1, fold_id] = nmse(Y_ts, Y_hat_ts)
        fold_id+=1

x = list(range(1, n_variables+1))
errors = np.array([np.mean(CV_err_svm_single_model_fs[i, :]) for i in range(n_
    variables)])
stds = np.array([np.std(CV_err_svm_single_model_fs[i, :]) for i in range(n_
    variables)])

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation_
    of NMSE")
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Selected features")
ax.set_ylabel("NMSE")
ax.set_xticks(ticks=x, labels=selected, rotation=45)
plt.legend()
plt.grid("on")
plt.show()

```



- Do the same for the two other models.

```
from sklearn.neural_network import MLPRegressor

CV_err_nnet_single_model = []

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    # Rescale input to 0-1
    min_X, max_X = np.min(X_tr), np.max(X_tr)
    X_tr = (X_tr-min_X)/(max_X-min_X)
    X_ts = (X_ts-min_X)/(max_X-min_X)

    # Rescale output to 0-1
    min_Y, max_Y = np.min(Y_tr), np.max(Y_tr)
    Y_tr_rescale = (Y_tr-min_Y)/(max_Y-min_Y)

    model = MLPRegressor(hidden_layer_sizes=(20,), max_iter=10000, learning_rate_
    ↪init=0.05, random_state=3)
    model.fit(X_tr, Y_tr_rescale)
    Y_hat_ts = (model.predict(X_ts)*(max_Y-min_Y))+min_Y
    CV_err_nnet_single_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_nnet_single_model):.5f}, std dev: {np.std(CV_err_
    ↪nnet_single_model):.5f}")
```

CV error: 0.74304, std dev: 0.11244

```

CV_err_nnet_single_model_fs = np.zeros((n_variables,10))
fold_id=0
for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    # Rescale input to 0-1
    min_X, max_X = np.min(X_tr), np.max(X_tr)
    X_tr = (X_tr-min_X)/(max_X-min_X)
    X_ts = (X_ts-min_X)/(max_X-min_X)

    # Rescale output to 0-1
    min_Y, max_Y = np.min(Y_tr), np.max(Y_tr)
    Y_tr_rescale = (Y_tr-min_Y)/(max_Y-min_Y)

    mutual_info_values = compute_mi_vector(X_tr, Y_tr)
    selected = []
    candidates = list(range(n))

    for j in range(n_variables):
        redundancy_score = np.zeros(len(candidates))
        if len(selected)>0:
            for ci, cidx in enumerate(candidates):
                col_c = X_tr.iloc[:, cidx].values
                mis_c = []
                for sidx in selected:
                    col_s = X_tr.iloc[:, sidx].values
                    cc = np.corrcoef(col_s, col_c)[0,1]
                    if abs(cc)==1:
                        cc=0.999999
                    mis_c.append(-0.5*np.log(1-cc**2))
                redundancy_score[ci] = np.mean(mis_c)
            mRMR_score = mutual_info_values[candidates] - redundancy_score
            best_idx = candidates[np.argmax(mRMR_score)]
            selected.append(best_idx)
            candidates.remove(best_idx)

        for nb_features in range(1, n_variables+1):
            feats = [X.columns[i] for i in selected[:nb_features]]
            Y_tr_rescale = Y_tr/10.0
            model = MLPRegressor(hidden_layer_sizes=(20,), max_iter=1000000, learning_
↪rate_init=0.005, random_state=3)
            model.fit(X_tr[feats], Y_tr_rescale)
            Y_hat_ts = model.predict(X_ts[feats])*10.0
            CV_err_nnet_single_model_fs[nb_features-1, fold_id] = nmse(Y_ts, Y_hat_ts)
            fold_id+=1

x = list(range(1, n_variables+1))
errors = np.array([np.mean(CV_err_nnet_single_model_fs[i, :]) for i in range(n_
↪variables)])
stds = np.array([np.std(CV_err_nnet_single_model_fs[i, :]) for i in range(n_
↪variables)])

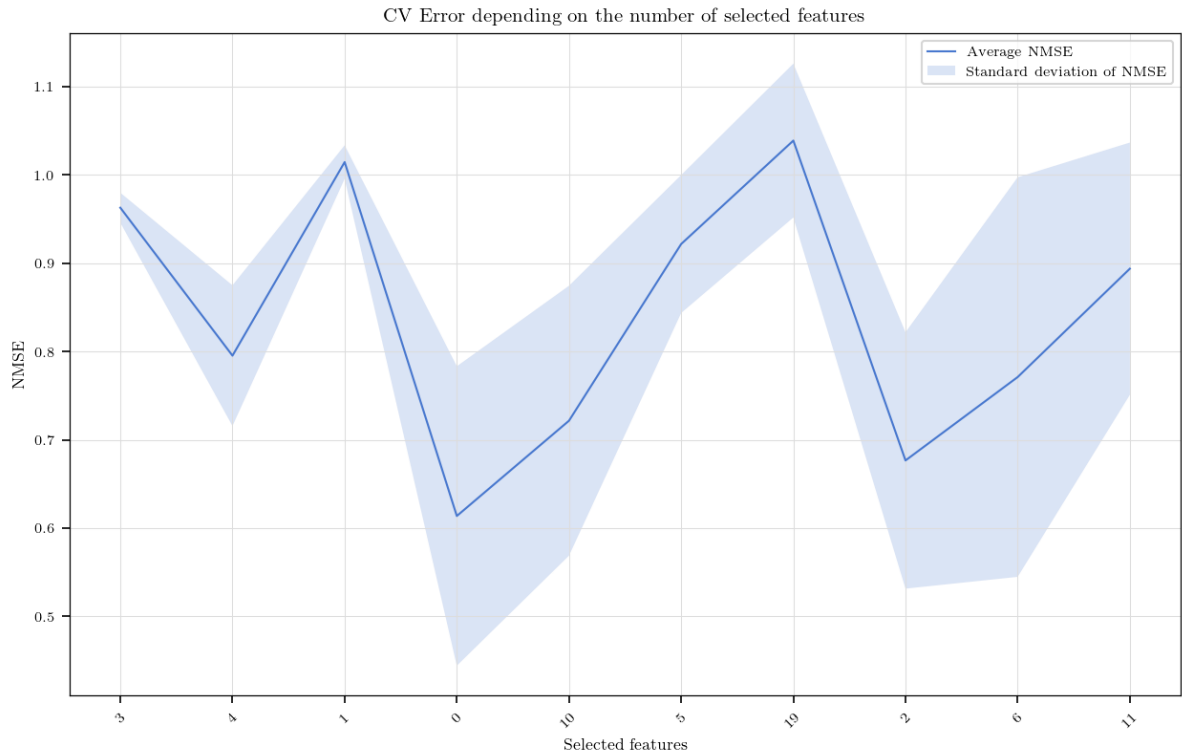
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation_
↪of NMSE")

```

(continues on next page)

(continued from previous page)

```
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Selected features")
ax.set_ylabel("NMSE")
ax.set_xticks(ticks=x, labels=selected, rotation=45)
plt.legend()
plt.grid("on")
plt.show()
```



```
from sklearn.neighbors import KNeighborsRegressor

CV_err_lazy_single_model = []
for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    model = KNeighborsRegressor(n_neighbors=5)
    model.fit(X_tr, Y_tr)
    Y_hat_ts = model.predict(X_ts)
    CV_err_lazy_single_model.append(nmse(Y_ts, Y_hat_ts))

print(f"CV error: {np.mean(CV_err_lazy_single_model):.5f}, std dev: {np.std(CV_err_
    lazy_single_model):.5f}")
```

CV error: 0.64745, std dev: 0.08826

```
# Feature selection for KNN using mRMR
CV_err_lazy_single_model_fs = np.zeros((n_variables,10))
fold_id=0
```

(continues on next page)

(continued from previous page)

```

for train_index, test_index in kf.split(X):
    X_tr, X_ts = X.iloc[train_index], X.iloc[test_index]
    Y_tr, Y_ts = Y[train_index], Y[test_index]

    mutual_info_values = compute_mi_vector(X_tr, Y_tr)
    selected = []
    candidates = list(range(n))

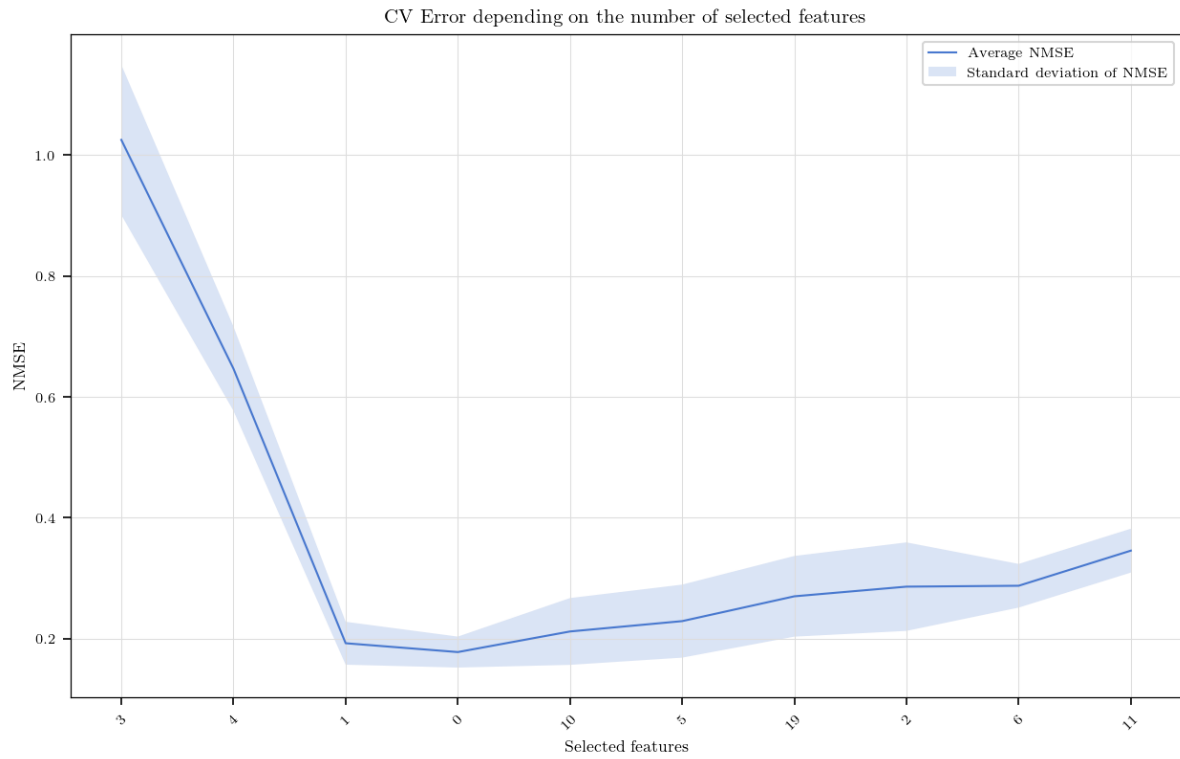
    for j in range(n_variables):
        redundancy_score = np.zeros(len(candidates))
        if len(selected)>0:
            for ci, cidx in enumerate(candidates):
                col_c = X_tr.iloc[:, cidx].values
                mis_c = []
                for sidx in selected:
                    col_s = X_tr.iloc[:, sidx].values
                    cc = np.corrcoef(col_s, col_c)[0,1]
                    if abs(cc)==1:
                        cc=0.999999
                    mis_c.append(-0.5*np.log(1-cc**2))
                redundancy_score[ci] = np.mean(mis_c)
            mRMR_score = mutual_info_values[candidates] - redundancy_score
            best_idx = candidates[np.argmax(mRMR_score)]
            selected.append(best_idx)
            candidates.remove(best_idx)

    for nb_features in range(1, n_variables+1):
        feats = [X.columns[i] for i in selected[:nb_features]]
        model = KNeighborsRegressor(n_neighbors=5)
        model.fit(X_tr[feats], Y_tr)
        Y_hat_ts = model.predict(X_ts[feats])
        CV_err_lazy_single_model_fs[nb_features-1, fold_id] = nmse(Y_ts, Y_hat_ts)
        fold_id+=1

x = list(range(1, n_variables+1))
errors = np.array([np.mean(CV_err_lazy_single_model_fs[i, :]) for i in range(n_
    variables)])
stds = np.array([np.std(CV_err_lazy_single_model_fs[i, :]) for i in range(n_
    variables)])

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, errors, label="Average NMSE")
ax.fill_between(x, errors - stds, errors + stds, alpha=0.2, label="Standard deviation_
    of NMSE")
ax.set_title("CV Error depending on the number of selected features")
ax.set_xlabel("Selected features")
ax.set_ylabel("NMSE")
ax.set_xticks(ticks=x, labels=selected, rotation=45)
plt.legend()
plt.grid("on")
plt.show()

```



CLASSIFICATION

Let us define the basic imports and a pprint function.

```
import numpy as np
import sklearn
import warnings
warnings.filterwarnings('ignore')
from tqdm.auto import tqdm
from IPython.display import display, Math
from numpyarray_to_latex.jupyter import to_ltx
import matplotlib.pyplot as plt
plt.style.use(['seaborn-v0_8-muted', 'practicals.mplstyle'])
def pprint(*args):
    res = ""
    for i in args:
        if type(i) == np.ndarray:
            res += to_ltx(i, brackets='[]')
        elif type(i) == str:
            res += i
    display(Math(res))
```

8.1 Introduction

In classification problem, the target is a category (a discrete variable) and its conditional distribution is

$$P(y|x)$$

where:

- $\mathbf{y} \in \{c_1, \dots, c_K\}$ represents the output variable (also called target)
- $x \in \mathbb{R}^n$ represents the vector of inputs (also called features)

In classification, the goal of learning is to return an estimator

$$h(x, \alpha)$$

of the conditional probability $P(\mathbf{y}|x)$, where α denotes the set of parameters of the model h .

The learning of the estimator is done on the basis of an available input/output training set D_N made of N observation pairs (x_i, y_i) where $x_i \in \mathbb{R}^n$.

8.2 Data generation

As seen in the theoretical course, there are multiple ways to generate datasets for binary classification problems.

8.2.1 Generative way:

First, generate the class $y_i \in \{0, 1\}$, $i \in \{1, \dots, N\}$, using the *a priori* probability, then generate the x values, given the **class conditional density** $p(x|y = y_i)$.

```
np.random.seed(1)
```

If we name the *a priori* probability $p_1 = P(y = 1)$, we can generate the Y dataset:

```
N = 2000
n = 2
p1 = 0.4

Y = np.random.choice([0, 1], size=N, p=[1-p1, p1])
```

Then, we can generate the X dataset, following two multivariate normal distribution (recall practical 1). We define two distributions, for each of the two classes. Then, depending on the y class of each item, we sample the corresponding multivariate normal distribution.

Here, to make sure that `cov0` and `cov1` are defined positive, we generate a random A matrix, then `cov` is computed as $A^T A$.

```
A0 = np.random.rand(n, n)
cov0 = A0.T @ A0
mu0 = (0, 0)

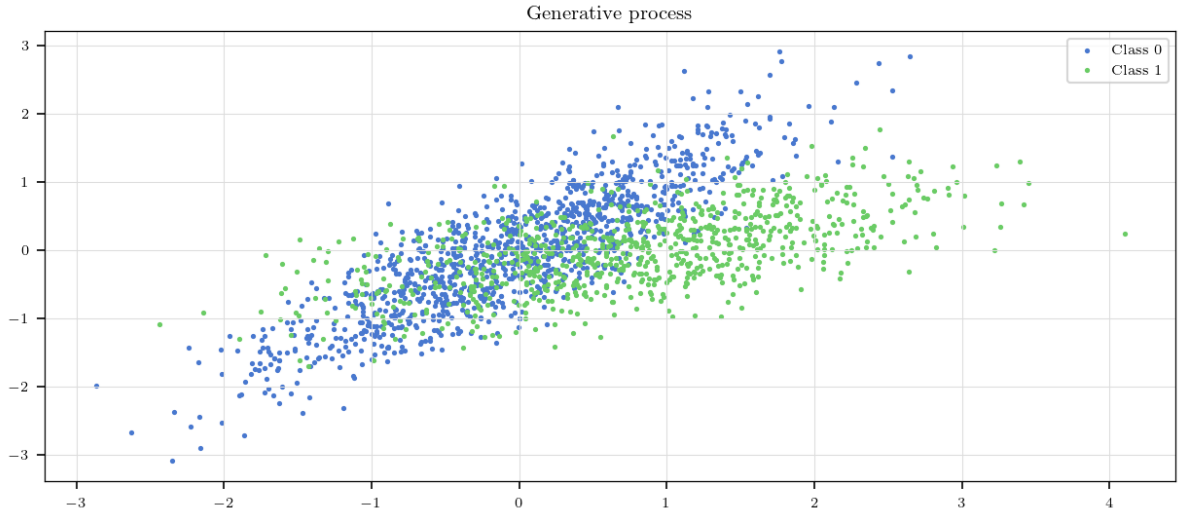
A1 = np.random.rand(n, n)
cov1 = A1.T @ A1
mu1 = (0.75, 0)
```

```
X = np.zeros((N, n))
for i in np.arange(N):
    if Y[i] == 1:
        X[i, :] = np.random.multivariate_normal(mu1, cov1, size=1)
    else:
        X[i, :] = np.random.multivariate_normal(mu0, cov0, size=1)
```

Then, we plot the generated data.

```
I0 = np.where(Y==0)
I1=np.where(Y==1)

plt.figure(figsize=(10, 4))
plt.scatter(X[I0, 0], X[I0, 1], marker='.', label="Class 0")
plt.scatter(X[I1, 0], X[I1, 1], marker='.', label="Class 1")
plt.grid("on")
plt.legend()
plt.title("Generative process")
plt.show()
```



8.2.2 Discriminative way

Otherwise, if we have the conditional distributions $P(y|x)$, we can first sample x , then determine the class y .

```
from scipy.stats import multivariate_normal
```

First, we sample x according to the probability density function:

$$p(x) = p_1 * p(x|y = 1) + (1 - p_1) * p(x|y = 0)$$

Those are mixture of gaussians (recall: practical 1).

```
X = np.zeros((N,n))
for i in np.arange(N):
    if np.random.uniform() < p1:
        X[i,:] = np.random.multivariate_normal(mu1, cov1, size=1)
    else:
        X[i,:] = np.random.multivariate_normal(mu0, cov0, size=1)
```

Then, using the Bayes theorem, we compute the y classes.

Recall the Bayes theorem

$$P(\mathbf{y} = y|x) = \frac{p_{\mathbf{x}}(x|\mathbf{y} = y)P(\mathbf{y} = y)}{p_{\mathbf{x}}(x)}$$

In order to compute the term $p_{\mathbf{x}}(x|\mathbf{y} = y)$, we need to make an hypothesis on the distribution of $\mathbf{x}$ according to $\mathbf{y} = y$. Here, since we know that the input features follow a gaussian law (if $\mathbf{y} = 0$, then $\mathbf{x} \sim \mathcal{N}(\mu_0, \Sigma_0)$ and if $\mathbf{y} = 1$, then $\mathbf{x} \sim \mathcal{N}(\mu_1, \Sigma_1)$), we can use the pdf function from the `multivariate_normal` library already presented.

In the binary case, we can rewrite the formula as

$$P(\mathbf{y} = 0|x) = \frac{p_{\mathbf{x}}(x|\mathbf{y} = 0)P(\mathbf{y} = 0)}{p_{\mathbf{x}}(x|\mathbf{y} = 0)P(\mathbf{y} = 0) + p_{\mathbf{x}}(x|\mathbf{y} = 1)P(\mathbf{y} = 1)}$$

The previous formula is directly implemented by:

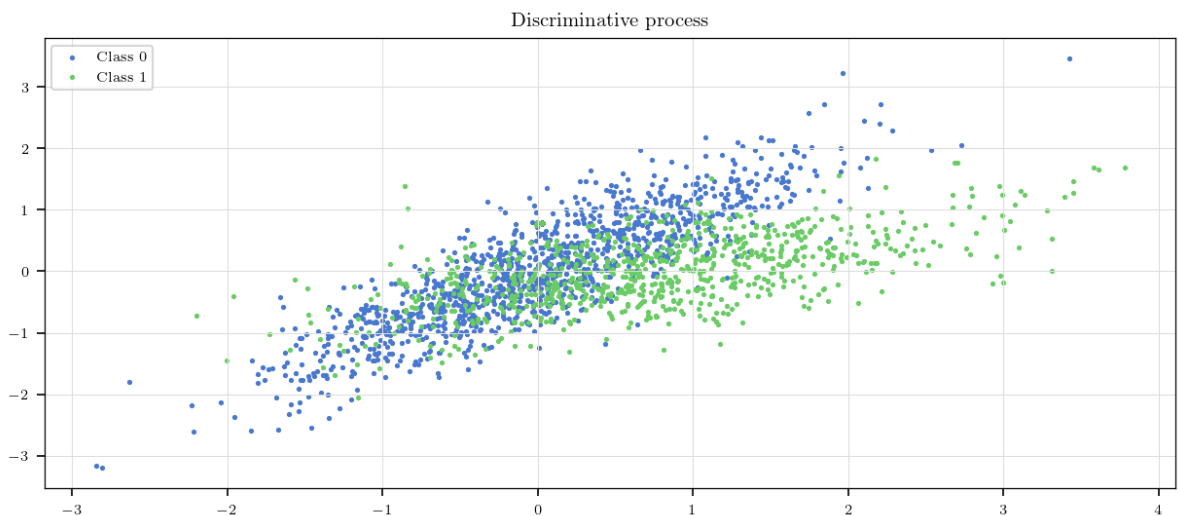
```
def condprob(x, cov1, mu1, cov0, mu0, p1):
    return multivariate_normal.pdf(x, mean=mu1, cov=cov1)*p1/(multivariate_normal.
    pdf(x, mean=mu1, cov=cov1)*p1+multivariate_normal.pdf(x, mean=mu0, cov=cov0)*(1-p1))
```

where $P(y = y|x)$ is computed by `condprob`.

```
Y = np.zeros((N, 1))
for i in np.arange(N):
    p1i = condprob(X[i, :], cov1, mu1, cov0, mu0, p1)
    Y[i] = np.random.choice([0, 1], size=1, p=[1-p1i, p1i])
```

```
I0 = np.where(Y==0)
I1 = np.where(Y==1)

plt.figure(figsize=(10, 4))
plt.scatter(X[I0, 0], X[I0, 1], marker='.', label="Class 0")
plt.scatter(X[I1, 0], X[I1, 1], marker='.', label="Class 1")
plt.grid("on")
plt.legend()
plt.title("Discriminative process")
plt.show()
```



Of course, the computed datasets follow the same distribution, no matter the method used.

8.3 Exercises

- For the lastly defined distributions, given three points $q = \{(0, 0), (3, 0), (0, 2)\}$, compute $P(y|x = q)$ for each class y . Compute $p(q|y = y)$ for each class y and $P(y = y)$ for each class y .
- Can you adapt the two procedures to get higher-dimensional data (i.e. x is not in $\mathbb{R}^2$ but in $\mathbb{R}^n$) ?
- Can you adapt the two procedures to get more-classes data (i.e. y is not in $\{0, 1\}$ but in $\{1, \dots, c_k\}$) ?

8.3.1 Question 1

Using Bayes' theorem:

$$P(\mathbf{y} = y | \mathbf{x} = \mathbf{q}) = \frac{p(\mathbf{q} | \mathbf{y} = y) P(\mathbf{y} = y)}{p(\mathbf{q})}$$

where the marginal likelihood $p(\mathbf{q})$ is given by:

$$p(\mathbf{q}) = \sum_y p(\mathbf{q} | \mathbf{y} = y) P(\mathbf{y} = y).$$

The likelihood $p(\mathbf{q} | \mathbf{y} = y)$ follows the multivariate normal density function (available in `scipy`):

$$p(\mathbf{q} | \mathbf{y} = y) = \frac{1}{(2\pi)^{d/2} |\Sigma_y|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{q} - \mu_y)^T \Sigma_y^{-1} (\mathbf{q} - \mu_y) \right),$$

where $d = 2$ is the dimensionality.

```
for q in [(0, 0), (3, 0), (0, 2)]:
    pq = multivariate_normal.pdf(q, mean=mu0, cov=cov0)*(1-p1) + multivariate_normal.
    pdf(q, mean=mu1, cov=cov1)*p1
    pprint(f"P(y=0|x={q}) = {multivariate_normal.pdf(q, mean=mu0, cov=cov0)*(1-p1) /
    pq:.4e}")

print()
for q in [(0, 0), (3, 0), (0, 2)]:
    pq = multivariate_normal.pdf(q, mean=mu0, cov=cov0)*(1-p1) + multivariate_normal.
    pdf(q, mean=mu1, cov=cov1)*p1
    pprint(f"P(y=1|x={q}) = {multivariate_normal.pdf(q, mean=mu1, cov=cov1)*p1/pq:.4e}
    ")

print()
for q in [(0, 0), (3, 0), (0, 2)]:
    pprint(f"P(x={q}|y=0) = {multivariate_normal.pdf(q, mean=mu0, cov=cov0):.4e}")

print()
for q in [(0, 0), (3, 0), (0, 2)]:
    pprint(f"P(x={q}|y=1) = {multivariate_normal.pdf(q, mean=mu1, cov=cov1):.4e}")

print()
pprint(f"P(y=0) = {1-p1}")
pprint(f"P(y=1) = {p1}")
```

$$P(y = 0 | x = (0, 0)) = 7.3086e - 01$$

$$P(y = 0 | x = (3, 0)) = 1.3140e - 09$$

$$P(y = 0 | x = (0, 2)) = 9.7560e - 01$$

$$P(y = 1 | x = (0, 0)) = 2.6914e - 01$$

$$P(y = 1 | x = (3, 0)) = 1.0000e + 00$$

$$P(y = 1 | x = (0, 2)) = 2.4404e - 02$$

$$P(x = (0, 0) | y = 0) = 4.1069e - 01$$

$$P(x = (3, 0) | y = 0) = 6.5642e - 12$$

$$P(x = (0, 2) | y = 0) = 3.5164e - 05$$

$$P(x = (0, 0) | y = 1) = 2.2685e - 01$$

$$P(x = (3, 0) | y = 1) = 7.4934e - 03$$

$$P(x = (0, 2) | y = 1) = 1.3194e - 06$$

$$P(y = 0) = 0.6$$

$$P(y = 1) = 0.4$$

8.3.2 Question 2

```
n = 5
Y = np.random.choice([0, 1], size=N, p=[1-p1, p1])

mu0 = np.zeros(n)
A0 = np.random.rand(n, n)
cov0 = A0.T @ A0

mu1 = np.array([0.75] + [0] * (n - 1))
A1 = np.random.rand(n, n)
cov1 = A1.T @ A1

X = np.zeros((N, n))
for i in range(N):
    if Y[i] == 1:
        X[i, :] = np.random.multivariate_normal(mu1, cov1, size=1)
    else:
        X[i, :] = np.random.multivariate_normal(mu0, cov0, size=1)
```

8.3.3 Question 3

```
np.random.seed(1)
n = 2
c_k = 4

class_probs = [0.2, 0.3, 0.35, 0.15]
Y = np.random.choice(np.arange(c_k), size=N, p=class_probs)

mus = [np.random.uniform(-1, 1, size=n) for _ in range(c_k)]
```

(continues on next page)

(continued from previous page)

```

covs = [np.random.rand(n, n) for _ in range(c_k)]
covs = [C.T @ C for C in covs]

X = np.zeros((N, n))
for i in range(N):
    cls = Y[i]
    X[i, :] = np.random.multivariate_normal(mus[cls], covs[cls], size=1)

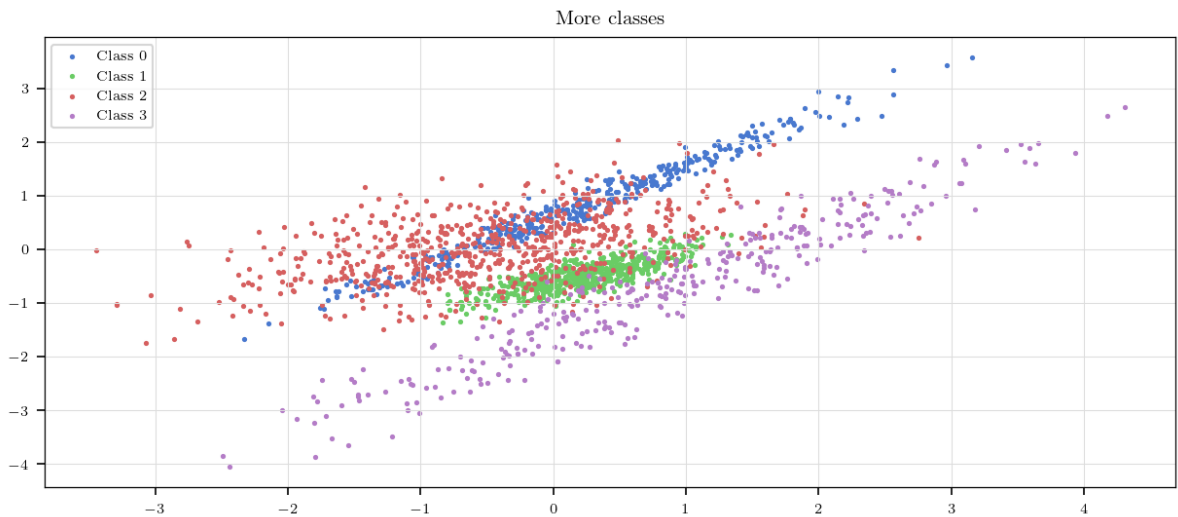
```

```

Is = [np.where(Y==i) for i in range(c_k)]

plt.figure(figsize=(10, 4))
for i in range(c_k):
    plt.scatter(X[Is[i], 0], X[Is[i], 1], marker='.', label=f"Class {i}")
plt.legend()
plt.grid("on")
plt.title("More classes")
plt.show()

```



Finally we regenerate binary data for the rest of the practical:

```

np.random.seed(1)
N = 4000
N_test = 1000
n = 2
p1 = 0.4

Y = np.random.choice([0, 1], size=N, p=[1-p1, p1])
A0 = np.array([[.45, .5], [.05, .35]])
cov0 = A0.T @ A0
mu0 = (0, 0)

A1 = np.array([[.32, -.4], [-.4, .32]])
cov1 = A1.T @ A1
mu1 = (.5, 0)
X = np.zeros((N, n))
for i in np.arange(N):
    if Y[i] == 1:

```

(continues on next page)

(continued from previous page)

```

X[i, :] = np.random.multivariate_normal(mu1, cov1, size=1)
else:
    X[i, :] = np.random.multivariate_normal(mu0, cov0, size=1)

Y_test = np.random.choice([0, 1], size=N_test, p=[1-p1, p1])
X_test = np.zeros((N_test, n))
for i in np.arange(N_test):
    if Y_test[i] == 1:
        X_test[i, :] = np.random.multivariate_normal(mu1, cov1, size=1)
    else:
        X_test[i, :] = np.random.multivariate_normal(mu0, cov0, size=1)

```

8.4 Metrics and model evaluations methods

In the previous practicals, when we were dealing with regression tasks, we defined different metrics for evaluating the quality of some model's predictions. Remind: we defined the Mean Squared Error, then, we normalized it to define the Normalized Mean Squared Error. We spoke about other metrics, such as the MAE. All those quantities, while highly informative when applied to continuous quantities, are not as useful when dealing with classes.

Let us consider a practical example. You are assigned the task to determine if some rocket components are working (1) or not (0). In this task, it seems clear that the fact of assigning *working* to a *non-working* piece is worse than assigning *non-working* to a *working* piece (you could lose astronauts). However, this is not reflected by the MSE: in both cases, assigning the wrong label would lead to an error of 1.

In binary classification, however, we can determine entirely the cases possible after prediction:

- Expected: 0, Predicted: 0 -> True Negative (TN)
- Expected: 1, Predicted: 1 -> True Positive (TP)
- Expected: 0, Predicted: 1 -> False Positive (FP)
- Expected: 1, Predicted: 0 -> False Negative (FN)

In this terminology, it is clear that the previous example suffers more from the False Positives than the False Negatives.

We can present those quantities in a matrix, called the *confusion matrix*:

	Real Class: Negative	Real Class: Positive
Classified as negative	TN	FN
Classified as positive	FP	TP

In `sklearn`, the function `confusion_matrix` builds it from the set of expected and predicted labels. That is, if a model outputs a score (which is the case for most of them), for a given threshold on those scores the classes are determined, then the confusion matrix is built on top of those predictions. There exists a convenient `ConfusionMatrixDisplay` class, but take care, the cases are inverted when compared to the course notation. A confusion matrix, however, is not so-to-say a scoring method. There are different values, derived from the confusion matrix quantities, that allows to score a given prediction set:

- The **precision**: $\frac{TP}{TP+FP}$
- The **recall**: $\frac{TP}{TP+FN}$
- The **F1 score**: $\frac{2TP}{2TP+FP+FN}$ which is the harmonic mean between the precision and the recall scores

- More generally, the **F Beta score**: $\frac{(1+\beta^2)TP}{(1+\beta^2)TP+FP+\beta^2FN}$
- The **accuracy**: $\frac{TP+TN}{TP+TN+FP+FN}$

However, those quantities often do not take into account the number of True Negatives. Note: most of those scoring methods are implemented in `sklearn`.

Let us define furthermore the *True Positive Rate*, i.e., the proportion of True Positive on all the indeed positive samples, and, conversely, the *False Positive Rate*, the proportion of False Positive on all the in fact positive samples.

8.5 Bayes classifier

Remind: for discrete inputs, the Bayes theorem states that:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

and this can be used as a classifier. More specifically, for continuous inputs, we have

$$P(\mathbf{y} = c_k | \mathbf{x} = x) = \frac{p_x(x | \mathbf{y} = c_k) P(\mathbf{y} = c_k)}{\sum_{k=1}^K p_x(x | \mathbf{y} = c_k) P(\mathbf{y} = c_k)}$$

Note, it requires to know what is the density function p_x . If we make the hypothesis that those follow a normal distribution (how could we test if this is valid ?). However, in practice, the μ and cov should be estimated from the data., we can use the `GaussianNB` class from `sklearn`.

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score

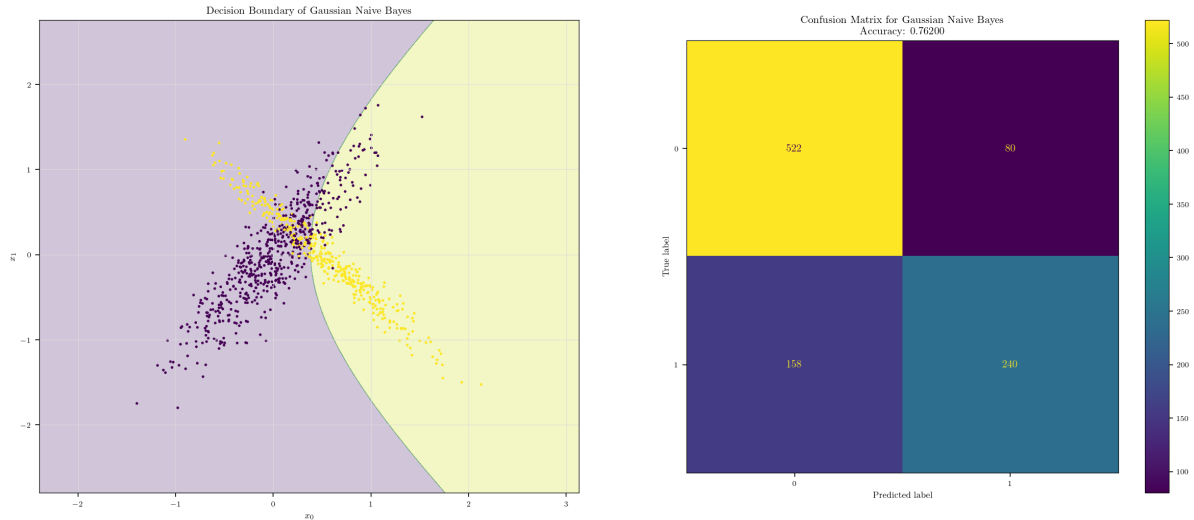
def plot_field(X, Y, model, name=""):
    x0_min, x0_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    x1_min, x1_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx0, xx1 = np.meshgrid(np.linspace(x0_min, x0_max, 1000),
                           np.linspace(x1_min, x1_max, 1000))

    Z = model.predict(np.c_[xx0.ravel(), xx1.ravel()]).reshape(xx0.shape)
    Y_hat = model.predict(X)
    fig, ax = plt.subplots(1, 2, figsize=(20, 8))
    ax[0].contourf(xx0, xx1, Z, alpha=0.25)
    ax[0].scatter(X[:, 0], X[:, 1], c=Y, marker='.')
    ax[0].grid("on")

    ax[0].set_xlabel("$x_0$")
    ax[0].set_ylabel("$x_1$")
    ax[0].set_title(f"Decision Boundary of {name}")
    ConfusionMatrixDisplay.from_predictions(Y, Y_hat, ax=ax[1])
    ax[1].set_title(f"Confusion Matrix for {name}\nAccuracy: {accuracy_score(Y, Y_hat):.5f}")
    plt.show()

model = GaussianNB().fit(X, Y.ravel())

plot_field(X_test, Y_test, model, "Gaussian Naive Bayes")
```



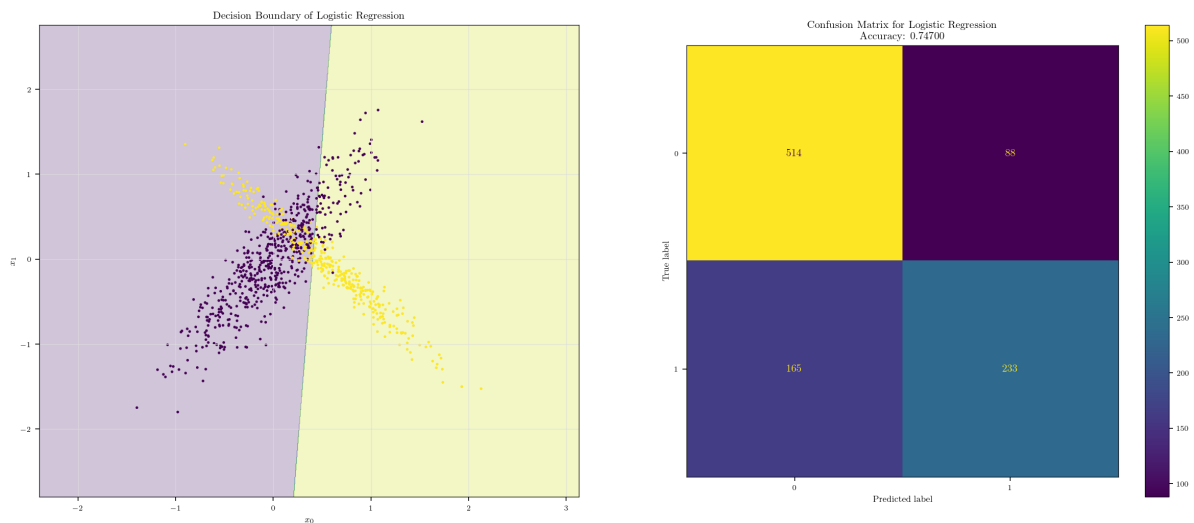
8.6 Linear model for classification

As it has been done for the regression tasks, we can use a linear model for classifying the previously generated points. As an example, in sklearn, we can use the `LogisticRegression` class:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression().fit(X, Y.ravel())

plot_field(X_test, Y_test, model, "Logistic Regression")
```



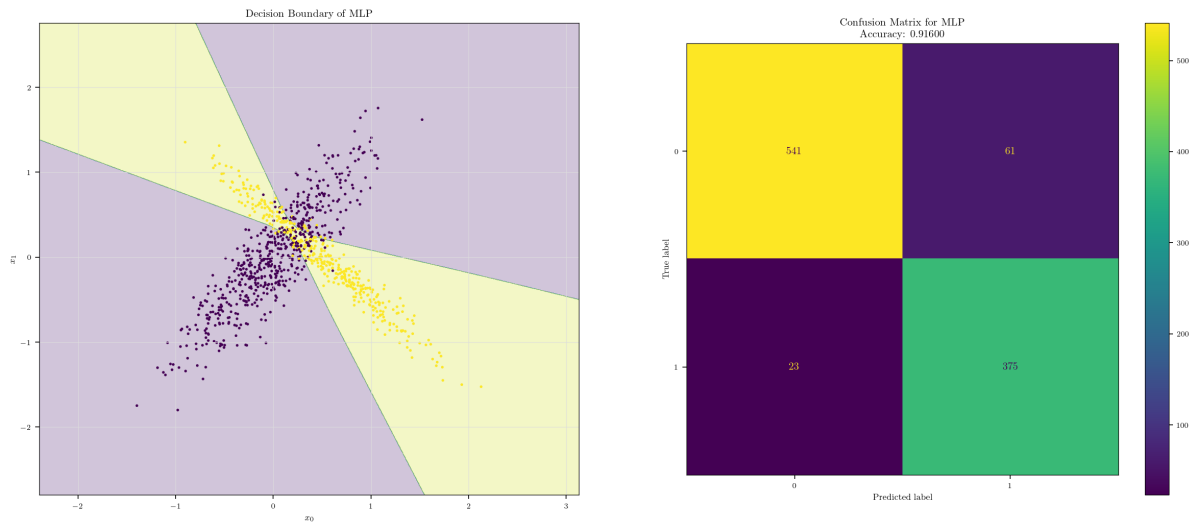
8.6.1 Multi-layers perceptron

Now, for fun, we can train another model to see how the space is then divided:

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(max_iter=1000).fit(X, Y.ravel())

plot_field(X_test, Y_test, model, "MLP")
```



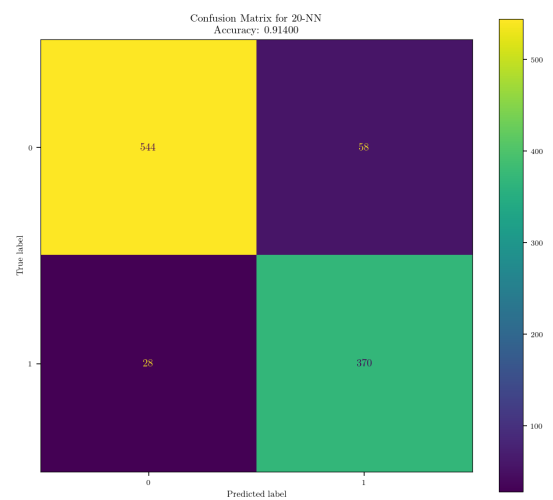
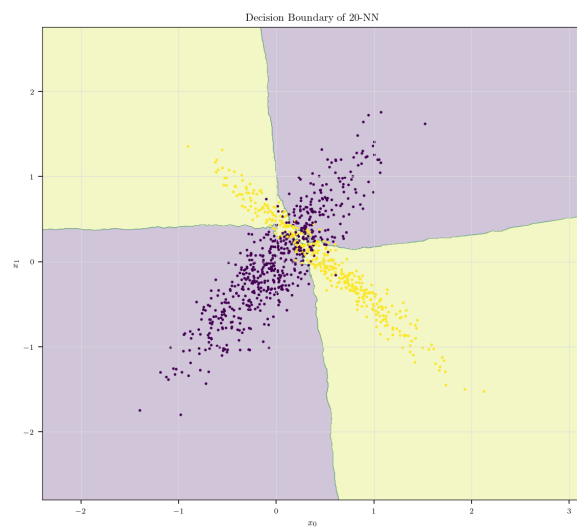
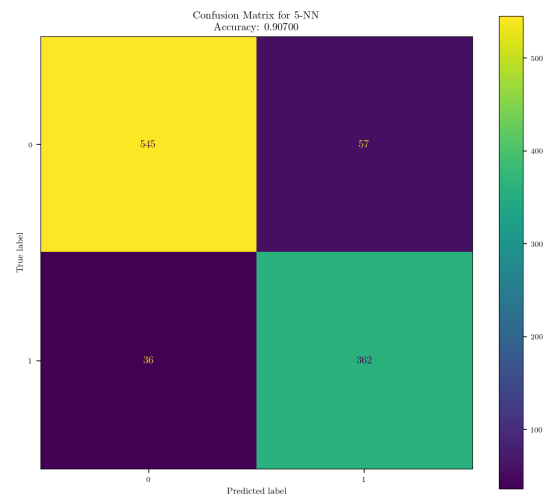
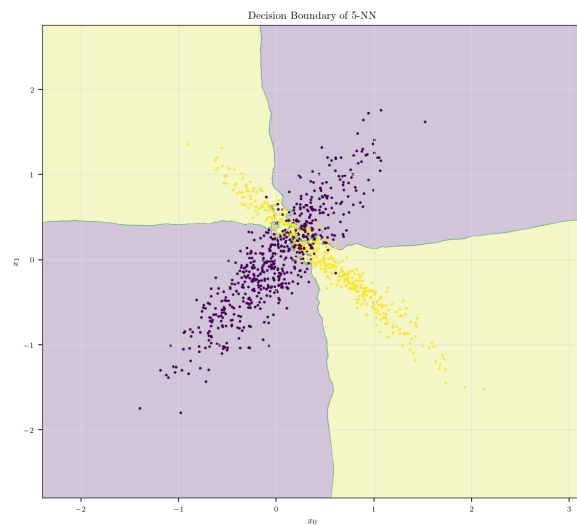
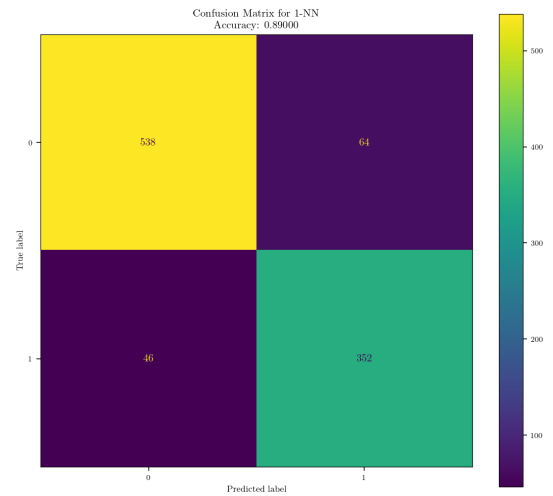
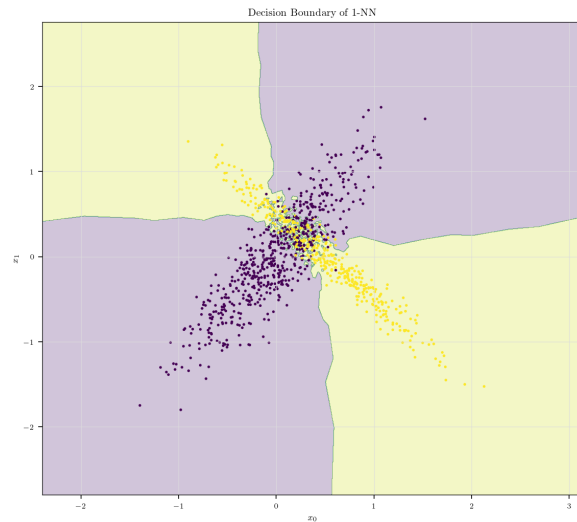
8.7 Exercises

8.7.1 KNN

- Implement a KNN classifier (`sklearn.neighbors.KNeighborsClassifier`).
- Vary k (e.g., 1, 5, 20) and visualize decision boundaries.
- Discuss underfitting/overfitting trade-offs.

```
from sklearn.neighbors import KNeighborsClassifier

for k in (1, 5, 20):
    model = KNeighborsClassifier(k).fit(X, Y.ravel())
    plot_field(X_test, Y_test, model, f"{k}-NN")
```



8.7.2 Bayes

- Compute the true Bayes decision boundary using the known densities $p(x|y)$.
- Compare it to the decision boundaries of logistic regression and KNN.
- Discuss the optimality of Bayes classification.

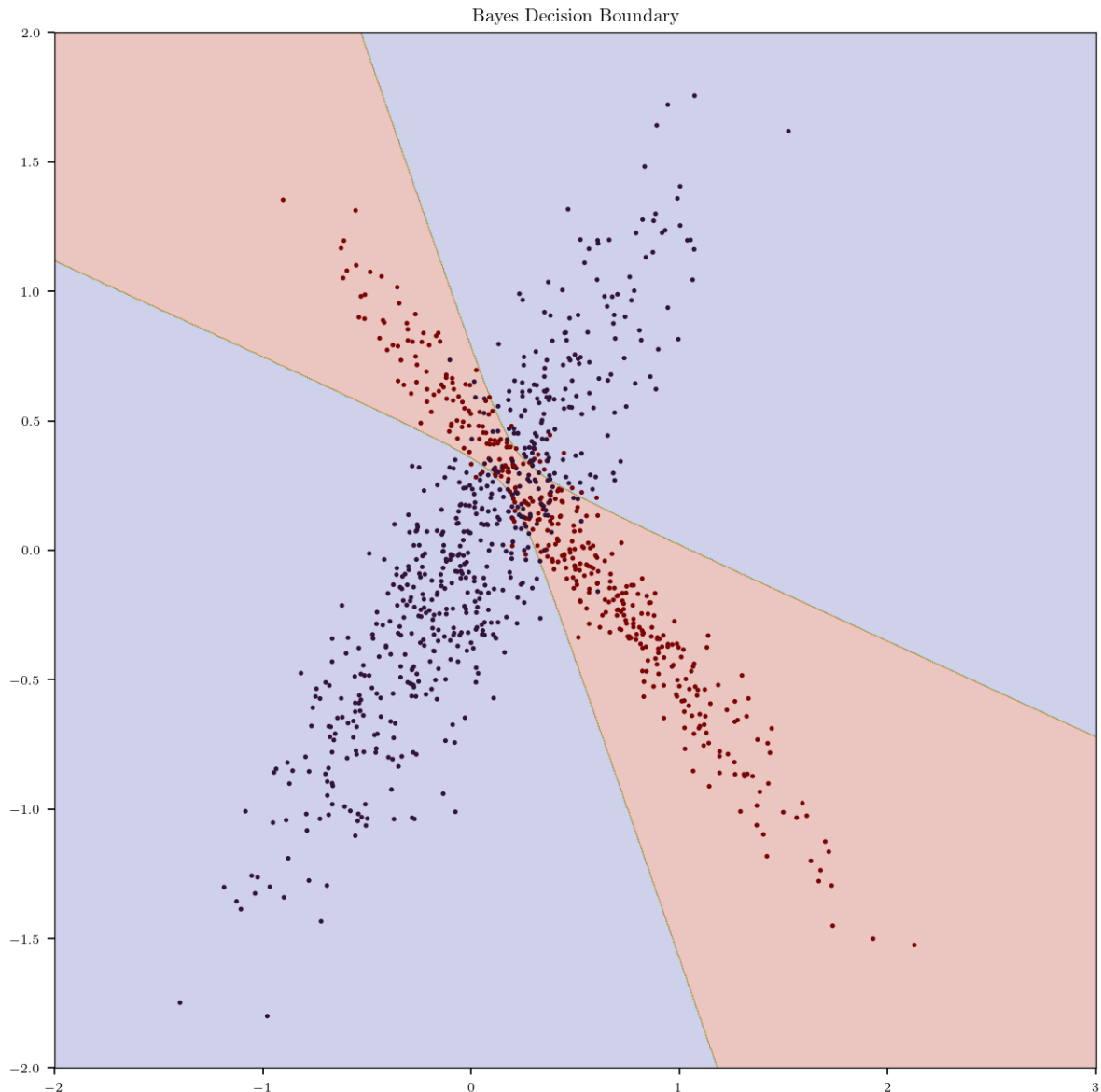
```
p_x_given_y0 = multivariate_normal(mean=mu0, cov=cov0)
p_x_given_y1 = multivariate_normal(mean=mu1, cov=cov1)

x1, x2 = np.meshgrid(np.linspace(-2, 3, 1000), np.linspace(-2, 2, 1000))
grid = np.c_[x1.ravel(), x2.ravel()]

posterior_1 = p1 * p_x_given_y1.pdf(grid)
posterior_0 = (1 - p1) * p_x_given_y0.pdf(grid)

bayes_boundary = (posterior_1 > posterior_0).reshape(x1.shape)

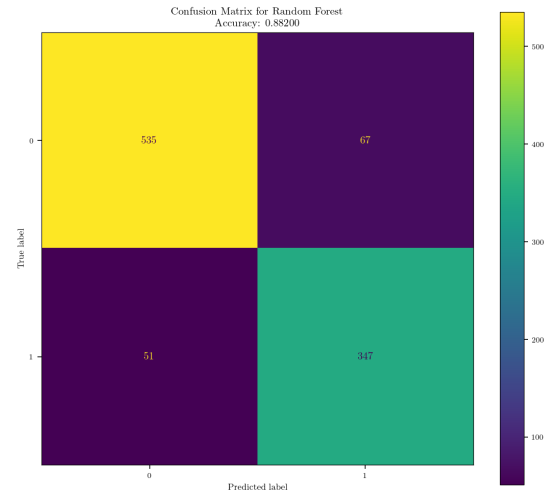
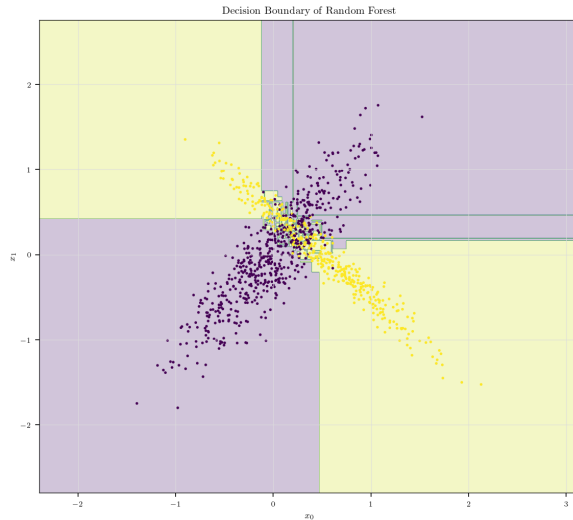
plt.figure(figsize=(10, 10))
plt.contourf(x1, x2, bayes_boundary/2, alpha=0.25, cmap="turbo")
plt.scatter(X_test[:, 0], X_test[:, 1], c=Y_test, marker='.', cmap="turbo")
plt.title("Bayes Decision Boundary")
plt.show()
```



8.7.3 Other models

- Use other models to classify tests' X and Y and show their decision boundaries.
- Compare their performances using the different metrics.

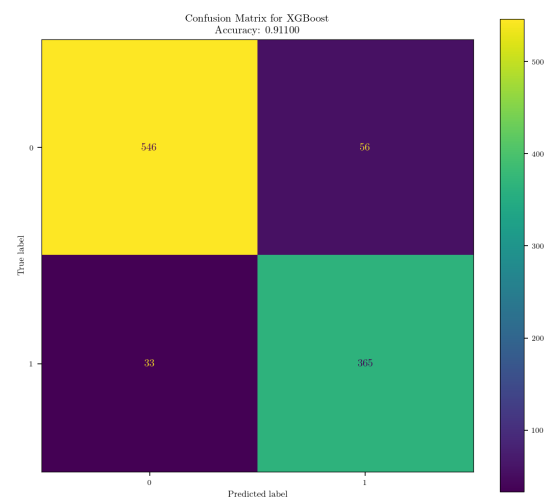
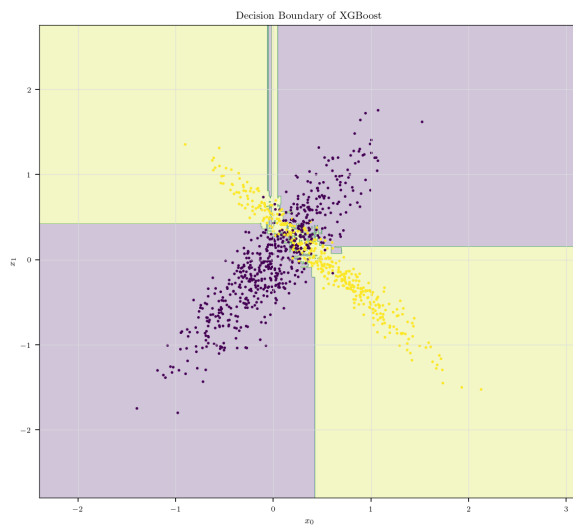
```
from sklearn.tree import DecisionTreeClassifier  
  
model = DecisionTreeClassifier().fit(X, Y.ravel())  
  
plot_field(X_test, Y_test, model, "Random Forest")
```



```
from xgboost import XGBClassifier

model = XGBClassifier().fit(X, Y.ravel())

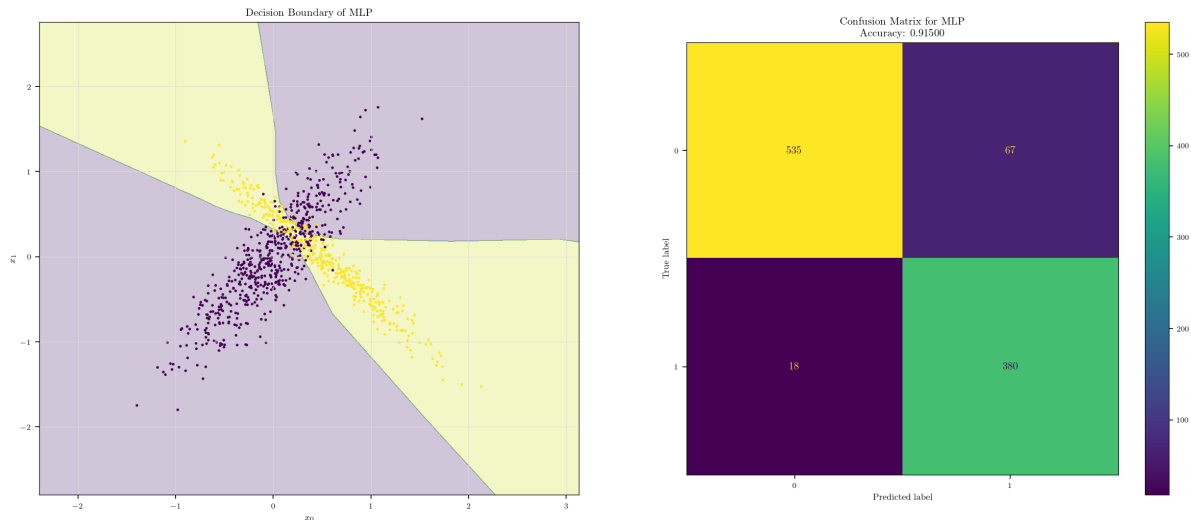
plot_field(X_test, Y_test, model, "XGBoost")
```



```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(8, 16, 8), max_iter=1000, random_state=1).
    fit(X, Y.ravel())

plot_field(X_test, Y_test, model, "MLP")
```



8.7.4 More classes

- Generate a dataset with two features x_1, x_2 , with classes in $\{0, 1, 2, 3\}$. The probability for each class is $[0.2, 0.3, 0.35, 0.15]$.
- Use a Logistic Regression model to classify the points.
- Use a RandomForestClassifier and a MLPClassifier to classify the points.
- For the three models, show the confusion matrix.
- Using the same visualization function as before, show the decision boundaries of the three models: how are those boundaries related to the way those models work?

```
np.random.seed(1)
n = 2
c_k = 4

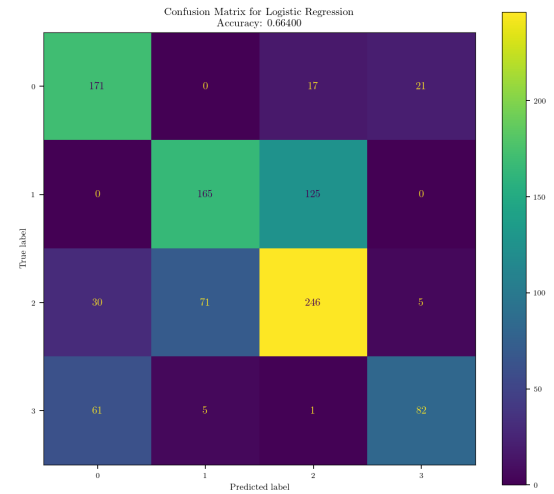
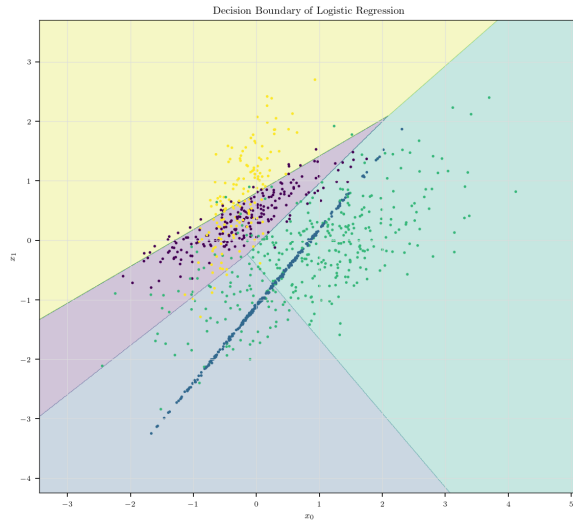
class_probs = [0.2, 0.3, 0.35, 0.15]
Y4 = np.random.choice(np.arange(c_k), size=N, p=class_probs)

mus = [np.random.uniform(-1, 1, size=n) for _ in range(c_k)]
covs = [np.random.rand(n, n) for _ in range(c_k)]
covs = [C.T @ C for C in covs]

X4 = np.zeros((N, n))
for i in range(N):
    cls = Y4[i]
    X4[i, :] = np.random.multivariate_normal(mus[cls], covs[cls], size=1)

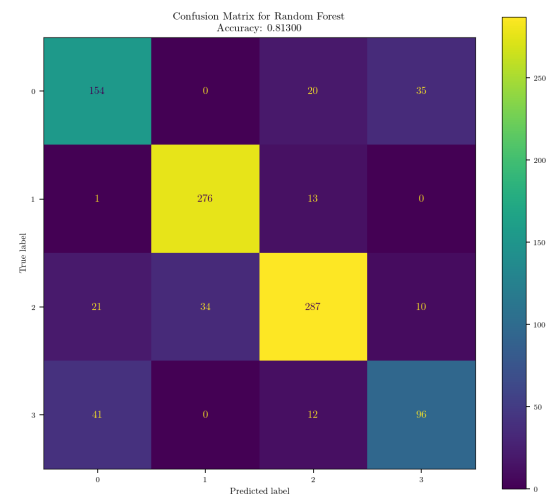
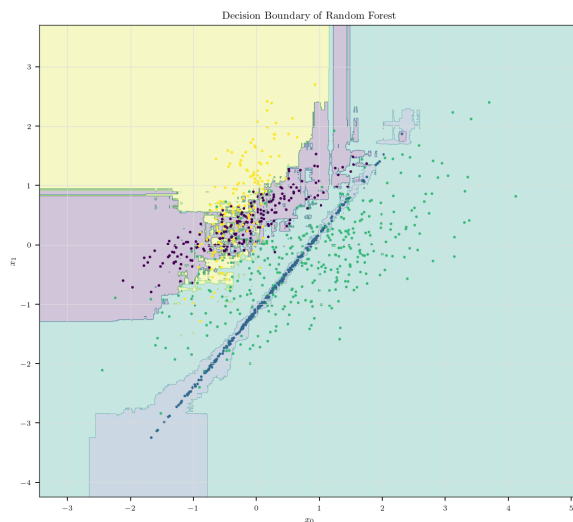
Y4_test = np.random.choice(np.arange(c_k), size=N_test, p=class_probs)
X4_test = np.zeros((N_test, n))
for i in range(N_test):
    cls = Y4_test[i]
    X4_test[i, :] = np.random.multivariate_normal(mus[cls], covs[cls], size=1)

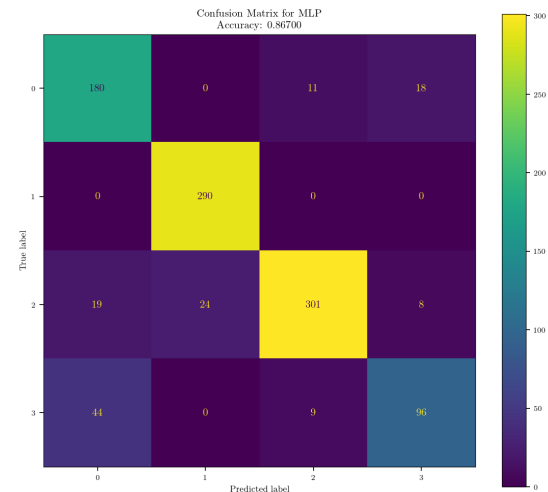
model = LogisticRegression().fit(X4, Y4.ravel())
plot_field(X4_test, Y4_test, model, "Logistic Regression")
```



```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier().fit(X4, Y4.ravel())
plot_field(X4_test, Y4_test, model, "Random Forest")

model = MLPClassifier(max_iter=1000).fit(X4, Y4.ravel())
plot_field(X4_test, Y4_test, model, "MLP")
```





8.8 ROC curve

Let us define two important notions:

- The **sensitivity**: $\frac{TP}{TP+FN}$. On all the indeed positive cases, what is the portion of correctly assigned positive?
- The **specificity**: $\frac{TN}{TN+FP}$. On all the indeed negative cases, what is the portion of correctly assigned negative?

Now, let us use a threshold model. That is, for a given threshold, if the sample is below, classify it as 0, otherwise classify it as 1. For multiple values of this threshold, the **ROC curve** (for *receiver operating characteristic*) is a plot in which the False Positive Rate is put in abscissa, and the sensibility in ordinate. The **precision/recall** plot displays the precision vs. the recall. Finally, the **lift** curve shows the proportion of *assigned positives* among all the points (the *alert level*).

We can define the notion of *area under the ROC curve* (AUROC, often called AUC). In the best case, the area under the ROC curve is 1.

8.8.1 Exercise

- For the previous dataset, implement (by hand):
 - A KNN Classifier
 - A Naïve Bayes Classifier
- where both outputs a score
- Then, for multiple threshold value, compute the True Positive rate and the False Positive rate of both models
- Plot the ROC curve of both models.
- Compute the AUROC for both models. How can you use this as a modelquality assessment ?

```
from scipy.stats import norm

def KNN(X, Y, q, k=15):
    N_local = X.shape[0]
    d = np.sqrt(np.sum((X - q) ** 2, axis=1))
    index = np.argsort(d)[:k]
```

(continues on next page)

(continued from previous page)

```

    return np.count_nonzero(Y[index] == 1) / k

def NB(X, Y, q):
    # Recall: we make the hypothesis that the the inputs are conditionnally_
    ↪independent
    N = len(Y)
    n = X.shape[1]
    Yhat = np.zeros(N)
    I1 = np.where(Y == 1)[0]
    I0 = np.where(Y == 0)[0]
    p1 = len(I1) / N
    p0 = 1 - p1
    p1x = 1
    p0x = 1
    for j in range(n):
        mean_I1 = np.mean(X[I1, j])
        sd_I1 = np.std(X[I1, j], ddof=1)
        mean_I0 = np.mean(X[I0, j])
        sd_I0 = np.std(X[I0, j], ddof=1)
        p1x = p1x * norm.pdf(q[j], loc=mean_I1, scale=sd_I1)
        p0x = p0x * norm.pdf(q[j], loc=mean_I0, scale=sd_I0)
    Yhat = p1x * p1 / (p1x * p1 + p0x * p0)
    return Yhat

```

```

plt.figure(figsize=(10, 4))
for model in [KNN, NB]:
    Y_hats_prob = []
    # Leave-one-out procedure
    for i in range(X.shape[0]):
        X_train = np.delete(X, i, axis=0)
        Y_train = np.delete(Y, i, axis=0)
        Y_hats_prob.append(model(X_train, Y_train, X[i]))

    Y_hats_prob = np.array(Y_hats_prob)

    thresholds = np.linspace(0, 1, 100)
    TPR = []
    FPR = []
    for t in thresholds:
        Y_hats = np.ones_like(Y)
        Y_hats[Y_hats_prob < t] = 0

        TP = np.sum((Y_hats == 1) & (Y == 1))
        TN = np.sum((Y_hats == 0) & (Y == 0))
        FP = np.sum((Y_hats == 1) & (Y == 0))
        FN = np.sum((Y_hats == 0) & (Y == 1))

        TPR.append(TP / (TP + FN) if (TP + FN) > 0 else 0)
        FPR.append(FP / (FP + TN) if (FP + TN) > 0 else 0)
    AUROC = 0
    for i in range(len(TPR)-1):
        AUROC += TPR[i+1]*(-FPR[i+1]+FPR[i])
    print(AUROC)
    plt.scatter(FPR, TPR)

plt.plot([0, 1], [0, 1], linestyle="dashed", color="gray")

```

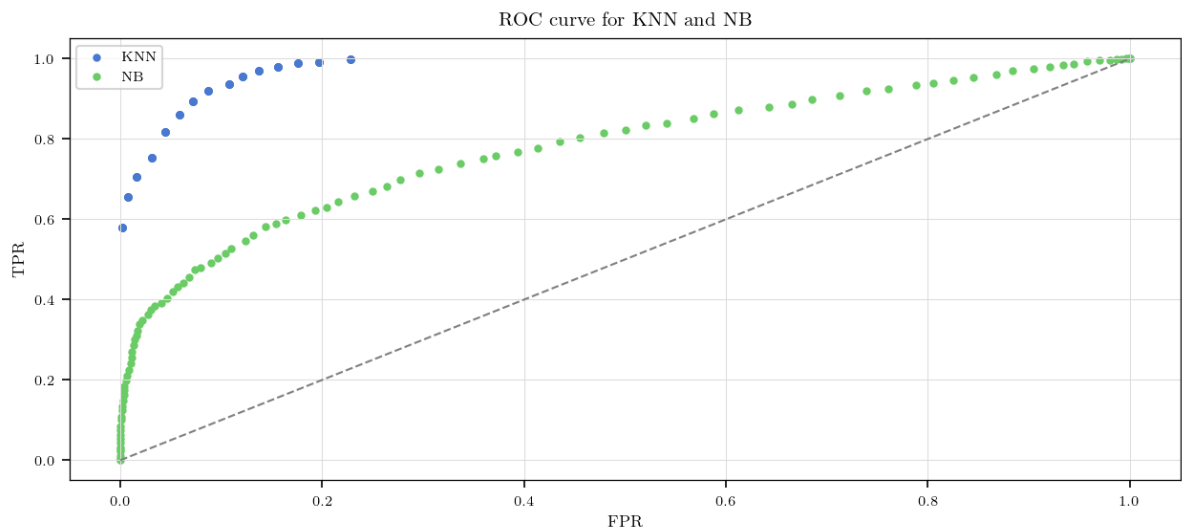
(continues on next page)

(continued from previous page)

```
plt.legend(["KNN", "NB"])
plt.xlabel("FPR")
plt.ylabel("TPR")
plt.title("ROC curve for KNN and NB")
plt.grid("on")
plt.show()
```

```
0.972294562756058
```

```
0.7735446279338773
```



For recall, those are a few models for classification that output a score. Try to display their ROC curve as well. See slides 28-33 from the theoretical course for more information.

8.8.2 More models

First, this model compute the mean of the two classes clusters, and then computes the score as a function of the distance between the query point and those two means.

```
def MD(X, Y, q):
    N_local = len(Y)
    n = X.shape[1]
    I1_local = np.where(Y == 1)[0]
    I0_local = np.where(Y == 0)[0]
    mu1 = np.mean(X[I1_local, :], axis=0)
    mu0 = np.mean(X[I0_local, :], axis=0)
    d1 = np.sum((q - mu1) ** 2)
    d0 = np.sum((q - mu0) ** 2)
    return np.exp(-d1) / (np.exp(-d1) + np.exp(-d0))
```

```
def QDA(Xtr, Ytr, q):
    n = Xtr.shape[1]
    Ntr = Xtr.shape[0]
    I0_local = np.where(Ytr == 0)[0]
    I1_local = np.where(Ytr == 1)[0]
```

(continues on next page)

(continued from previous page)

```

P0 = len(I0_local) / Ntr
P1 = 1 - P0
mu0 = np.mean(Xtr[I0_local, :], axis=0)
Sigma0 = np.cov(Xtr[I0_local, :], rowvar=False)
mu1 = np.mean(Xtr[I1_local, :], axis=0)
Sigma1 = np.cov(Xtr[I1_local, :], rowvar=False)
xts = np.array([q])
diff0 = xts - mu0
inv_Sigma0 = np.linalg.inv(Sigma0)
quad0 = diff0.dot(inv_Sigma0).dot(diff0.T)[0, 0]
g0 = -0.5 * quad0 - n/2 * np.log(2 * np.pi) - 0.5 * np.log(np.linalg.det(Sigma0))
↪+ np.log(P0)
diff1 = xts - mu1
inv_Sigma1 = np.linalg.inv(Sigma1)
quad1 = diff1.dot(inv_Sigma1).dot(diff1.T)[0, 0]
g1 = -0.5 * quad1 - n/2 * np.log(2 * np.pi) - 0.5 * np.log(np.linalg.det(Sigma1))
↪+ np.log(P1)
return np.exp(g1) / (np.exp(g0) + np.exp(g1))

```

```

def LDA(Xtr, Ytr, q):
    n = Xtr.shape[1]
    Ntr = Xtr.shape[0]
    I0_local = np.where(Ytr == 0)[0]
    I1_local = np.where(Ytr == 1)[0]
    P0 = len(I0_local) / Ntr
    P1 = 1 - P0
    sigma2 = np.mean(np.var(Xtr, axis=0, ddof=1))
    mu0 = np.mean(Xtr[I0_local, :], axis=0)
    mu1 = np.mean(Xtr[I1_local, :], axis=0)
    g0 = -1/(2*sigma2) * np.linalg.norm(q-mu0) + np.log(P0)
    g1 = -1/(2*sigma2) * np.linalg.norm(q-mu1) + np.log(P1)
    return np.exp(g1) / (np.exp(g0) + np.exp(g1))

```

8.9 Unequal costs

When the cost associated with False Negatives and False Positive are not equal, the computation cannot be used as is. Let us define two costs: L_{FP} and L_{FN} . Keep in mind, here, that we consider the two other costs $L_{TP}, L_{TN} = 0$. The expected cost for each possible decision is $L(1|x) = P(\mathbf{y} = 0|x)L_{FP}$ and $L(0|x) = P(\mathbf{y} = 1|x)L_{FN}$. The two *a posteriori* probabilities $P(y|x)$ can be estimated from the data. If we have $P(\mathbf{y} = 1|x)$, for a sample x , we want to assign the class that minimizes the expected cost.

We predict

$$\hat{y} = \begin{cases} 1, & \text{if } L(1|x) < L(0|x) \Leftrightarrow (1 - P(y = 1|x))L_{FP} < P(y = 1|x)L_{FN}, \\ 0, & \text{otherwise.} \end{cases}$$

We can rewrite this as

$$P(\mathbf{y} = 1|x) > \frac{L_{FN}}{L_{FP} + L_{FN}}$$

- If $L_{FP} = L_{FN}$, then $\frac{L_{FN}}{L_{FP} + L_{FN}} = \frac{1}{2}$. This is the classic case, where we output the maximum a posteriori class.
- If $L_{FP} > L_{FN}$, then it is better to output $y = 1$, this reduces the number of FP.
- Otherwise, it is better to output $y = 0$.

8.9.1 Exercise

- In the generated data, suppose that having a FN has a cost of 1 while having a FP has a cost of 0.2. What is the threshold and the optimal class ?

The threshold is $\frac{1}{1.2} = 0.833$. The optimal class is therefore the class 0.

CONCLUSION

Throughout this course, you've taken a deep dive into the **statistical and algorithmic foundations** of machine learning. Starting with probabilistic methods and Monte Carlo simulations, you've built up your understanding step by step:

- You learned how to simulate data and use probabilistic tools to model uncertainty.
- You developed **linear models** and extended them using regularization and basis transformations.
- You explored **tree-based models**, pipelines, and saw the importance of preprocessing and feature engineering.
- You implemented **neural networks** for regression tasks and trained them using PyTorch.
- You investigated **ensemble methods**, **feature selection**, and how to combine multiple models.
- Finally, you tackled **classification problems**, including model evaluation using precision, recall, and ROC curves.

What you've acquired isn't just a toolkit of models, it's a **methodology**. The real power lies in knowing **how to ask the right questions** and **choose the right tools** for the task at hand.

9.1 A Practical Checklist: Building a Machine Learning Pipeline

Whenever you're faced with a new problem in the real world, keep these questions in mind. They form a structured approach for building and evaluating any ML model:

9.1.1 Understanding the Task

- **Is this a classification or regression problem?**
 - If it's classification:
 - * How many classes are there?
 - * Is it a binary or multiclass problem? If multiclass, should we use one-vs-all or another strategy?
 - If it's regression:
 - * Should the output be normalized?

9.1.2 Understanding the Inputs

- **How many features are there?**
- **Can/should we normalize them?**
- **Are some features categorical?**
 - If yes, can we encode them (e.g., one-hot encoding, ordinal encoding)?
- **Can we reduce dimensionality?**
 - How can we detect and discard useless or redundant features?

9.1.3 Exploring the Data

- **What is the size of the dataset?** (number of rows and columns)
- **What are the main characteristics of the data?**
 - Is there correlation or redundancy between features?
- **What does the data look like when plotted?**
 - What types of visualizations best reveal its structure?

9.1.4 Evaluating Models

- **What performance metrics are appropriate?**
 - Accuracy, MSE, precision, recall, ROC AUC, etc.
- **How do we assess model quality?**
 - Visual inspection of residuals?
 - Cross-validation scores?
- **What does the bias/variance tradeoff look like for our model?**
- **Which model is most justifiable and interpretable for the task?**

9.1.5 Thinking Critically

- **How do we validate the model's generalization capacity?**
 - Do we have a test set or cross-validation strategy?
- **What assumptions are we making by choosing this model?**
 - Linearity? Independence? Normality?
- **Is there a simpler solution that performs just as well?**
 - Occam's razor applies here too.
- **What is the cost of being wrong?**
 - Especially important in medical, legal, or financial applications.

9.2 Final Words

Machine learning is not just about applying models: it's about **thinking critically**, **analyzing data**, and **selecting tools** wisely. This course has given you the statistical backbone to understand what's happening *under the hood*, and the programming practice to start applying it to real problems.

Your next steps: dive deeper into advanced topics like unsupervised learning, probabilistic graphical models, or deep learning. But always return to the foundations when things get unclear: they're your compass.

And remember: **models may change, but methodology stays.**

BIBLIOGRAPHY